

Ingeniería Informática
2024-2025

Trabajo Fin de Grado

“ Desarrollo de un simulador de
sistemas de ficheros para entornos
distribuidos y paralelos ”

Manuel Gómez-Plana Rodríguez

Tutor

Félix García Carballeira
Leganés, Madrid, España
Septiembre 2025



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

En los últimos años, en el mundo del desarrollo de software ha existido una tendencia de desacoplar los componentes que conforman una aplicación con el objetivo de proporcionar una solución flexible, escalable y mantenible. Las arquitecturas distribuidas se han establecido como el modelo predominante en paradigmas como el mundo IoT, las aplicaciones cloud o las redes sociales. Adicionalmente, existe una responsabilidad latente en los desarrolladores de software que les encomienda a realizar un código óptimo, no sólo por cuestiones temporales, sino que también por el interés en el ahorro de recursos energéticos. Motivado por estos dos puntos, este proyecto surge con el objetivo de crear un simulador orientado a aplicaciones distribuidas, que permita medir con precisión el tiempo y la energía consumida por los actores que conforman dicha aplicación.

La simulación consta de un modelo cliente-servidor, donde los servidores exponen un sistema de ficheros accesible por los clientes, que pueden enviar las operaciones deseadas que se harán sobre un sistema de ficheros simulado. Este simulador permite también definir la topología de la red que conecta a los clientes con los servidores, así como las características de las máquinas que contiene a cada actor. Como resultado de la simulación, se podrán observar unas gráficas generadas automáticamente que permiten visualizar diferentes métricas temporales y energéticas.

La implementación del simulador se llevó a cabo usando el framework de simulación *SimGrid* y su interfaz *S4U* definida en C++. Otros lenguajes de programación, como *Python*; y otras herramientas, como *Gnuplot*, se usan para proveer de más funcionalidades al sistema final, construyendo así una aplicación que cubre la configuración, ejecución y visualización de los resultados de la simulación.

Este documento contiene una especificación de todo el proyecto, que abarca desde una descripción extensa de los objetivos y la motivación, hasta la implementación del sistema final con sus respectivas pruebas de validación y verificación, pasando por varios capítulos esenciales que tratan el estado del arte y discurren sobre las decisiones de diseño tomadas, entre otros.

Esta memoria concluye con un análisis de los objetivos planteados inicialmente, y con unos pensamientos finales sobre el proceso de desarrollo, seguido de un listado de proyectos futuros a realizar sobre esta herramienta para ampliar sus funcionalidades.

Palabras clave: Simulación - SimGrid - Computación Distribuida - Computación Paralela - Sistema de Ficheros

DEDICATORIA

Este trabajo ha sido posible gracias a la ayuda de muchas personas. Quiero agradecer a mi tutor, Félix García Carballeira, que no sólo me ha propuesto la idea de este TFG, sino que también me ha ayudado enormemente a lo largo de este, resolviendo mis dudas y proponiéndome nuevas ideas para mejorar este proyecto.

Quiero agradecer a todas las personas que he conocido a lo largo de la carrera, especialmente a mi grupo de amigos, “Pan con Pan”. Con ellos, he comenzado y he acabado la carrera y ahora que estoy escribiendo esto, no podría imaginármela sin ellos. Me han animado, me han retado, me he reído con ellos y también me he enfadado, pero, lo que es más importante, ellos me han mostrado un constante referente al que aspirar, no sólo como ingeniero, sino como persona.

En especial, quiero agradecer a mi pareja, que me da fuerzas todos los días y me apoya a seguir esforzándome, a seguir trabajando y a no darme por vencido cuando las cosas van mal. Sin su apoyo, cercanía y cariño, dudo que esta etapa universitaria hubiera sido ni una parte de lo acogedora y alegre de lo que deseaba.

Quiero agradecer, finalmente, a mi familia. Quiero agradecer a mis abuelos, los cuáles han mirado por mí toda mi vida y se que todos están o hubiesen estado muy orgullosos de mí. Quiero agradecer a mi hermana que, a pesar de nuestros roces, se que quiere lo mejor para mí y me pica para ser la mejor versión de mí. Por último, pero no por ello menos importante, quiero agradecer a mis padres que me han provisto con una educación y unos valores, un cariño incondicional y un apoyo inquebrantable, y que me han ayudado en todas las etapas de mi vida para ser quien soy. Sin ellos, no podría haber llegado a donde me enorgullezco de estar a día de hoy.

Gracias a todos.

ÍNDICE GENERAL

1. INTRODUCCIÓN.	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Estructura del documento	2
2. ESTADO DEL ARTE.	5
2.1. Arquitecturas	5
2.1.1. Arquitecturas monolíticas	5
2.1.2. Arquitecturas distribuidas	7
2.1.3. Comparación entre las arquitecturas	10
2.1.4. Conclusión de las arquitecturas	10
2.2. Simulación	11
2.2.1. Definición.	11
2.2.2. Contexto histórico	11
2.2.3. Simulación vs Emulación	13
2.2.4. Retos de los simuladores	14
2.3. Simuladores.	15
2.3.1. SimGrid.	15
2.3.2. Simuladores relacionados	15
2.3.3. Modelos de simulación	17
3. ANÁLISIS	21
3.1. Propuesta	21
3.2. Requisitos.	21
3.2.1. Requisitos de usuarios	22
3.2.2. Requisitos de sistema	32
3.3. Casos de uso	44
3.4. Trazabilidad.	50

4. DISEÑO	55
4.1. Tecnologías empleadas	55
4.1.1. SimGrid	55
4.1.2. Gnuplot	56
4.1.3. Lenguajes de programación	58
4.2. Modelo cliente-servidor	58
4.2.1. Simulación pequeña	59
4.2.2. Simulación clusterizada	59
4.3. Plataforma y acciones en la simulación	61
4.4. Políticas	62
4.4.1. Política secuencial	62
4.4.2. Política de replicación	62
4.4.3. Expand	63
4.5. Operaciones.	63
4.5.1. Crear un fichero	64
4.5.2. Leer un fichero	64
4.5.3. Escribir un fichero	64
4.5.4. Eliminar un fichero.	65
4.5.5. Simular cómputo	65
4.5.6. Ficheros de operaciones	65
4.6. Interfaces	66
5. IMPLEMENTACIÓN Y DESPLIEGUE	67
5.1. Estructura del proyecto	67
5.2. Servidores.	69
5.2.1. Servidor en los ficheros	69
5.2.2. Núcleo del servidor	72
5.2.3. Operaciones.	74
5.2.4. Medición de tiempo y energía.	77
5.3. Clientes	79
5.3.1. Cliente en los ficheros	79
5.3.2. Núcleo del cliente	81

5.3.3. Políticas	82
5.4. Integración con Gnuplot	87
5.5. Interfaces	89
5.5.1. Interfaces finales	89
5.5.2. Programación de las interfaces	90
5.5.3. Diseño de Interfaces	93
5.6. Despliegue	96
5.6.1. Dependencias	96
5.6.2. Ejecución	96
6. VALIDACIÓN Y VERIFICACIÓN	98
6.1. Pruebas de verificación	98
6.2. Pruebas de validación	106
6.3. Prueba completa	111
6.3.1. Caso de uso	111
6.3.2. Benchmark IOR	111
6.3.3. Configuración de la simulación	113
6.3.4. Resultados de la prueba completa	116
6.3.5. Prueba escalada	118
7. PLANIFICACIÓN Y PRESUPUESTO	121
7.1. Metodología	121
7.2. Estimación temporal	122
7.3. Presupuesto	124
7.3.1. Costes directos	124
7.3.2. Costes indirectos	125
7.3.3. Costes totales	126
7.3.4. Oferta	126
8. MARCO REGULATORIO Y ENTORNO SOCIO-ECONÓMICO	128
8.1. Marco regulatorio	128
8.1.1. Licencias	128
8.1.2. Leyes aplicables	128
8.2. Entorno socio-económico	129

9. CONCLUSIONES Y TRABAJOS FUTUROS	131
9.1. Conclusiones	131
9.1.1. Conclusiones del proyecto.	131
9.1.2. Conclusiones personales.	131
9.2. Trabajos futuros	132
BIBLIOGRAFÍA	134

ÍNDICE DE FIGURAS

2.1	Componentes de una aplicación monolítica	6
3.1	Diagrama de casos de uso	44
4.1	Simulación pequeña	59
4.2	Simulación clusterizada	60
4.3	Comunicación de los clusters	60
4.4	Ejemplo de la política de replicación	62
4.5	Ejemplo de la política Expand	63
4.6	Diagrama de flujo de CREATE	64
4.7	Diagrama de flujo de READ	64
4.8	Diagrama de flujo de WRITE	64
4.9	Diagrama de flujo de DELETE	65
4.10	Diagrama de flujo de COMPUTE_TIME	65
5.1	Diagrama de flujo del servidor	73
5.2	Llamadas a las operaciones	74
5.3	Llamadas a las políticas de acceso	82
5.4	Ejemplo de tiempo por servidor	88
5.5	Ejemplo de energía por servidor	88
5.6	Ejemplo de tiempo por operaciones por servidor	89
6.1	Resultados de tiempo por servidor	116
6.2	Resultados de energía por servidor	117
6.3	Resultados de tiempo por operaciones por servidor	117
6.4	Resultados de tiempo por servidor en la prueba escalada	118
6.5	Resultados de energía por servidor en la prueba escalada	119
6.6	Resultados de tiempo por operaciones por servidor en la prueba escalada	119
7.1	Diagrama de Gantt del proyecto	123

ÍNDICE DE TABLAS

2.1	Comparación de Arquitecturas	10
2.2	Simulación vs Emulación	13
2.3	Comparación de simuladores	17
3.1	Plantilla para los requisitos de usuario	23
3.2	Requisito RU_CA_01	23
3.3	Requisito RU_CA_02	23
3.4	Requisito RU_CA_03	24
3.5	Requisito RU_CA_04	24
3.6	Requisito RU_CA_05	24
3.7	Requisito RU_CA_06	24
3.8	Requisito RU_CA_07	25
3.9	Requisito RU_CA_08	25
3.10	Requisito RU_CA_09	25
3.11	Requisito RU_CA_10	26
3.12	Requisito RU_CA_11	26
3.13	Requisito RU_CA_12	26
3.14	Requisito RU_CA_13	27
3.15	Requisito RU_CA_14	27
3.16	Requisito RU_CA_15	27
3.17	Requisito RU_CA_16	28
3.18	Requisito RU_CA_17	28
3.19	Requisito RU_CA_18	28
3.20	Requisito RU_CA_19	28
3.21	Requisito RU_CA_20	29
3.22	Requisito RU_CA_21	29
3.23	Requisito RU_RE_01	29
3.24	Requisito RU_RE_02	30

3.25 Requisito RU_RE_03	30
3.26 Requisito RU_RE_04	30
3.27 Requisito RU_RE_05	31
3.28 Requisito RU_RE_06	31
3.29 Plantilla para los requisitos de sistema	32
3.30 Requisito RS_FN_01	33
3.31 Requisito RS_FN_02	33
3.32 Requisito RS_FN_03	33
3.33 Requisito RS_FN_04	34
3.34 Requisito RS_FN_05	34
3.35 Requisito RS_FN_06	34
3.36 Requisito RS_FN_07	35
3.37 Requisito RS_FN_08	35
3.38 Requisito RS_FN_09	35
3.39 Requisito RS_FN_10	36
3.40 Requisito RS_FN_11	36
3.41 Requisito RS_FN_12	36
3.42 Requisito RS_FN_13	37
3.43 Requisito RS_FN_14	37
3.44 Requisito RS_FN_15	37
3.45 Requisito RS_FN_16	38
3.46 Requisito RS_FN_17	38
3.47 Requisito RS_FN_18	38
3.48 Requisito RS_FN_19	39
3.49 Requisito RS_FN_20	39
3.50 Requisito RS_FN_21	39
3.51 Requisito RS_FN_22	40
3.52 Requisito RS_NF_01	40
3.53 Requisito RS_NF_02	40
3.54 Requisito RS_NF_03	41
3.55 Requisito RS_NF_04	41

3.56	Requisito RS_NF_05	41
3.57	Requisito RS_NF_06	42
3.58	Requisito RS_NF_07	42
3.59	Requisito RS_NF_08	42
3.60	Requisito RS_NF_09	43
3.61	Requisito RS_NF_10	43
3.62	Requisito RS_NF_11	43
3.63	Plantilla de los casos de uso	45
3.64	Caso de Uso CU1	46
3.65	Caso de Uso CU1.1	46
3.66	Caso de Uso CU1.2	47
3.67	Caso de Uso CU1.3	47
3.68	Caso de Uso CU2	48
3.69	Caso de Uso CU3	48
3.70	Caso de Uso CU3.1	49
3.71	Caso de Uso CU4	49
3.72	Caso de Uso CU4.1	49
3.73	Caso de Uso CU4.2	50
3.74	Matriz de Trazabilidad de Requisitos de usuario de capacidades	51
3.75	Matriz de Trazabilidad de Requisitos de usuario de restricciones	51
3.76	Matriz de Trazabilidad de Requisitos de sistema funcionales	52
3.77	Matriz de Trazabilidad de Requisitos de sistema no funcionales	53
4.1	Herramientas de graficado	57
5.1	Elementos comunes de un servidor en la plataforma	70
5.2	Elementos de un cluster de servidores en la plataforma	71
5.3	Argumentos de la política Expand	84
5.4	Interfaces implementadas	90
5.5	Heurísticas de Nielsen en las interfaces	94
5.6	Principios de diseño en las interfaces	95
6.1	Plantilla para las pruebas	98

6.2	Requisitos de sistema excluidos de las pruebas de verificación	99
6.3	Prueba VER-01	100
6.4	Prueba VER-02	100
6.5	Prueba VER-03	101
6.6	Prueba VER-04	101
6.7	Prueba VER-05	102
6.8	Prueba VER-06	103
6.9	Prueba VER-07	103
6.10	Prueba VER-08	104
6.11	Prueba VER-09	104
6.12	Prueba VER-10	105
6.13	Prueba VER-11	105
6.14	Matriz de Trazabilidad de las Pruebas de Verificación	106
6.15	Prueba VAL-01	107
6.16	Prueba VAL-02	108
6.17	Prueba VAL-03	108
6.18	Prueba VAL-04	109
6.19	Matriz de Trazabilidad de las Pruebas de Validación	110
6.20	Resultados del benchmark <i>IOR</i>	112
7.1	Coste del personal	124
7.2	Coste del equipamiento	125
7.3	Información del proyecto	126
7.4	Oferta final	126

1. INTRODUCCIÓN

Este apartado contiene una introducción al proyecto que describe la motivación que ha llevado a hacer el mismo, el planteamiento de los objetivos que se pretenden alcanzar y una breve descripción de la estructura de este documento.

1.1. Motivación

Este trabajo surge de varios puntos de motivación cuya naturaleza es acoplada. El primer punto de motivación es el creciente interés y la proliferación de las arquitecturas distribuidas en los últimos años. A diferencia de las arquitecturas monolíticas sobre las que se construían tradicionalmente el software, las arquitecturas distribuidas ofrecen una serie de ventajas como la escalabilidad, la replicabilidad, la mantenibilidad a la hora de desarrollar código, la flexibilidad de los componentes... [1] Debido a estas fortalezas que permiten un desarrollo y un despliegue mantenible, muchos sistemas de diferentes naturalezas y ámbitos, como las redes de entrega de contenido (CDN), los comercios on-line o las redes sociales, han optado por el uso de arquitecturas distribuidas [2].

El segundo punto de motivación es el interés por la gestión de los recursos desde el punto de vista del despliegue de aplicaciones, concretamente, el consumo de energía y de tiempo de una aplicación, ya que un objetivo de cualquier software es lograr el mayor rendimiento posible. En un mundo distribuido donde las aplicaciones se despliegan en distintos nodos interconectados entre sí, el número de máquinas sobre las que se despliegue el software es un factor crítico que afecta directamente al consumo energético de estas máquinas. Además, este consumo es influido por diversos factores difíciles de controlar, siendo uno de ellos el tiempo que estas máquinas están activas, ya sea realizando tareas de cómputo o de lectura y escritura de ficheros.

Este TFG se ha enmarcado dentro del contexto del proyecto investigación “High Performance Storage Systems for HPC and Big Data (Expand)” financiado por la Agencia Estatal de Investigación.

1.2. Objetivos

El objetivo de este proyecto, alineado con los puntos de motivación anteriormente expuestos, es el desarrollo de un simulador que permita medir con suficiente precisión el consumo temporal y energético de los servidores desplegados en una plataforma para una aplicación distribuida, en este caso, un sistema de ficheros distribuido que usa un patrón cliente-servidor. Este sistema de simulación será desarrollado con el framework de simulación *SimGrid* por razones que se explican en el apartado [4.1.1. SimGrid](#).

Como el objetivo es el desarrollo de un simulador que pueda aportar información sobre la energía y el tiempo de ejecución que consumen un grupo de servidores, este proyecto tiene el objetivo adicional de presentar la información mediante unos gráficos generados automáticamente tras finalizar la simulación. Esta funcionalidad se implementará con la herramienta *Gnuplot*, elegida por razones que se explican en el apartado [4.1.2. Gnuplot](#).

Granularmente, se definen los siguientes objetivos y sub-objetivos:

- **Obj_1:** Dar al usuario la capacidad de definir una plataforma con servidores y clientes interconectados entre sí.
 - **Obj_1.1:** Dar al usuario la capacidad de definir parámetros de los distintos actores y hosts que conforman la simulación.
 - **Obj_1.2:** Dar al usuario la capacidad de definir rutas de una red que conecten a los clientes con los servidores.
- **Obj_2:** Dar al usuario la capacidad de definir operaciones de: creación, lectura, escritura y eliminado de ficheros (CRUD); que los clientes puedan enviar a los servidores.
- **Obj_3:** Desarrollar un código que permita la implementación de una comunicación segura y paralela entre los clientes y los servidores.
 - **Obj_3.1:** Permitir que un servidor pueda recibir operaciones de varios clientes.
- **Obj_4:** Medir con una precisión razonable el tiempo y la energía que los servidores consumen a lo largo de la simulación.
- **Obj_5:** Mostrar el consumo energético y temporal de los servidores al finalizar la simulación.
- **Obj_6:** Generar unas gráficas al finalizar la simulación que muestren los datos de consumo energético y temporal.

1.3. Estructura del documento

El documento se estructura en los siguientes capítulos:

- *Capítulo 1. Introducción:* este capítulo sirve como una breve introducción del proyecto, donde se presentan la motivación que ha llevado a la realización del mismo y los objetivos que se plantean.
- *Capítulo 2. Estado del arte:* este capítulo trata sobre el estado de la tecnología relacionada con el proyecto como: el uso de arquitecturas distribuidas, la simulación y los simuladores.

- *Capítulo 3. Análisis:* este capítulo contiene un resumen de la propuesta, seguido de la elicitación de los requisitos y la definición los casos de uso relacionados con esta herramienta, finalizando este punto con las matrices de trazabilidad entre los requisitos y los casos de uso.
- *Capítulo 4. Diseño:* este capítulo contiene una justificación a nivel de diseño de las herramientas empleadas, así como una explicación tanto de la arquitectura de la simulación como de la estructura de los ficheros del proyecto.
- *Capítulo 5. Implementación y despliegue:* este capítulo contiene una explicación del código de los distintos módulos que conforman la aplicación final, así como una explicación para desplegar o ejecutar la simulación.
- *Capítulo 6. Validación y verificación:* este capítulo contiene las pruebas necesarias para validar y verificar la simulación contra los objetivos, los requisitos y los casos de uso.
- *Capítulo 7. Planificación y presupuesto:* este capítulo contiene un justificación de la metodología usada en este proyecto, así como una estimación del tiempo necesario para realizarlo y un desglose del presupuesto que concluye en una oferta final.
- *Capítulo 8. Marco regulatorio y entorno socio-económico:* este capítulo habla sobre las distintas licencias necesarias y leyes aplicables a este proyecto, finalizando con un análisis del impacto a nivel socio-económico de este proyecto.
- *Capítulo 9. Conclusiones y trabajos futuros:* este capítulo cierra la memoria del trabajo con unas conclusiones sobre el simulador y un apartado donde se discuten trabajos futuros relacionados con el mismo.

2. ESTADO DEL ARTE

Este capítulo trata sobre el estado actual de las distintas herramientas y tecnologías relacionadas con este proyecto. Primero, se explora la diferencia entre las arquitecturas monolíticas y distribuidas. Después, se habla de los simuladores, proporcionando una definición clara y exponiendo sus diferencias con los emuladores. Por último, este capítulo cierra hablando de los simuladores, introduciendo el framework de simulación *SimGrid* y realizando una comparativa entre una variedad de herramientas de simulación.

2.1. Arquitecturas

El primer tema a tratar son las arquitecturas sobre las que se construye el software. Convencionalmente, se hace una distinción entre dos tipos principales de arquitecturas: las arquitecturas monolíticas y las arquitecturas distribuidas. Este apartado contiene un análisis de los dos tipos de arquitecturas, concluyendo con una comparación entre ambas.

2.1.1. Arquitecturas monolíticas

La primera arquitectura a tratar es la arquitectura monolítica. Este apartado se divide en cinco subapartados: definición de la arquitectura monolítica, contexto histórico, componentes que suelen formar parte de una aplicación monolítica, características y casos de uso de esta arquitectura.

Definición

La arquitectura monolítica se define como “un modelo de software tradicional mediante el cuál una base de código maneja varias funciones de negocio” [3]. Se entiende también la arquitectura monolítica como “un modelo tradicional de un programa de software, que es construido como una unidad auto-contenida e independiente de otras aplicaciones” [4].

En otras palabras, la arquitectura monolítica se puede ver como un enfoque tradicional por el que todo el código de una aplicación, la cuál cuenta con varias responsabilidades de naturaleza no necesariamente similar, se desarrolla de forma altamente acoplada. Es por ello que este concepto no hace sólo referencia a la forma de desplegar el código, sino que también hace referencia a la forma en la que se desarrolla el código.

Desde un punto de vista de despliegue, las aplicaciones que siguen este tipo de patrón suelen ser desplegadas como una unidad [5], independientemente de si esta unidad se trata de un ejecutable, un contenedor de *Docker*, una máquina virtual...

Contexto histórico

Las arquitecturas monolíticas fueron de los primeros modelos de desarrollo de software creados. No es posible acreditar a una persona con la invención de este modelo, pero varias empresas que desarrollaban software en las décadas de 1960 y de 1970, como puede ser IBM, definitivamente influyeron en los arquetipos de desarrollo de software [1].

Los programas que se desarrollaron en los primeros años de la computación eran aplicaciones pequeñas y autocontenidas, generalmente usadas para el cálculo numérico [6]. Estas aplicaciones distan del complejo software que se desarrolla hoy en día, lleno de interconexiones entre módulos a través de la red. Es por ello que esta primera aplicación de los ordenadores influyó en la creación de programas que seguían un patrón de arquitecturas monolíticas.

Componentes

Las aplicaciones que siguen este patrón normalmente contienen los siguientes componentes [3], si bien siempre pueden ser adaptados a las necesidades de la aplicación:

- **Interfaz de usuario:** que maneja lo que ve el usuario, ya sea texto, imágenes, vídeos...
- **Base de datos:** normalmente suelen ser bases de datos relacionales que cuentan con varias tablas.
- **Aplicación del lado del servidor:** que maneja los recursos del servidor o accede a la base de datos.

Gráficamente, los componentes interactúan siguiendo el esquema de la figura 2.1:

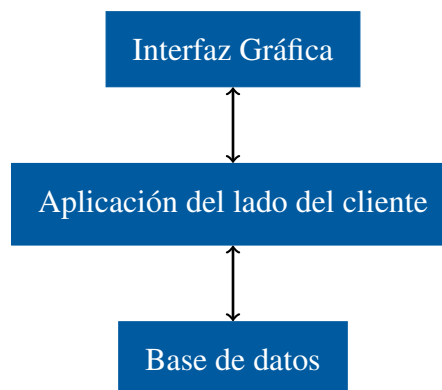


Fig. 2.1. Componentes de una aplicación monolítica

Características

Las arquitecturas monolíticas cuentan con las siguientes ventajas [3]:

- Facilidad en el desarrollo de aplicaciones.
- Despliegue sencillo, ya que se despliegan como unidades de software.
- Facilidad de mantenimiento, ya que no cuentan con una gran lista de componentes.
- Seguridad mejorada al ser un sistema cerrado en el que el procesamiento de datos está totalmente contenido.

A pesar de estas ventajas, las arquitecturas monolíticas presentan una serie de desventajas o inconvenientes [3]:

- Resistencia a la introducción de nuevas tecnologías, ya que las aplicaciones están altamente acopladas y la introducción de una nueva tecnología puede causar problemas de integración o de compatibilidad.
- Escalabilidad reducida, ya que cualquier necesidad de escalado, por pequeña que sea, puede implicar una reconstrucción de la aplicación, con un despliegue más problemático.

Es por ello que, a pesar de contar con facilidades de desarrollo y despliegue a un nivel reducido, un sistema escalable nunca seguirá el patrón de arquitectura monolítica.

Casos de uso

Las aplicaciones monolíticas, de acuerdo con sus ventajas y sus limitaciones, se usan en escenarios donde el código no es muy complejo, pero donde el rendimiento es clave [1]. Algunos casos de uso comunes incluyen aplicaciones de empresa, como la planificación de recursos de las empresas (ERP, Enterprise Resource Planning); o aplicaciones de escritorio que se ejecutan en una sola máquina, como software de procesamiento de palabras. Adicionalmente, algunas aplicaciones web tempranas usaban modelos monolíticos.

2.1.2. Arquitecturas distribuidas

Tras haber explorado las arquitecturas monolíticas, es necesario explicar las arquitecturas distribuidas. Primero, se proporcionará una definición clara, tras lo que se explicará el contexto histórico de estas arquitecturas. Este apartado continúa con un listado y explicación de varios tipos de arquitecturas distribuidas, finalizando mediante la exposición de algunos de los casos de uso más comunes para este tipo de arquitectura.

Definición

Las arquitecturas distribuidas se definen como “un modelo donde componentes de software que residen en ordenadores se comunican y coordinan sus acciones para conseguir un objetivo común” [7]. Esto se opone completamente a la noción en la que se basan las arquitecturas monolíticas, ya que los componentes no tienen una misma base de código y no se despliegan en la misma máquina bajo la misma unidad de despliegue, características intrínsecas al modelo monolítico.

Así, ya se encuentran las primeras diferencias, las arquitecturas distribuidas están pensadas para ser ejecutadas en máquinas diferentes de manera modular. El propio adjetivo “distribuido” avisa sobre esta naturaleza, una aplicación no es algo que pertenezca a una máquina, es algo que se encuentra distribuido entre varias, lo cual escapa de la centralización de recursos y computación en la que se basa el modelo monolítico.

Con todo esto, se puede decir que una aplicación sigue una arquitectura distribuida si el código de la misma se encuentra dividido en varios componentes, cada uno con su respectiva función de negocio, y que se ejecutan en máquinas distintas.

Contexto histórico

La computación distribuida surgió alrededor de las décadas de 1960 y 1970, y nació de la mano de la compartición de recursos. La compartición de recursos es un término que hace referencia a cuando una máquina “comparte” uno o varios de sus recursos con otra, la cual puede acceder de manera remota a dichos recursos como si fuesen propios [8], con el fin de completar una tarea que, de otra manera, sería demasiado costosa como para realizarla en una sola máquina.

Los sistemas distribuidos se originaron debido a esta necesidad de compartición de recursos a lo que se juntó el desarrollo de las primeras redes de ordenadores, como ARPANET, un proyecto que habilitó la comunicación entre los ordenadores mediante la conmutación de paquetes [9].

A pesar de este desarrollo temprano, no fue hasta el final de la década de 1990 y el principio de la década de 2000 que se comenzó a cambiar al desarrollo de software con arquitecturas distribuidas. Los principales impulsores de este cambio fueron: la necesidad de una mayor escalabilidad y flexibilidad, la rápida atención que recibió Internet en sus primeros años y el surgimiento masivo de aplicaciones basadas en web que demostraron que las aplicaciones monolíticas tenían muchos limitantes [1].

Distintos tipos de arquitecturas distribuidas, tales como SOA (Service-Oriented Architecture) o la arquitectura de microservicios, surgieron en torno a las décadas de 2000 y de 2010, ganando una rápida fama y adopción, siendo usadas por compañías pioneras como Amazon [1].

Sub-Arquitecturas

Tal y cómo se ha mencionado anteriormente, dentro de la noción de las arquitecturas distribuidas existen varias “sub-arquitecturas”, como son la arquitectura orientada a servicios (SOA) y la arquitectura de microservicios. Es conveniente, por lo tanto, explicar en qué consisten este par de arquitecturas.

La arquitectura SOA es una manera de diseñar software que permite reusar los elementos usando las interfaces que cada servicio tiene a través de una red con un lenguaje común [10]. Un servicio se define como una unidad autónoma diseñada para realizar una tarea específica, un concepto similar a los módulos mencionados anteriormente. A este servicio se puede acceder remotamente, y se puede actualizar de forma independiente, ambas cualidades de un módulo de una arquitectura distribuida.

Esta arquitectura, pese a sus beneficios, no es tan dominante como lo fue desde que los microservicios se comenzaron a usar. Esto se debe principalmente a que SOA funciona bien para aplicaciones grandes, pero lograr la escalabilidad para aplicaciones pequeñas y específicas es más complicado [11]. Es ahí donde entra en juego la arquitectura de microservicios.

La arquitectura de microservicios se basa en granularizar al máximo las funciones de los servicios de la arquitectura SOA, proporcionando así programas pequeños (respecto al alcance del código, no respecto a su complejidad o tamaño) que sean desarrollados, desplegados y escalados de manera independiente [12].

Esta arquitectura prolifera en una gran variedad de campos, como la computación en la nube, los dispositivos IoT, la computación en el edge... Sin embargo, a pesar de ser una de las arquitecturas que más ha proliferado en los últimos años, tiene una serie de desventajas [12]:

- La gestión de los microservicios, ya que la complejidad de la gestión escala con el número de microservicios.
- Sobrecoste debido a la comunicación, ya que la mayoría de los microservicios realizan llamadas sobre las interfaces de programación de la aplicación (API) de los otros microservicios, incurriendo en un sobrecoste debido a la comunicación a través de la red. Este sobrecoste causa una mayor latencia en el tiempo de respuesta de estos microservicios.
- Aumento en los costes de mantenimiento respecto a las arquitecturas monolíticas tradicionales.
- Complejidad aumentada en el despliegue.

Estas desventajas pueden jugar un papel clave a la hora de elegir qué arquitectura usar, ya que, si el número de servicios es reducido, los sobrecostes intrínsecos a las arquitecturas distribuidas pueden resultar demasiado ineficientes.

Casos de uso

Las arquitecturas distribuidas pueden ser usadas en una gran variedad de escenarios, como aplicaciones web de alta escala, aplicaciones cloud nativas [1], plataformas IoT, computación en nodos edge, redes sociales... Estos casos de uso demuestran el potencial que tiene la separación de intereses y la compartición de recursos si se usan en los ámbitos adecuados.

2.1.3. Comparación entre las arquitecturas

Tras haber explicado las distintas arquitecturas y las ventajas que ofrece cada una, se ofrece la tabla 2.1 como resumen para comparar las diferencias entre una y otra.

TABLA 2.1.
COMPARACIÓN DE ARQUITECTURAS

Monolítica	Distribuida
Código ejecutado como una unidad	Código dividido en componentes
No hay necesidad de comunicación	Los componentes se han de comunicar a través de la red
Código fácil de desarrollar y desplegar	Desarrollo y despliegue de cada componente algo más difícil
Código muy difícil de actualizar y mantener	Código fácilmente actualizable y mantenible
Poca escalabilidad y flexibilidad	Muy flexible y escalable
Recursos limitados a la máquina	Compartición de recursos

2.1.4. Conclusión de las arquitecturas

Tras todo lo expuesto en esta sección, se concluye que la elección de la arquitectura de la aplicación a desarrollar es totalmente dependiente de la naturaleza del programa. Sin embargo, es clara la tendencia actual, ya que las aplicaciones han evolucionado a un carácter distribuido debido a la necesidad de escalar los sistemas de producción y a la ventaja que ofrece la compartición de recursos y la separación de responsabilidades.

2.2. Simulación

Esta sección explora el estado actual del paradigma de las simulaciones. Para ello, primero, se definirá qué es una simulación, pasando luego por una breve introducción al contexto histórico de las simulaciones, continuando con las diferencias entre *simular* y *emular*, y concluyendo el apartado con los retos que afrontan los simuladores.

2.2.1. Definición

La simulación se define como “el uso de un ordenador para representar las respuestas dinámicas de un sistema mediante el comportamiento de otro sistema modelado a partir del primero” [13]. En otras palabras, una simulación se refiere al uso de un programa informático que abstrae un sistema para replicar el comportamiento de dicho sistema.

Los simuladores se emplean en una gran variedad de escenarios, ya sea para recrear un modelo que no existe en la naturaleza, simular un sistema que sería demasiado complejo o caro de construir, estudiar el comportamiento con unas situaciones que podrían ser difíciles de conseguir o situaciones que no son seguras de imitar, estudiar el comportamiento de un sistema al variar ciertos componentes [13]... Los simuladores son cruciales porque pueden estudiar un sistema en un entorno sin riesgos, artificial y variable según las necesidades del investigador.

Esta herramienta se extiende más allá de su uso para el estudio de sistemas, ya que también se puede utilizar para ciertos tipos de entrenamiento. Uno de los ejemplos más conocido de los últimos años es el famoso *Microsoft Flight Simulator*. Este simulador, reconocido por la *Sociedad Aeronáutica Real* como un simulador que “cambia las reglas del juego” [14], es usado por profesionales para mantener sus habilidades y familiarizarse con los sistemas aeronáuticos debido a su realismo.

El uso de los simuladores también desafía las barreras de los ámbitos que puede alcanzar. Desde simuladores numéricos que permiten calcular el impacto de una explosión nuclear [15], hasta la simulación usada en escuelas de negocio para que los estudiantes puedan aprender en un entorno seguro y real (adoptada actualmente por cerca de un 98 % de las escuelas estadounidenses acreditadas) [16], el uso de esta tecnología abarca una enorme cantidad de ámbitos.

2.2.2. Contexto histórico

El concepto de simulación es algo que se puede acuñar desde antes del nacimiento y desarrollo de los ordenadores. Al comienzo del siglo XX, el matemático y químico William Sealy Gosset publicó ciertos resultados estadísticos significativos bajo el alias “Student”, comenzando con un paper publicado en 1908 lo que eventualmente se convertiría en la distribución t de Student, una distribución de probabilidad usada para modelar

datos con un tamaño de muestra pequeño que siguen una distribución normal [17]. Gosset no contaba con resultados analíticos completos, por lo que usó una forma cruda de simulación manual que le permitiese verificar la forma exacta de la función de densidad de la probabilidad usada para la distribución [18].

El siguiente avance significativo en el campo de la simulación se dio en la década del 1940 debido a dos avances singulares. El primero fue la construcción de los primeros ordenadores eléctricos de uso general, siendo el ENIAC uno de los más famosos. Este primer hito posibilitó el segundo avance, la aplicación del método de Monte Carlo para resolver algunos de los problemas surgidos en el diseño de la bomba de hidrógeno [18] por parte de los Estados Unidos durante la Segunda Guerra Mundial. El método de Monte Carlo se convirtió así en una de las primeras simulaciones computadas en el ENIAC, uno de los predecesores de la computación moderna.

A partir de la década de 1950, cuando los ordenadores de uso común estaban cada vez más disponibles, las técnicas de simulación comenzaron a crecer y se empezó a ver su utilidad en otros campos más allá de la física. Fue alrededor de esos años cuando Keith Douglas Tocher desarrolló el Programa de Simulación General (GSP por sus siglas en inglés), un programa capaz de construir simulaciones de plantas industriales comprendidas por un conjunto de máquinas cuyo estado rotaba entre varios estados de disponibilidad [18], marcando así un nuevo hito en la historia de la simulación. Los méritos de Tocher no se quedaron ahí, ya que también contribuyó al paradigma de la simulación de varias formas, siendo una de las más importantes el diagrama de ciclos de actividad (ACD), un método para describir las interacciones entre objetos de un sistema [19] usado extensivamente en las enseñanzas relacionadas con la simulación en el Reino Unido [18].

Los proyectos subsiguientes relacionados con el campo de la simulación más relevantes fueron [18]:

- **Sistema de Simulación de Propósito General (GPSS):** desarrollado por IBM en la década del 1960, este programa fue diseñado para facilitar el modelado de simulaciones relacionadas con sistemas de telecomunicación complejos. La facilidad de uso hizo que el GPSS de IBM se convirtiera en uno de los lenguajes de instrucciones de simulación más usados en Estados Unidos.
- **SIMSCRIPT:** desarrollado en 1963 en *FORTRAN*, este programa estaba pensado para ser usado por usuarios que no fueran expertos en el uso de ordenadores.
- **SIMULA:** un proyecto creado con apoyo de Univac, SIMULA I se desarrolló como una extensión del ALGOL 60, convirtiéndose, tras su posterior desarrollo y evolución, en uno de los lenguajes de programación más influyentes en el campo de la simulación.

Si bien hay más avances en el campo de la simulación posteriores a estos, la mayoría son desarrollos de herramientas, no descubrimientos o desarrollos que cambiasen el paradigma de la misma manera que los avances explicados anteriormente hicieron.

2.2.3. Simulación vs Emulación

La emulación es un término importante en este contexto y que algunas personas pueden usarlo como sustituto de la palabra simulación de manera indiferente. Sin embargo, estos dos términos son drásticamente diferentes en un aspecto clave. Un emulador es “un software que permite que el comportamiento interno del sistema operativo de un ordenador imite el comportamiento de otro sistema informático” [20]. En otras palabras, mientras que una simulación se refiere a un programa informático que **asbtrae** un sistema para replicar su comportamiento, la emulación se refiere a un programa informático que **implementa** un sistema para replicar su comportamiento. Como se puede intuir, la emulación de un sistema consume muchos más recursos (CPU, memoria, energía) que la simulación de un sistema.

Otro apartado en el que difieren estas dos técnicas es el uso general para el que se aplican. Mientras que los simuladores son más usados para estudios y experimentos, los emuladores generalmente son usados para ejecutar un software, unas herramientas u otros componentes que están pensados para otros sistemas [20].

El uso de los emuladores no se restringe solo a la imitación del hardware, sino que se extiende al mundo de la virtualización. La virtualización es un tipo de emulación que, en vez de emular hardware, emula software. Por ejemplo, una máquina virtual de Linux estaría emulando el sistema operativo de Linux, usando la máquina del usuario y no una máquina emulada.

Una diferencia crucial entre la emulación y la simulación es que los emuladores, al imitar más a bajo nivel el comportamiento del sistema que los simuladores, son generalmente más lentos. Esto se debe a que los emuladores de hardware traducen las instrucciones que están escritas para la CPU de la máquina donde se ejecuta el emulador para que la CPU del hardware emulado las pueda entender. Esto pasa por traducir las instrucciones hasta el código máquina, lo que causa un retraso que afecta a la velocidad de la emulación [20].

Algunas de las diferencias más significativas entre la emulación y la simulación se presentan en la tabla 2.2 [20]-[22]:

TABLA 2.2.
SIMULACIÓN VS EMULACIÓN

Parámetro	Simulación	Emulación
Uso	Estudios y experimentos	Sustitución de un sistema
Precisión	La precisión depende del simulador, menos precisos por lo general	Inherentemente más precisos al replicar un sistema
Velocidad	Generalmente más rápidos	Generalmente más lentos
Entorno	Sólo Software	Hardware y Software

2.2.4. Retos de los simuladores

Una vez definido el concepto de simulación, se discurrirá sobre los distintos retos que tienen los simuladores actualmente. Antes de ello, hay que aportar algo de claridad sobre el tipo de experimentos que se pueden realizar ya que, de forma general, se pueden dividir en tres clases [23]:

1. **Experimentos *in vivo***: consisten en la experimentación con un sistema real. Por desgracia, no siempre son posibles y, aunque sean posibles, nunca se podrá probar tantas configuraciones del sistema como las que se pueden usando una simulación.
2. **Experimentos *in vitro***: consisten en el estudio de un sistema mediante la emulación, algo que puede tener problemas debido a que es un tipo de experimento que consume mucho tiempo. Este problema no sólo afecta a la longitud del experimento, si no que también afecta a la variabilidad de los resultados. Incluso aunque estos dos motivos no fuesen demasiado problemáticos, un sistema complejo requerirá del hardware adecuado, lo cual aumentará el coste del experimento debido al coste de obtención, mantenimiento y alimentación de las máquinas.
3. **Experimentos *in silico***: consisten en realizar abstracciones del sistema, es decir, simulaciones. A las posibilidades de experimentar en un entorno realista y variable se le añade el beneficio de la disminución significativa del coste del hardware.

Continuando con este razonamiento, los experimentos *in silico* parecen la mejor alternativa, no sólo cuestan menos, sino que ofrecen entornos de experimentación mucho más extensos que las dos alternativas. Sin embargo, los experimentos *in silico* cuentan con sus propios problemas.

El primero es la precisión, ya que la simulación deberá obtener unos resultados que no varíen significativamente respecto a un experimento *in vivo*. El segundo es la escalabilidad, ya que la simulación deberá poder correr experimentos grandes o rápidos [23]. Al igual que con la mayoría de sistemas, estos dos parámetros se ven enfrentados el uno al otro. Una simulación que use procesos más intensivos tardará más pero será más precisa que una simulación que utilice modelos menos complejos.

Otro problema que presentan los simuladores para este proyecto en específico es que su uso para experimentos en plataformas distribuidas y paralelas es escaso respecto a otras ramas donde su uso ha sido más prominente [23]. Además, los simuladores de este campo son específicos a su ámbito de uso. Por ejemplo, los simuladores que estudien el comportamiento de una plataforma conectada mediante cables serán drásticamente distintos a los usados para una plataforma conectada mediante *Wi-Fi*, causando así que sean los propios investigadores los que crean un simulador para sus estudios.

Es por ello que un paso importante de este proyecto es buscar un simulador de plataformas distribuidas y paralelas que cumpla con los requisitos de precisión y velocidad y que, adicionalmente, pueda simular el tipo adecuado de plataformas.

2.3. Simuladores

Tras hablar sobre las simulaciones, este último apartado cerrará el estado del arte hablando sobre las herramientas de simulación principales del panorama actual. Para ello, primero se hablará sobre *SimGrid*, la herramienta usada para llevar a cabo las simulaciones de este proyecto. Tras esto, se mencionarán una serie de herramientas alternativas que sirven para realizar simulaciones.

2.3.1. SimGrid

SimGrid se define como “un framework para desarrollar simuladores de aplicaciones distribuidas ejecutadas en plataformas distribuidas...” [24]. Pese a no ser un simulador per se, *SimGrid* permite desarrollar simuladores mediante una serie de clases, funciones y métodos agrupados en las diversas interfaces con las que cuenta.

Este framework de uso abierto lanzado en 1999 ha ayudado a llevar a cabo cientos de publicaciones científicas relacionadas con la simulación [24], desde artículos de conferencia hasta artículos de revistas científicas o tesis universitarias [25]. Así, este framework ha demostrado su adopción por la comunidad científica.

Gracias a pruebas teóricas y experimentales, *SimGrid* ha demostrado ser una herramienta precisa, versátil y escalable, mayormente debido a su acercamiento analítico a los problemas de simulación de cómputo, red y almacenamiento a los que se enfrentan los simuladores [23], [24]. Esta solución proporciona unos resultados rápidos y razonablemente precisos, enfrentando así los retos que se han explicado con anterioridad.

La programabilidad lograda por *SimGrid* se ve reforzada por la variedad de lenguajes de programación en los que se puede emplear este framework, pasando por *Python*, *C* y *C++*. La interfaz empleada en este proyecto y las razones para su elección se explicarán en el capítulo 4. *Diseño*.

2.3.2. Simuladores relacionados

El primer paso que hay que hacer para hablar de simuladores que guarden relación con *SimGrid* es establecer el significado de dos conceptos: la simulación *off-line* y la simulación *on-line*. La simulación *off-line* es un enfoque basado en la obtención de unos logs de computación y comunicación mediante la ejecución de la simulación en una plataforma real, datos que luego son repetidos en una plataforma distinta con unas especificaciones de hardware distintas [23]. La simulación *off-line* tiene un problema, y es que los logs de eventos están realmente unidos a la ejecución de la simulación, la cual depende de parámetros específicos a la máquina original. Algunos ejemplos de simuladores *off-line* son: *SimGrid*, *PSINS* [26], *LogGOPSim* [27] y *BigSim* [28].

La alternativa a la simulación *off-line* es la simulación *on-line*, que es otro acerca-

miento mediante el que la simulación se ejecuta en una plataforma host que intenta imitar el comportamiento de la plataforma donde se desea ejecutar la simulación. Una pega de este tipo de simuladores es que los recursos necesarios para llevar a cabo la simulación siempre serán mayores a los recursos consumidos si la aplicación simulada se ejecutase en la plataforma simulada. Por lo tanto, la escalabilidad de la simulación solo se alcanza mediante una escalabilidad horizontal o vertical del hardware, es decir, incurriendo en un gasto mayor [23]. Algunos ejemplos de estos simuladores son *MPI-SIM* [29] o *LAPSE* [30].

Estos dos acercamientos son una manera de dividir los simuladores, pero no la única, ya que se pueden clasificar también por el ámbito dentro del mundo de la simulación de programas MPI (Interfaz de Paso de Mensajes). Un área en la que se han desarrollado simuladores es la de *computación en malla* (*grid computing*), un tipo de computación en el que se combina la capacidad de cómputo de varias máquinas separadas geográficamente en una infraestructura de computación [31]. *SimGrid* es uno de los simuladores de computación en malla más usados actualmente, seguido también por *GridSim* [32].

Otro paradigma de la computación para el que se han desarrollado algunos simuladores es la famosa computación en la nube (*cloud computing*), teniendo simuladores como *GroudSim* [33], que permite también simular computación en malla; *CloudSim* [34], que se construye bajo los mismos principios que *GridSim* pero que expone interfaces para simular sistemas en la nube; e *iCanCloud* [35], desarrollado específicamente para simular plataformas en la nube.

Un ámbito de computación adicional para el que se han desarrollado simuladores son los sistemas de computación de voluntarios (*volunteer computing*), es decir, sistemas que tienen una gran cantidad de nodos volátiles y que son individualmente adueñados [23]. Algunos ejemplos de estos simuladores son: *BOINC* [36], el simulador de este tipo más utilizado en la actualidad; *SimBA* [37], un simulador que modela los clientes de *BOINC* como unos autómatas finitos basados en estudios de probabilidad de disponibilidad; y *EmBOINC* [38], que emula el código de los servidores *BOINC*.

El último ámbito de computación reseñable es la computación *peer-to-peer* o *P2P*. En este área, la mayoría de simuladores deciden sacrificar un poco de precisión para lograr una mayor escalabilidad, pudiendo así simular una gran cantidad de hosts [23]. Los simuladores de este campo más famosos son *PeerSim* [39] y *OverSim* [40]. La tabla 2.3 compara y se resume los simuladores presentados [23].

TABLA 2.3.
COMPARACIÓN DE SIMULADORES

Nombre	Computación	Enfoque	Aplicación
PSINS	HPC	Off-line	Rastreado de ejecución
LogGOPSIm	HPC	Off-line	Rastreado de ejecución
BigSim	HPC	Off-line	Rastreado de ejecución
MPI-SIM	HPC	On-line	Específica del programa
LAPSE	Distribuida	On-line	Rastreado de ejecución
SimGrid	Grid	Off-line	Rastreado de ejecución, especificación abstracta, específica del programa
GridSim	Grid	Off-line	Específica del programa
GroudSim	Grid, Cloud	Off-line	Específica del programa
CloudSim	Cloud	Off-line	Específica del programa
iCanCloud	Cloud	Off-line	Específica del programa
BOINC	Volunteer	Off-line	Específica del programa
SimBA	Volunteer	Off-line	Especificación abstracta, específica del programa
EmBOINC	Volunteer	Off-line	Especificación abstracta, específica del programa
PeerSim	P2P	Off-line	Especificación abstracta
OverSim	P2P	Off-line	Específica del programa

2.3.3. Modelos de simulación

Los simuladores como *SimGrid* normalmente modelan los recursos de un ordenador de diversas formas. Desde la CPU hasta la memoria, los discos, la red... todos los recursos tienen una vasta literatura que explora las mejores maneras de modelar cada recurso. Es por ello que este apartado analiza cómo son implementados algunos de los recursos en los simuladores.

CPU

Existen varias formas de modelar la CPU. Si se desea una precisión muy alta, se puede usar la simulación a nivel de ciclos, un tipo de simulación ocasionalmente usada en simuladores de computación de alto rendimiento (HPC) que suele ser lenta y poco escalable [23]. Sin embargo, la precisión conseguida por este tipo de modelo no siempre alcanza los estándares esperados, y, adicionalmente, la instanciación de un modelo de CPU usando esta técnica puede ser lenta si se pretende lograr una buena precisión en la arquitectura a simular.

Para solventar estos problemas, los simuladores normalmente usan un enfoque basa-

do en retardos de computación usando modelos analíticos simples [23]. Estos modelos simplemente calculan el tiempo que una tarea ocupa en el procesador usando operandos como el tamaño de la tarea, la velocidad del procesador y un factor aleatorio opcional. Este simple cálculo se puede ver en la siguiente formula:

$$ComputeTime = \frac{ComputeSize}{ComputeSpeed} + RandomFactor$$

El tamaño y la velocidad se pueden medir, por ejemplo, en FLOPS (Operaciones de coma flotante por segundo). Adicionalmente, la velocidad de cómputo se puede obtener mediante el uso de benchmarks sobre la plataforma a simular. Este modelo, si bien es simple y limitado al no poder simular características específicas de las arquitecturas de los distintos procesadores, obtiene unos resultados razonablemente precisos en relación a su complejidad [23].

Almacenamiento

A pesar de lo que pueda parecer, el modelado de componentes de almacenamiento como discos duros es extremadamente complicado. Los simuladores normalmente no suelen modelar el almacenamiento, ya sea porque no es necesario o porque se puede delegar su implementación al programador de la simulación. Si bien existen algunos simuladores que implementan un modelo que consigue una precisión alta mediante el análisis discreto y granular de las operaciones relacionadas con el almacenamiento, como es el caso del simulador *DiskSim* [41], la mayoría de simuladores que tienen un modelo de almacenamiento suelen tener implementaciones más simples.

Este el caso de simuladores como *SimGrid*, *CloudSim* o *GridSim*, los cuales emplean modelos basados en parámetros como el tiempo de búsqueda (fijo o aleatorio) o un ratio de transferencia fijo para implementar una abstracción simple del almacenamiento. Este tipo de modelos, a diferencia de los modelos de CPU, no son tan representativos de la realidad ya que hay ciertos aspectos como el uso de cachés o la localidad en la memoria que no se suelen ver implementados [23]. A pesar de todo, el simulador *iCanCloud* implementa un modelo no trivial y muy sofisticado que discretiza el acceso a memoria en bloques y simula el tiempo de acceso usando el principio de localidad.

Red

Todos los simuladores de un tipo de computación que requiera comunicación entre nodos implementan algún tipo de modelo de red, si bien los enfoques son muy diversos. Una alternativa muy precisa es la simulación a nivel de paquetes de red, que implementa protocolos de red completos, y normalmente es usada por simuladores designados para el estudio del comportamiento de la red. Como se puede esperar, la alta precisión compromete la velocidad y la escalabilidad de la simulación, por lo que no todos los simuladores

pueden o deciden usar este modelo [23]. Este es el caso de los simuladores de malla, exceptuando *SimGrid*, que ofrece una interfaz para usar este modelo de red para aquellas simulaciones donde esperar un largo tiempo hasta su finalización no es un problema.

Otros simuladores implementan modelos analíticos que, inherentemente, son menos precisos al ignorar eventos típicos de la red, como la saturación de las redes; o propiedades asociadas a los enlaces que la conforman, como la banda ancha o la latencia. Como se puede esperar, esta decisión de diseño limita la precisión y usabilidad de algunos simuladores si la simulación realiza muchas operaciones a través de la red.

Otro enfoque que se puede tomar es el establecimiento de una serie de casos base medidos a través de distintas configuraciones de la red en un simulador con un modelo de red a nivel de paquete [23]. Si bien esto puede resultar en una simulación más precisa, la veracidad de los datos recogidos es proporcional al número de configuraciones de red probadas, número que en la realidad suele resultar inferior a lo ideal.

Plataforma de simulación

Los simuladores también deben decidir la manera en la que se modela la plataforma de la simulación. El modelado de la misma pasa desde el instanciado de los modelos anteriormente mencionados, ya sea mediante valores constantes o valores calculados analíticamente que son establecidos usando texto o algún lenguaje de programación; hasta la interconexión de los elementos de una red (hosts, routers, clusters...) [23].

Acciones de la simulación

Tras modelar la plataforma, es necesario definir la forma en la que los actores hacen uso de los recursos de la misma, para lo que hay varios enfoques. Uno de ellos es mediante la anteriormente explicada simulación *off-line*, con la que se consiguen trazas de eventos que serán repetidos en la plataforma simulada.

Otra forma en la que se pueden explicitar las acciones de los actores es mediante la realización de una descripción formal de la ejecución de aplicación que se ejecuta en la plataforma. Esta representación, a pesar de ser compacta y escalable, puede no ser conseguida siempre debido a la diferencia entre la rigidez inherente de este enfoque y la potencial variabilidad de una aplicación real [23].

Dada las limitaciones de los dos enfoques anteriormente explicados, muchos simuladores optan por una tercera opción, dejar que el usuario describa las acciones usando un set de instrucciones que puede llamar cuando quiera [23]. Este acercamiento proporciona un mayor grado de libertad al usuario, que puede describir mediante las instrucciones provistas una aplicación que posteriormente será simulada en la plataforma.

3. ANÁLISIS

Este capítulo contiene un análisis de las funcionalidades que debe prestar el simulador. El objetivo de este capítulo es proporcionar una propuesta realista, bien definida y trazable que exponga claramente las capacidades del sistema a desarrollar. Para ello, primero se hará un resumen de la propuesta para tener en mente la idea que se quiere llevar a cabo en el proyecto. Luego, se elicitarán los requisitos funcionales y no funcionales para definir concretamente lo que se espera del simulador. Una vez que se hayan mostrado todos los requisitos, se expondrán los casos de uso del simulador, finalizando este apartado con unas matrices de trazabilidad que permitirán verificar el sistema final.

3.1. Propuesta

El objetivo de este proyecto es la realización de un simulador preciso de un sistema de ficheros para entornos distribuidos y paralelos. Este simulador se llevará a cabo usando una interfaz del framework *SimGrid* definida en el lenguaje de programación C++. Además, se proporcionarán las herramientas adecuadas para que los usuarios del simulador puedan definir cómodamente la plataforma de simulación y las acciones de los actores presentes en la misma. La simulación seguirá un patrón cliente-servidor, donde los servidores alojen los sistemas de ficheros distribuidos que pueden ser accedidos por los clientes, que mandarán operaciones CRUD a los servidores.

Como punto adicional que no se ha mencionado en ningún punto anterior, el usuario del simulador podrá decidir qué políticas de acceso a los servidores usará cada cliente y respecto a qué servidor. Estas políticas se explican detalladamente en el apartado [4.4. Políticas](#). De momento, sólo hay que saber que estas políticas dictaminan la manera en la que los clientes acceden a los servidores.

Por último, tal y como se ha indicado en el capítulo [1.2. Objetivos](#), el usuario podrá acceder a una serie de gráficas generadas automáticamente tras la simulación que muestren la información relacionada con el consumo energético y temporal de los servidores.

3.2. Requisitos

En este apartado se describen los requisitos del proyecto, haciendo una subdivisión entre aquellos requisitos que sean de sistema y aquellos que sean de usuario. Todos estos requisitos se escribirán siguiendo los principios detallados en la *Guía de especificación de requisitos de software* del IEEE [42]. Los requisitos seguirán los siguientes principios del texto anteriormente citado (nótese la diferencia entre requisito de software y la palabra “requisito”, que hace referencia a una necesidad):

- **Claridad:** un requisito de software es no ambiguo o claro si y sólo si solo acepta una interpretación.
- **Completo:** un requisito de software es completo si incluye todos los requisitos relacionados con una funcionalidad, con un desempeño, con atributos, con interfaces externas o con restricciones de diseño.
- **Verificable:** un requisito de software es verificable si y sólo si todos los requisitos contenidos en el mismo son verificables, esto es, que se puede comprobar su adecuado funcionamiento con un proceso finito y eficiente.
- **Consistente:** un requisito de software es consistente si y sólo si ningún requisito o conjunto de requisitos definidos en el mismo están en conflicto.
- **Modificable:** un requisito de software es modificable si y sólo si la estructura y el estilo del mismo son tal que permitan cualquier cambio para modificar el requisito de software.
- **Trazable:** un requisito de software es trazable si el origen de sus requisitos es claro y facilitan la referencia futura.
- **Usable durante las fases de operación y mantenimiento:** los requisitos de software cumplirán con las necesidades de la fase de mantenimiento y operación, incluyendo el reemplazo eventual del software.

Con estos principios en mente, se han desarrollado todos los requisitos del proyecto.

3.2.1. Requisitos de usuarios

En esta sección se definen los requisitos de usuario, aquellos derivados de las necesidades de los usuarios del sistema final. Primero, hay que hacer una distinción entre los dos tipos de requisitos de usuario: los requisitos de capacidades y los de restricciones. Mientras que los requisitos de capacidades definen aquello que el usuario espera poder hacer en el sistema, los requisitos de restricciones definen aquello que el usuario no puede hacer debido a las limitaciones del sistema.

Para poder llevar a cabo esta tarea de elicitación, se define la tabla 3.1, que se usa como referencia para escribir los requisitos. En ella, se pueden ver los campos que cada requisito deberá tener y la definición de los mismos.

TABLA 3.1.
PLANTILLA PARA LOS REQUISITOS DE USUARIO

ID: RU_XX_YY	
Descripción	Descripción del requisito
Necesidad	Necesidad del requisito (Baja, Media, Alta)
Prioridad	Prioridad del requisito (Baja, Media, Alta)
Estabilidad	Cómo de probable es que no cambie el requisito (Baja, Media, Alta)
Verificabilidad	Cómo de fácil es verificar que el requisito se cumple (Fácil, Media, Difícil)

Es necesario mencionar que el *ID* del requisito se define como una cadena unívoca donde el valor “XX” se sustituirá por la cadena “CA” o “RE”, dependiendo si es un requisito de capacidades o de restricciones, respectivamente; y la cadena “YY” se sustituirá por un identificador numérico autoincremental independiente entre los dos tipos de los requisitos. Una vez que se han aclarado estos puntos, se comienza la elicitación de los requisitos.

TABLA 3.2.
REQUISITO RU_CA_01

ID: RU_CA_01	
Descripción	El usuario deberá ser capaz de añadir servidores a la plataforma de la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.3.
REQUISITO RU_CA_02

ID: RU_CA_02	
Descripción	El usuario deberá ser capaz de modificar los parámetros de los servidores de la plataforma de la simulación
Necesidad	Alta
Prioridad	Media
Estabilidad	Media
Verificabilidad	Media

TABLA 3.4.
REQUISITO RU_CA_03

ID: RU_CA_03	
Descripción	El usuario deberá ser capaz de eliminar servidores de la plataforma de la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.5.
REQUISITO RU_CA_04

ID: RU_CA_04	
Descripción	El usuario deberá ser capaz de añadir clientes a la plataforma de la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.6.
REQUISITO RU_CA_05

ID: RU_CA_05	
Descripción	El usuario deberá ser capaz de modificar los parámetros de los clientes de la plataforma de la simulación
Necesidad	Baja
Prioridad	Baja
Estabilidad	Media
Verificabilidad	Alta

TABLA 3.7.
REQUISITO RU_CA_06

ID: RU_CA_06	
Descripción	El usuario deberá ser capaz de eliminar clientes de la plataforma de la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.8.
REQUISITO RU_CA_07

ID: RU_CA_07	
Descripción	El usuario deberá ser capaz de añadir enlaces de red a la plataforma de la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.9.
REQUISITO RU_CA_08

ID: RU_CA_08	
Descripción	El usuario deberá ser capaz de modificar los parámetros de los enlaces de red de la plataforma de la simulación
Necesidad	Baja
Prioridad	Baja
Estabilidad	Media
Verificabilidad	Alta

TABLA 3.10.
REQUISITO RU_CA_09

ID: RU_CA_09	
Descripción	El usuario deberá ser capaz de eliminar enlaces de red de la plataforma de la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.11.
REQUISITO RU_CA_10

ID: RU_CA_10	
Descripción	El usuario deberá ser capaz de añadir rutas compuestas de enlaces de red que unan a los clientes con los servidores a la plataforma de la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.12.
REQUISITO RU_CA_11

ID: RU_CA_11	
Descripción	El usuario deberá ser capaz de modificar los parámetros y los enlaces que forman las rutas entre los clientes y los servidores de la plataforma de la simulación
Necesidad	Baja
Prioridad	Baja
Estabilidad	Media
Verificabilidad	Alta

TABLA 3.13.
REQUISITO RU_CA_12

ID: RU_CA_12	
Descripción	El usuario deberá ser capaz de eliminar las rutas que unen los clientes con los servidores de la plataforma de la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.14.
REQUISITO RU_CA_13

ID: RU_CA_13	
Descripción	El usuario deberá ser capaz de definir las operaciones que los clientes mandarán a los servidores durante la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Difícil

TABLA 3.15.
REQUISITO RU_CA_14

ID: RU_CA_14	
Descripción	El usuario deberá ser capaz de modificar las operaciones que los clientes mandarán a los servidores durante la simulación
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Media

TABLA 3.16.
REQUISITO RU_CA_15

ID: RU_CA_15	
Descripción	El usuario deberá ser capaz de eliminar las operaciones que los clientes mandarán a los servidores durante la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.17.
REQUISITO RU_CA_16

ID: RU_CA_16	
Descripción	El usuario deberá poder definir la política de acceso de cada cliente de forma individual
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Difícil

TABLA 3.18.
REQUISITO RU_CA_17

ID: RU_CA_17	
Descripción	El usuario deberá poder modificar la política de acceso de cada cliente de forma individual
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.19.
REQUISITO RU_CA_18

ID: RU_CA_18	
Descripción	El usuario deberá ser capaz de devolver la plataforma de la simulación a un estado por defecto
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.20.
REQUISITO RU_CA_19

ID: RU_CA_19	
Descripción	El usuario deberá ser capaz de borrar todas las acciones de los clientes mediante una acción
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.21.
REQUISITO RU_CA_20

ID: RU_CA_20	
Descripción	El usuario deberá poder acceder a las gráficas generadas tras la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.22.
REQUISITO RU_CA_21

ID: RU_CA_21	
Descripción	El usuario deberá poder acceder a la información de consumo energético-temporal al finalizar la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.23.
REQUISITO RU_RE_01

ID: RU_RE_01	
Descripción	El sistema deberá contar con una interfaz que controle y restrinja la modificación de la plataforma de la simulación
Necesidad	Alta
Prioridad	Media
Estabilidad	Media
Verificabilidad	Difícil

TABLA 3.24.
REQUISITO RU_RE_02

ID: RU_RE_02	
Descripción	El sistema deberá contar con una interfaz que controle y restrinja la modificación de las acciones de los actores de la simulación
Necesidad	Alta
Prioridad	Media
Estabilidad	Media
Verificabilidad	Difícil

TABLA 3.25.
REQUISITO RU_RE_03

ID: RU_RE_03	
Descripción	El usuario deberá ejecutar la simulación mediante un script que se encargue de pasarle al ejecutable los argumentos adecuados
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.26.
REQUISITO RU_RE_04

ID: RU_RE_04	
Descripción	El sistema deberá validar la configuración de la plataforma de la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Media

TABLA 3.27.
REQUISITO RU_RE_05

ID: RU_RE_05	
Descripción	El sistema deberá validar la configuración de las acciones de los actores de la simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil

TABLA 3.28.
REQUISITO RU_RE_06

ID: RU_RE_06	
Descripción	El sistema debe ser multiplataforma
Necesidad	Baja
Prioridad	Baja
Estabilidad	Media
Verificabilidad	Difícil

3.2.2. Requisitos de sistema

Una vez se han elicitado los requisitos de usuarios, se expondrán los requisitos de sistema, aquellos derivados de los requisitos de usuarios pero centrados en lo que deberá hacer el sistema final. Antes de hacer la elicitación de los requisitos del sistema, se definen dos clases de requisitos: los funcionales, que se encargan de describir funcionalidades concretas del sistema; y los no funcionales, que se encargan de describir restricciones impuestas por el software.

Para realizar la elicitación, y de la misma manera que con los otros requisitos, se define la tabla 3.29 para usar como referencia.

TABLA 3.29.
PLANTILLA PARA LOS REQUISITOS DE SISTEMA

ID: RS_XX_YY	
Descripción	Descripción del requisito
Necesidad	Necesidad del requisito (Baja, Media, Alta)
Prioridad	Prioridad del requisito (Baja, Media, Alta)
Estabilidad	Cómo de probable es que no cambie el requisito (Baja, Media, Alta)
Verificabilidad	Cómo de fácil es verificar que el requisito se cumple (Fácil, Media, Difícil)
Origen	Referencia a los requisitos de usuario que inspiraron este requisito

De la misma manera que antes, el valor “XX” del identificador del requisito se sustituirá por “FN” o “NF”, dependiendo de si el requisito es funcional o no funcional, respectivamente; y el valor “YY” se sustituirá por un valor numérico autoincremental independiente entre los dos tipos de los requisitos. Una vez que se han aclarado estos puntos, se procede a la elicitación de los requisitos de sistema:

TABLA 3.30.
REQUISITO RS_FN_01

ID: RS_FN_01	
Descripción	El sistema debe tener una interfaz que permita la modificación controlada de la plataforma de simulación
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Difícil
Origen	<i>RU_CA_01, RU_CA_02, RU_CA_03, RU_CA_04, RU_CA_05, RU_CA_06, RU_CA_07, RU_CA_08, RU_CA_09, RU_CA_10, RU_CA_11, RU_CA_12, RU_CA_18, RU_RE_01</i>

TABLA 3.31.
REQUISITO RS_FN_02

ID: RS_FN_02	
Descripción	La interfaz de la plataforma de simulación debe permitir la gestión de los servidores
Necesidad	Alta
Prioridad	Alta
Estabilidad	Media
Verificabilidad	Media
Origen	<i>RU_CA_01, RU_CA_02, RU_CA_03</i>

TABLA 3.32.
REQUISITO RS_FN_03

ID: RS_FN_03	
Descripción	La interfaz de la plataforma de simulación debe permitir la gestión de los clientes
Necesidad	Alta
Prioridad	Media
Estabilidad	Media
Verificabilidad	Media
Origen	<i>RU_CA_04, RU_CA_05, RU_CA_06</i>

TABLA 3.33.
REQUISITO RS_FN_04

ID: RS_FN_04	
Descripción	La interfaz de la plataforma de simulación debe permitir la gestión de los enlaces de red
Necesidad	Media
Prioridad	Baja
Estabilidad	Media
Verificabilidad	Fácil
Origen	<i>RU_CA_07, RU_CA_08, RU_CA_09</i>

TABLA 3.34.
REQUISITO RS_FN_05

ID: RS_FN_05	
Descripción	La interfaz de la plataforma de simulación debe permitir la gestión de las rutas entre los clientes y los servidores
Necesidad	Media
Prioridad	Baja
Estabilidad	Media
Verificabilidad	Fácil
Origen	<i>RU_CA_10, RU_CA_11, RU_CA_12</i>

TABLA 3.35.
REQUISITO RS_FN_06

ID: RS_FN_06	
Descripción	El sistema debe definir un estado por defecto de la plataforma de la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_18</i>

TABLA 3.36.
REQUISITO RS_FN_07

ID: RS_FN_07	
Descripción	La interfaz de la plataforma de simulación debe permitir la recuperación del estado definido como el estado por defecto
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_18</i>

TABLA 3.37.
REQUISITO RS_FN_08

ID: RS_FN_08	
Descripción	El sistema debe tener una interfaz que permita la gestion controlada de las acciones de los actores de la plataforma
Necesidad	Alta
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Difícil
Origen	<i>RU_CA_13, RU_CA_14, RU_CA_15, RU_CA_16, RU_CA_17, RU_CA_19, RU_RE_02</i>

TABLA 3.38.
REQUISITO RS_FN_09

ID: RS_FN_09	
Descripción	La interfaz de las acciones de los actores de la plataforma debe permitir la gestión de las operaciones de cada cliente de forma individualizada
Necesidad	Alta
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_CA_13, RU_CA_14, RU_CA_15</i>

TABLA 3.39.
REQUISITO RS_FN_10

ID: RS_FN_10	
Descripción	La interfaz de las acciones de los actores de la plataforma debe permitir la gestión de las políticas de acceso de cada cliente de forma individualizada
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_16, RU_CA_17</i>

TABLA 3.40.
REQUISITO RS_FN_11

ID: RS_FN_11	
Descripción	El sistema debe definir un estado por defecto de las acciones de los actores de la simulación
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_19</i>

TABLA 3.41.
REQUISITO RS_FN_12

ID: RS_FN_12	
Descripción	La interfaz de los actores de la simulación debe permitir la recuperación del estado definido como el estado por defecto
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_19</i>

TABLA 3.42.
REQUISITO RS_FN_13

ID: RS_FN_13	
Descripción	El sistema debe implementar una forma de comunicación entre los clientes y los servidores
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_CA_13, RU_CA_14, RU_CA_15</i>

TABLA 3.43.
REQUISITO RS_FN_14

ID: RS_FN_14	
Descripción	Los servidores deben implementar las operaciones simuladas
Necesidad	Alta
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Difícil
Origen	<i>RU_CA_13, RU_CA_14, RU_CA_15</i>

TABLA 3.44.
REQUISITO RS_FN_15

ID: RS_FN_15	
Descripción	El sistema debe medir con precisión el consumo de tiempo de cada operación
Necesidad	Alta
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Difícil
Origen	<i>RU_CA_20, RU_CA_21</i>

TABLA 3.45.
REQUISITO RS_FN_16

ID: RS_FN_16	
Descripción	El sistema debe medir con precisión el consumo energético de cada servidor
Necesidad	Alta
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Difícil
Origen	<i>RU_CA_20, RU_CA_21</i>

TABLA 3.46.
REQUISITO RS_FN_17

ID: RS_FN_17	
Descripción	El sistema debe mostrar la información sobre el consumo temporal al finalizar la simulación
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_21</i>

TABLA 3.47.
REQUISITO RS_FN_18

ID: RS_FN_18	
Descripción	El sistema debe mostrar la información sobre el consumo energético al finalizar la simulación
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_21</i>

TABLA 3.48.
REQUISITO RS_FN_19

ID: RS_FN_19	
Descripción	El sistema debe generar una gráfica de consumo de tiempo por servidor al finalizar la simulación
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_CA_20</i>

TABLA 3.49.
REQUISITO RS_FN_20

ID: RS_FN_20	
Descripción	El sistema debe generar una gráfica de consumo de energía por servidor al finalizar la simulación
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_CA_20</i>

TABLA 3.50.
REQUISITO RS_FN_21

ID: RS_FN_21	
Descripción	El sistema debe generar una gráfica de consumo de tiempo agrupador por servidor y por tipo de operación al finalizar la simulación
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_CA_20</i>

TABLA 3.51.
REQUISITO RS_FN_22

ID: RS_FN_22	
Descripción	El sistema informar al usuario en caso de que haya algún error durante la ejecución de la simulación
Necesidad	Alta
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_CA_20</i>

TABLA 3.52.
REQUISITO RS_NF_01

ID: RS_NF_01	
Descripción	La simulación debe estar implementada usando la interfaz S4U del framework de simulación <i>SimGrid</i>
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil
Origen	N/A

TABLA 3.53.
REQUISITO RS_NF_02

ID: RS_NF_02	
Descripción	Los interfaces deben estar implementadas en algún lenguaje que permita su ejecución en una interfaz de línea de comandos
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil
Origen	N/A

TABLA 3.54.
REQUISITO RS_NF_03

ID: RS_NF_03	
Descripción	Los gráficos deben ser generados usando la herramienta <i>Gnuplot</i>
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_CA_20</i>

TABLA 3.55.
REQUISITO RS_NF_04

ID: RS_NF_04	
Descripción	El sistema de logs de la simulación debe ser implementado usando la herramienta <i>XBT</i> del framework <i>SimGrid</i>
Necesidad	Alta
Prioridad	Alta
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_04, RU_RE_05</i>

TABLA 3.56.
REQUISITO RS_NF_05

ID: RS_NF_05	
Descripción	Las interfaces deben seguir en la medida de lo posible las Heurísticas de Nielsen
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_01, RU_RE_02</i>

TABLA 3.57.
REQUISITO RS_NF_06

ID: RS_NF_06	
Descripción	Las interfaces deben seguir en la medida de lo posible los patrones de diseño de interfaces
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_01, RU_RE_02</i>

TABLA 3.58.
REQUISITO RS_NF_07

ID: RS_NF_07	
Descripción	La simulación debe funcionar para los sistemas operativos Linux y MacOS
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_06</i>

TABLA 3.59.
REQUISITO RS_NF_08

ID: RS_NF_08	
Descripción	El script de ejecución de la simulación se debe adaptar para los sistemas operativos Linux y MacOS
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_03, RU_RE_06</i>

TABLA 3.60.
REQUISITO RS_NF_09

ID: RS_NF_09	
Descripción	La simulación deberá comprobar que la plataforma de la simulación está bien definida antes de la simulación, acabando el programa en caso de que encuentre algún error
Necesidad	Alta
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Media
Origen	<i>RU_RE_04</i>

TABLA 3.61.
REQUISITO RS_NF_10

ID: RS_NF_10	
Descripción	La simulación deberá comprobar que las acciones de los actores (clientes) están bien definidas antes de la simulación, acabando el programa en caso de que encuentre algún error
Necesidad	Media
Prioridad	Media
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_05</i>

TABLA 3.62.
REQUISITO RS_NF_11

ID: RS_NF_11	
Descripción	Las acciones llevadas a cabo en las interfaces gráficas deben tener un tiempo de respuesta rápido
Necesidad	Media
Prioridad	Baja
Estabilidad	Alta
Verificabilidad	Fácil
Origen	<i>RU_RE_01, RU_RE_02</i>

3.3. Casos de uso

Una vez que se han presentado los requisitos del sistema, es momento de presentar los casos de uso, que permiten representar las acciones que el usuario realiza en el sistema. Para hablar de los casos de uso, primero se presenta el diagrama 3.1 realizado en *UML* (Lenguaje Unificado de Modelado) [43].

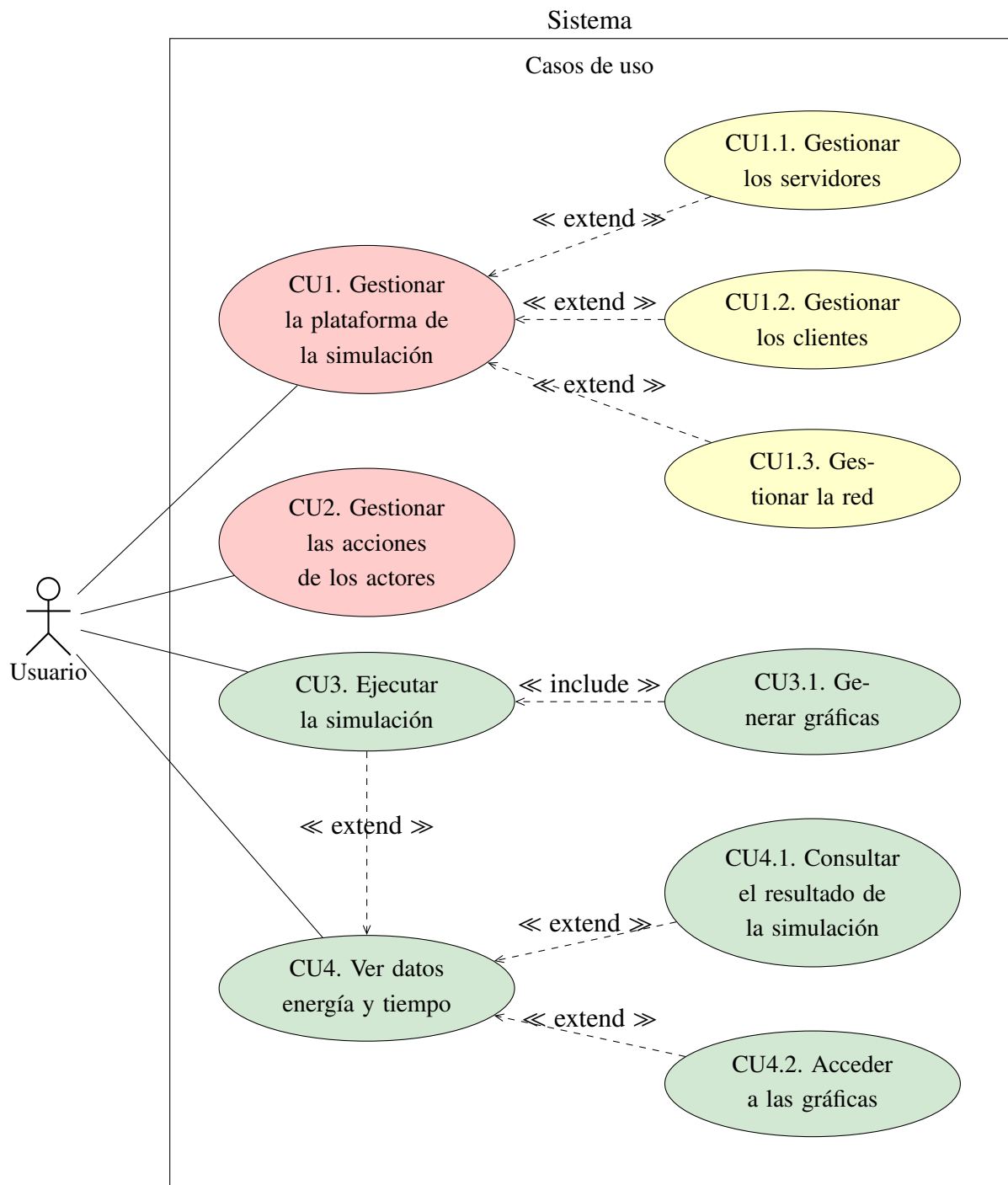


Fig. 3.1. Diagrama de casos de uso

El diagrama de casos de uso clasifica los casos de uso en un grado de dificultad que se corresponde con el color que tienen. En este caso, verde es un caso de uso fácil, amarillo es un caso de uso medio y rojo es un caso de uso difícil. Cabe decir que esta clasificación no se elige en función de la dificultad de la implementación, sino en el número de interacciones que el usuario tiene que realizar para llevar a cabo el caso de uso. Los casos de uso sencillos, como ejecutar la simulación o ver los datos de energía y tiempo, requieren de pocas interacciones; con un comando o con unos pocos clics se pueden llevar a cabo. Sin embargo, los casos de uso complicados, como la gestión de la configuración de la simulación, pueden requerir de muchas más interacciones por parte del usuario.

Una vez que esto se ha aclarado, es conveniente aportar una información algo más detallada de cada uno de estos casos de uso. Para hacer esto, se usará la tabla 3.63, cuyos apartados se describen en detalle.

TABLA 3.63.
PLANTILLA DE LOS CASOS DE USO

ID: CUX	
Nombre	Nombre que lleva el caso de uso dentro del diagrama
Actor	Ente que lleva a cabo el caso de uso
Objetivo	Descripción breve de en qué consiste el objetivo de este caso de uso
Descripción	Pasos que tiene que hacer el actor
Precondiciones	Lista de precondiciones
Postcondiciones	Lista de postcondiciones

A continuación se puede visualizar la información referente a cada caso de uso.

TABLA 3.64.
CASO DE USO CU1

ID: CU1	
Nombre	Gestionar la plataforma de la simulación
Actor	Usuario
Objetivo	Gestionar la plataforma de la simulación y realizar los cambios necesarios para lograr la configuración de los actores deseada
Descripción	1. Ejecutar la interfaz de gestión de plataformas 2. Cambiar la información de los servidores en su respectiva sección, si se desea 3. Cambiar la información de los clientes en su respectiva sección, si se desea 4. Cambiar la información de la red en su respectiva sección, si se desea 5. Acabar el programa
Precondiciones	N/A
Postcondiciones	La información de la plataforma se habrá actualizado en los ficheros pertinentes

TABLA 3.65.
CASO DE USO CU1.1

ID: CU1.1	
Nombre	Gestionar los servidores
Actor	Usuario
Objetivo	Realizar modificaciones en los servidores desplegados en la plataforma
Descripción	1. Cambiar la información de los servidores en su respectiva sección, si se desea 2. Acabar el programa
Precondiciones	Tener la interfaz de las plataformas ejecutándose
Postcondiciones	La información de los servidores en la plataforma se habrá actualizado en los ficheros pertinentes

TABLA 3.66.
CASO DE USO CU1.2

ID: CU1.2	
Nombre	Gestionar los clientes
Actor	Usuario
Objetivo	Realizar modificaciones en los clientes desplegados en la plataforma
Descripción	1. Cambiar la información de los clientes en su respectiva sección, si se desea 2. Acabar el programa
Precondiciones	Tener la interfaz de las plataformas ejecutándose
Postcondiciones	La información de los clientes en la plataforma se habrá actualizado en los ficheros pertinentes

TABLA 3.67.
CASO DE USO CU1.3

ID: CU1.3	
Nombre	Gestionar la red
Actor	Usuario
Objetivo	Realizar modificaciones en la red de la plataforma
Descripción	1. Cambiar la información de la red en su respectiva sección, si se desea 2. Acabar el programa
Precondiciones	Tener la interfaz de las plataformas ejecutándose
Postcondiciones	La información de la red de la plataforma se habrá actualizado en los ficheros pertinentes

TABLA 3.68.
CASO DE USO CU2

ID: CU2	
Nombre	Gestionar las acciones de los actores
Actor	Usuario
Objetivo	Gestionar las operaciones y las políticas de los clientes desplegados en la plataforma
Descripción	1. Ejecutar la interfaz de los actores 2. Cambiar las operaciones de un cliente en su respectiva sección, si se desea 3. Cambiar la política de un cliente en su respectiva sección, si se desea 4. Acabar el programa
Precondiciones	N/A
Postcondiciones	La información de los actores de la simulación se habrá actualizado en los ficheros pertinentes

TABLA 3.69.
CASO DE USO CU3

ID: CU3	
Nombre	Ejecutar la simulación
Actor	Usuario
Objetivo	Ejecutar la simulación con la plataforma y los actores deseados
Descripción	1. Ejecutar el script adecuado al sistema operativo de la máquina donde se ejecute la simulación
Precondiciones	Tener una configuración de la plataforma y una de los actores de la plataforma Tener la simulación compilada
Postcondiciones	Los ficheros de datos necesarios para las gráficas se habrán generado La información de la simulación se mostrará por pantalla

TABLA 3.70.
CASO DE USO CU3.1

ID: CU3.1	
Nombre	Generar las gráficas
Actor	Usuario
Objetivo	Generar las gráficas a partir de los datos generados al ejecutar la simulación
Descripción	N/A, es automático
Precondiciones	Ejecutar la simulación (CU3)
Postcondiciones	Las gráficas se habrán generado adecuadamente

TABLA 3.71.
CASO DE USO CU4

ID: CU4	
Nombre	Ver datos de energía y tiempo
Actor	Usuario
Objetivo	Visualizar los datos del resultado de la simulación
Descripción	1. Visualizar los resultados de la ejecución en la interfaz de línea de comandos, si se desea 2. Acceder a la carpeta de resultados y clickar en la imagen del gráfico que se quiere observar
Precondiciones	La simulación ha sido ya ejecutada
Postcondiciones	La información se mostrará en la pantalla del usuario

TABLA 3.72.
CASO DE USO CU4.1

ID: CU4.1	
Nombre	Consultar el resultado de la simulación
Actor	Usuario
Objetivo	Ver el resultado de la simulación en la interfaz de línea de comandos
Descripción	1. Visualizar los resultados de la ejecución en la interfaz de línea de comandos, si se desea
Precondiciones	La simulación ha sido ya ejecutada
Postcondiciones	La información se mostrará en la pantalla del usuario

TABLA 3.73.
CASO DE USO CU4.2

ID: CU4.2	
Nombre	Acceder a las gráficas
Actor	Usuario
Objetivo	Visualizar la gráfica deseada con los datos de la ejecución de la simulación
Descripción	1.Acceder a la carpeta de resultados y clickar en la imagen del gráfico que se quiere observar
Precondiciones	La simulación ha sido ya ejecutada
Postcondiciones	La información se mostrará en la pantalla del usuario

3.4. Trazabilidad

El último apartado de este capítulo contiene la información relacionada con la trazabilidad de los requisitos respecto a los casos de uso. Para representar esta relación, se presentan las siguientes matrices de trazabilidad de requisitos y casos de uso, que muestran qué requisitos tienen que ver con qué casos de uso. Como el número de requisitos es grande, se presentarán 4 matrices en función del tipo de requisito que contenga cada una. A continuación, se muestran dichas matrices.

TABLA 3.74.

MATRIZ DE TRAZABILIDAD DE REQUISITOS DE USUARIO DE
CAPACIDADES

	CU1	CU1.1	CU1.2	CU1.3	CU2	CU3	CU3.1	CU4	CU4.1	CU4.2
RU_CA_01	✓	✓								
RU_CA_02	✓	✓								
RU_CA_03	✓	✓								
RU_CA_04	✓		✓							
RU_CA_05	✓		✓							
RU_CA_06	✓		✓							
RU_CA_07	✓			✓						
RU_CA_08	✓			✓						
RU_CA_09	✓			✓						
RU_CA_10	✓			✓						
RU_CA_11	✓			✓						
RU_CA_12	✓			✓						
RU_CA_13					✓					
RU_CA_14					✓					
RU_CA_15					✓					
RU_CA_16					✓					
RU_CA_17					✓					
RU_CA_18	✓									
RU_CA_19					✓					
RU_CA_20							✓	✓	✓	
RU_CA_21								✓		✓

TABLA 3.75.

MATRIZ DE TRAZABILIDAD DE REQUISITOS DE USUARIO DE
RESTRICCIONES

	CU1	CU1.1	CU1.2	CU1.3	CU2	CU3	CU3.1	CU4	CU4.1	CU4.2
RU_RE_01	✓									
RU_RE_02		✓								
RU_RE_03						✓				
RU_RE_04	✓					✓				
RU_RE_05					✓	✓				
RU_RE_06	✓	✓	✓	✓	✓	✓	✓			

TABLA 3.76.
MATRIZ DE TRAZABILIDAD DE REQUISITOS DE SISTEMA
FUNCIONALES

	CU1	CU1.1	CU1.2	CU1.3	CU2	CU3	CU3.1	CU4	CU4.1	CU4.2
RS_FN_01	✓									
RS_FN_02	✓	✓								
RS_FN_03	✓		✓							
RS_FN_04	✓			✓						
RS_FN_05	✓			✓						
RS_FN_06	✓									
RS_FN_07	✓									
RS_FN_08					✓					
RS_FN_09					✓					
RS_FN_10					✓					
RS_FN_11					✓					
RS_FN_12					✓					
RS_FN_13						✓				
RS_FN_14						✓				
RS_FN_15						✓				
RS_FN_16						✓				
RS_FN_17						✓		✓	✓	
RS_FN_18						✓		✓	✓	
RS_FN_19						✓	✓	✓		✓
RS_FN_20						✓	✓	✓		✓
RS_FN_21						✓	✓	✓		✓
RS_FN_22						✓				

TABLA 3.77.

MATRIZ DE TRAZABILIDAD DE REQUISITOS DE SISTEMA NO FUNCIONALES

	CU1	CU1.1	CU1.2	CU1.3	CU2	CU3	CU3.1	CU4	CU4.1	CU4.2
RS_NF_01	✓				✓	✓				
RS_NF_02	✓				✓					
RS_NF_03							✓			
RS_NF_04						✓				
RS_NF_05	✓				✓					
RS_NF_06	✓				✓					
RS_NF_07	✓				✓	✓	✓			
RS_NF_08						✓				
RS_NF_09	✓					✓				
RS_NF_10					✓	✓				
RS_NF_11	✓				✓					

4. DISEÑO

Tras el análisis, viene el diseño, que trata las decisiones tomadas para el desarrollo del simulador. En orden de aparición, el capítulo habla de: las herramientas y tecnologías usadas, el modelo cliente-servidor que se ha decidido modelar, la descripción de las plataformas y las acciones de la simulación, las políticas de acceso definidas para los clientes, las operaciones CRUD definidas y el desarrollo de unas interfaces que permitan modificar la simulación.

4.1. Tecnologías empleadas

El primer punto a argumentar son las tecnologías empleadas. Desde un punto de vista de diseño, es necesario explicar el uso de *SimGrid* como framework de simulación, el uso de *Gnuplot* como herramienta de graficado y los lenguajes de programación usados en el resto del software de este proyecto.

4.1.1. SimGrid

Como ya se ha explicado en el apartado [2.3. Simuladores](#), existe una gran variedad de simuladores en el panorama actual y *SimGrid* es uno de ellos. Surge entonces la pregunta: ¿por qué se elegiría *SimGrid* en vez de cualquier otro?

La respuesta se fundamenta en varias bases. La primera es que el proyecto se adecúa muy bien al paradigma de la computación en malla, de la que *SimGrid* es uno de los principales representantes a nivel de simuladores. Esto viene dado por la naturaleza propia de las comunicaciones entre clientes y servidores, así como el carácter de compartición de los recursos, ya que el servidor comparte un sistema de ficheros a todos los clientes que lo necesiten.

La segunda base sobre la que se fundamenta esta elección es la programabilidad y versatilidad que ofrece este framework. Muchos simuladores, como se ha visto en la tabla [2.3](#), no ofrecen una interfaz que permita realizar un simulador a un nivel tan personalizado como *SimGrid*. Su variedad de interfaces, de lenguajes de programación y su documentación [44] hacen que programar una aplicación simulada en *SimGrid* sea una experiencia más sencilla que en el resto de simuladores.

El tercer pilar sobre el que se fundamenta la elección de *SimGrid* como framework de simulación es el amplio número y variedad de programaciones científicas que han usado este framework para sus simulaciones. Como se ha mencionado en el apartado [2.3. Simuladores](#), existen muchos simuladores que son desarrollados por los investigadores para que cumplan con sus necesidades. Sin embargo, debido a la programabilidad que

ofrece, *SimGrid* se ha visto usado en una gran cantidad de publicaciones científicas [25], lo que además refuerza otras características del simulador, como unos resultados precisos o un tiempo de simulación adecuado.

4.1.2. Gnuplot

La herramienta *Gnuplot* se usa en este proyecto para generar una serie de gráficas tras el final de la simulación. Al igual que con *SimGrid*, existen una serie de alternativas que se podrían usar en este proyecto. En este apartado se exploran algunas herramientas de graficado y se argumenta el uso de *Gnuplot*.

Estudio de alternativas

Las alternativas a *Gnuplot* son extensas y, debido a esto, sus naturalezas varían ampliamente. Desde librerías de *Python* hasta programas con su propia interfaz gráfica, existe un amplio rango de opciones a considerar. Lo primero a hacer es, sin embargo, una enumeración de las características que se desean en la herramienta para graficar:

- **Facilidad de integración:** la herramienta debe ser fácilmente integrable con la simulación.
- **Facilidad de ejecución:** la herramienta debe ser fácilmente ejecutable.
- **Buen rendimiento:** la herramienta no debe tardar demasiado en generar los gráficos.
- **Resultados profesionales:** la herramienta debe producir gráficos con un resultado profesional.
- **Disponibilidad:** la disponibilidad de la herramienta se valorará positivamente, penalizando aquellas que usen licencias.
- **Multiplataforma:** la herramienta debe funcionar en distintos sistemas operativos.
- **Dependencias:** una gran cantidad de dependencias para usar la herramienta será negativo.

Teniendo estas características en mente, se puede comenzar a enumerar las alternativas más conocidas. La primera es el uso de librerías de *Python* como *Pandas* [45], *Seaborn* [46] y *Matplotlib* [47]. Estas librerías cumplen con requisitos como la facilidad de integración y de ejecución, así como la disponibilidad y la simpleza. Esta herramienta es usada en una gran variedad de ámbitos profesionales, desde el análisis de datos hasta la inteligencia artificial, así que se puede asumir que los resultados son profesionales. Sin duda, es una gran alternativa a *Gnuplot*. Su punto más débil, sin embargo, es su dependencia a

Python, y cualquier cambio en el lenguaje de programación o en las propias herramientas puede causar un problema en el código que genera las gráficas.

Otras alternativas son el uso de programas con sus propias interfaces, como *R* [48] o *Matlab* [49]. Estas herramientas multiplataforma proveen unos resultados altamente profesionales, pero no poseen otras características altamente valoradas en este proyecto. Por ejemplo, ninguna tiene una facilidad de integración tan grande como el resto de las alternativas, y el rendimiento puede dejar que desear a veces. Además, si bien *R* es gratis, *Matlab* necesita licencias para funcionar, lo que afecta mucho a su viabilidad como alternativa.

Si bien existen una variedad de alternativas mucho mayor, como el uso de *LaTeX* para realizar gráficas mediante código que se podría automatizar, o el uso de *JavaScript* y de algunas librerías como *D3.js* o *Chart.js*; las alternativas anteriormente presentadas son las herramientas con más adopción de uso en el entorno profesional. Es por ello que sólo se compararán estas herramientas con la mencionada *Gnuplot*.

Elección de Gnuplot

Una vez elicidadas unas alternativas para la herramienta de graficado, es necesario argumentar el uso de *Gnuplot* en vez de ellas. *Gnuplot* es una herramienta cuya integrabilidad está al nivel de las librerías de *Python*, ya que con un script se pueden generar gráficas con una calidad profesional, lo que adicionalmente le da una facilidad de ejecución enorme.

Además, el carácter *open-source* de *Gnuplot* garantiza la disponibilidad de la herramienta en el proyecto. Las dependencias se minimizan al solamente tener que instalar esta herramienta que, muy positivamente, es multiplataforma y se puede ejecutar sin problemas en varios sistemas operativos. Si bien *Gnuplot* puede tener una ligera barrera de entrada respecto a otras herramientas, la documentación es extensa, lo que facilita su aprendizaje correcto y rápido. Estos son los motivos por los que se ha elegido *Gnuplot* respecto a sus alternativas. Se presenta la tabla 4.1 para comparar estas herramientas.

TABLA 4.1.
HERRAMIENTAS DE GRAFICADO

Características	Seaborn	Matplotlib	R	Matlab	Gnuplot
Integración	Fácil	Fácil	Difícil	Difícil	Fácil
Ejecución	Fácil	Fácil	Difícil	Difícil	Fácil
Rendimiento	Rápido	Rápido	Medio	Medio-Lento	Muy rápido
Resultados	Profesional	Profesional	Profesional	Profesional	Profesional
Disponibilidad	Gratis	Gratis	Gratis	Freemium	Gratis
Multiplataforma	Si	Si	Si	Si	Si
Dependencias	Muchas	Muchas	Algunas	Add-ons	Pocas

4.1.3. Lenguajes de programación

El uso de los lenguajes de programación es muy directo en este proyecto, ya que solo hacen falta dos, *Python* y *C++*. Empezando por *Python*, este se ha elegido para programar las interfaces descritas en los apartados [3.2.1. Requisitos de usuarios](#) y [3.2.2. Requisitos de sistema](#). Este lenguaje se ha elegido dada su versatilidad y facilidad. Un posterior análisis de las interfaces explica más detalladamente por qué se ha elegido *Python*.

En cuanto a *C++*, este se usa para programar la simulación. Como se ha mencionado en varias ocasiones, *SimGrid* cuenta con varias interfaces. La interfaz *S4U*, mencionada en el [Requisito RS_NF_01](#), es la interfaz principal del framework *SimGrid* y, previsiblemente, está programada en *C++*. Debido a la gran cantidad de funciones, objetos e interfaces que esta contiene, se ha decidido usar *C++* para programar la simulación.

4.2. Modelo cliente-servidor

Una vez que se ha hablado de las tecnologías empleadas, es momento de hablar del modelo cliente-servidor que se sigue en la simulación. Este modelo se ha elegido ya que es una manera simple de representar la noción de un servicio distribuido como puede ser un sistema de ficheros.

Este modelo permite estudiar varios aspectos de la simulación, como el consumo de recursos de un servidor, el tiempo empleado en comunicaciones a través de la red, o el gasto de recursos necesarios para realizar un acceso seguro y paralelo a un servidor.

El modelo cliente-servidor se puede representar en la plataforma de la simulación con 4 tipos de entidades:

- **Cliente:** es una máquina conectada a un servidor a través de la red. Un cliente puede mandar operaciones CRUD a un servidor y obtener una respuesta del mismo indicando si la operación se ha podido llevar a cabo o no. Adicionalmente, un cliente puede enviar una operación para que el servidor realice un tiempo determinado de cómputo, influyendo así en el resultado de la simulación.
- **Servidor:** es una máquina a la que se conectan los clientes a través de la red. El servidor es el encargado de ejecutar las operaciones CRUD y de cómputo que le llegan a través de la red, así como de mantener un sistema de ficheros simulado.
- **Enlaces:** es una abstracción de la idea de un enlace físico que cuenta con parámetros como el ancho de banda o el retardo.
- **Rutas:** es un conjunto de enlaces que unen un host de origen con un host de destino.

Este modelo de comunicación es en lo que se fundamenta el núcleo de la simulación. Un principio de esta simulación es que no hay un límite en el número de clientes que se

pueden comunicar con un servidor y, por lo tanto, se tendrá que implementar una forma segura y paralela en la que los clientes puedan comunicarse con los servidores.

Un aspecto que preocupa de este modelo es su escalabilidad. Es por ello que se han ideado dos tipos de simulaciones que se pueden hacer sobre el simulador final, las cuales afectan directamente a la morfología de la plataforma: simulaciones pequeñas y simulaciones clusterizadas.

4.2.1. Simulación pequeña

Esta simulación sigue los principios explicados anteriormente, sin ningún cambio a las entidades de la plataforma o a las acciones que los clientes pueden enviar a los servidores. Gráficamente, una simulación pequeña puede tener la forma descrita en la figura 4.1.

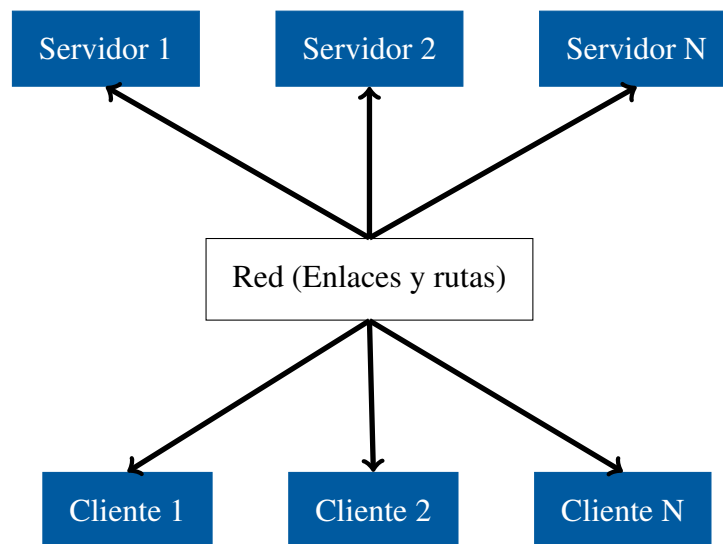


Fig. 4.1. Simulación pequeña

Esta simulación tiene, por lo tanto, clientes individuales que se comunican con servidores individuales. Este acercamiento tiene varios pros, el principal de ellos siendo su simplicidad; sin embargo, si se quiere realizar una simulación grande, con muchos servidores y muchos clientes, el escalado puede ser una tarea tediosa. Es por ello que se han ideado las simulaciones clusterizadas.

4.2.2. Simulación clusterizada

Las simulaciones clusterizadas son como las simulaciones pequeñas, ya que los actores son, a un nivel funcional, iguales. Sin embargo, aquí se sustituyen los clientes y los servidores por clusters de clientes y de servidores, respectivamente. De esta manera, se pueden definir un gran número de clientes o servidores que comparten la misma máquina host, es decir, las mismas propiedades. Así, este modelo logra una escalabilidad mucho

más rápida que las simulaciones pequeñas, donde se tendrían que definir uno a uno cada uno de los clientes y los servidores, y realizar rutas que conectasen a unos con otros.

Este enfoque, a pesar de no ser perfecto debido a que no se pueden definir múltiples clientes o servidores con características distintas, sí permite definir operaciones de manera individual a cada cliente o grupo de clientes de un cluster. Esta granularidad permite tratar cada cliente y cada servidor de cada cluster como si fuesen individuos; la única restricción es que comparten las características de la máquina donde están alojados. Gráficamente, una simulación clusterizada tiene la forma descrita en la figura 4.2.

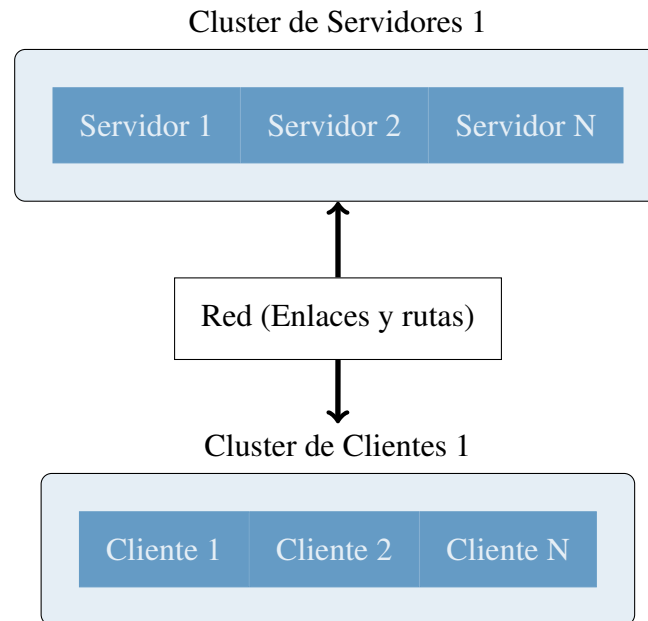


Fig. 4.2. Simulación clusterizada

Adicionalmente, los nodos internos de cada cluster se comunican cada uno con un bus, que se comunica con la salida del cluster a la red. Esto se representa en la figura 4.3:

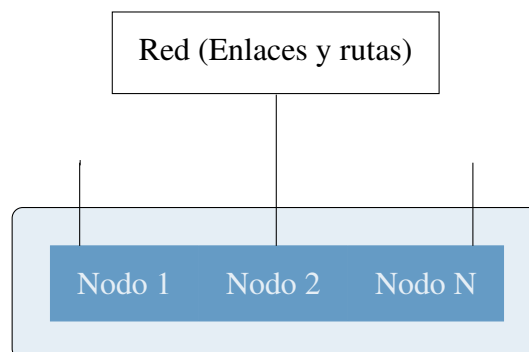


Fig. 4.3. Comunicación de los clusters

Ambos enfoques deberán ser implementados en la versión final de la simulación, dejando como responsabilidad al usuario saber cuándo usar cada tipo.

4.3. Plataforma y acciones en la simulación

Para implementar las plataformas y las acciones en la simulación, *SimGrid* ofrece un método que carga la información a partir de dos tipos de ficheros: los ficheros de plataforma y los ficheros de despliegue.

Los ficheros de plataforma definen la plataforma de la simulación mediante un fichero *XML* (Lenguaje de Marcado Extensible). Siguiendo los ejemplos presentados en las figuras 4.1 y 4.2, las plataformas se podrían representar mediante un *XML*. Por ejemplo, un servidor y un cliente unidos por una ruta se definen mediante el siguiente *XML*:

```

1  <platform version="4.1">
2    <zone id="zone_1" routing="Full">
3      <AS id="AS0" routing="Full">
4
5        <!-- Servidores -->
6        <host id="Servidor_1" speed="100Mf" />
7
8        <!-- Clientes -->
9        <host id="Cliente_1" speed="100Mf"/>
10
11       <!-- Red -->
12       <link id="Enlace_1" bandwidth="1000MBps" latency="5us"/>
13
14       <route symmetrical="YES" src="Servidor_1" dst="Cliente_1">
15         <link_ctn id="Enlace_1"/>
16       </route>
17     </AS>
18   </zone>
19 </platform>

```

Los ficheros de despliegue, en cambio, permiten definir las acciones de los actores de la simulación mediante otro fichero *XML*. Para ello, cada actor debe tener un host en la plataforma donde se pueda alojar. En el contexto de este proyecto, los únicos actores son los clientes, que deben mandar las operaciones a los servidores. Un ejemplo de los ficheros de despliegue podría ser el siguiente:

```

1  <platform version="4.1">
2
3    <!-- Clientes -->
4    <actor host="Cliente_1" function="cliente" />
5
6  </platform>

```

En estos ejemplos, se pueden observar que los actores y los hosts tienen una serie de parámetros, como la velocidad o la banda ancha. Estos parámetros obligatorios se explican en los apartados 5.2.1. *Servidor en los ficheros* y 5.3.1. *Cliente en los ficheros*.

4.4. Políticas

Tal y como se ha hablado en el [Requisito RS_FN_10](#), los clientes deben tener distintas políticas de acceso, que dictaminan la manera en la que los clientes interactúan con los servidores. Es por ello que se ha decidido implementar tres políticas distintas: secuencial, distribuida y Expand.

4.4.1. Política secuencial

En la política secuencial, el cliente simplemente especifica el servidor con el que se comunicará directamente, consiguiendo así una relación 1:1 entre el cliente y el servidor. Subsiguientemente, todas las operaciones se mandarán al servidor elegido y se realizarán sobre su sistema de ficheros simulado.

4.4.2. Política de replicación

La política de replicación es algo más compleja que la secuencial, ya que define una comunicación 1:N entre un cliente y varios servidores. El objetivo de esta política es replicar las operaciones de un cliente con un número predefinido de servidores, factor al que se le llamará “factor de replicación”.

Para simplificar las cosas, la replicación se hará de manera cíclica en los servidores, es decir, se comenzará por mandar una operación a un servidor en particular, y luego se mandará la operación a los servidores subsiguientes en orden ascendente, haciendo siempre el módulo entre el índice del servidor y el número de servidores para evitar mandar una operación a un servidor inexistente. Esto se realizará hasta que se hayan enviado tantas operaciones como indica el factor de replicación o como número de servidores haya, evitando así operaciones duplicadas. Visualmente, esta distribución circular se puede ver en la figura 4.4 que muestra un ejemplo donde el cliente manda una operación con un factor de replicación 3.

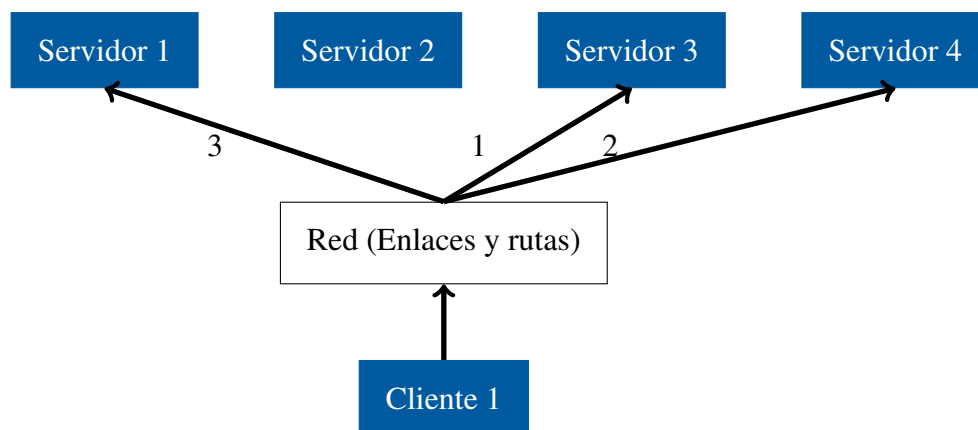


Fig. 4.4. Ejemplo de la política de replicación

En este ejemplo, primero se accede al servidor 3, luego al 4 y luego al 1. La forma de determinar el servidor al que se accede primero se puede realizar de forma semi-aleatoria, pudiendo así generar el mismo orden cada vez que se quiera mandar una operación. La decisión que se tomó para implementar esto se explica en el apartado [5.3.3. Políticas](#).

4.4.3. Expand

Expand es la última política a implementar, explicada en detalle en el paper “A new Ad-Hoc parallel file system for HPC environments based on the Expand parallel file system” [50]. De manera resumida, esta política permite dividir un fichero en varios sub-ficheros que son distribuidos entre los distintos servidores. Además, se genera un fichero de metadatos que contiene información sobre el orden de estos sub-ficheros, el servidor donde se aloja cada sub-fichero o el tamaño de partición en el que se divide el fichero.

Por ejemplo, en un sistema con tres servidores y un cliente, si se establece un tamaño de partición de 200 Bytes y el cliente querría escribir 700 Bytes sobre un fichero, el resultado en los servidores sería el mostrado en la figura [4.5](#).

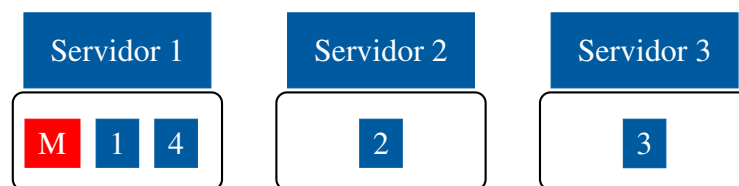


Fig. 4.5. Ejemplo de la política Expand

En este ejemplo se puede ver que el fichero de metadatos y el primer sub-fichero se almacenan en el primer servidor. Además, el orden de los servidores se puede elegir de cuatro maneras [50] pero, por cuestiones de simplicidad del sistema final, sólo se implementarán 3: aleatorio, elegido por el usuario y cíclico. Estas estrategias de distribución se explicarán más en profundidad en el apartado [5.3.3. Políticas](#).

4.5. Operaciones

Como se ha mencionado a lo largo del documento, los clientes pueden mandar operaciones a los servidores. Estas son operaciones CRUD sobre un sistema de ficheros, a las que se ha decidido agregar una operación para simular un tiempo de cómputo. Granularmente, las operaciones se definen en los siguientes apartados.

4.5.1. Crear un fichero

La primera operación a definir es la de crear un fichero, identificada por la palabra clave “CREATE”. El cliente sólo tendrá que mandar el nombre del fichero a crear al servidor. El flujo de la operación se visualiza en el diagrama de secuencia de la figura 4.6.

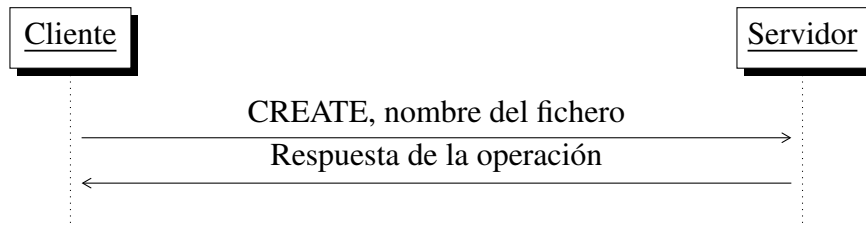


Fig. 4.6. Diagrama de flujo de CREATE

4.5.2. Leer un fichero

La segunda operación a definir es la de leer un fichero, identificada por la palabra clave “READ”. El cliente sólo tendrá que mandar el nombre del fichero a leer al servidor. El flujo de la operación se visualiza en el diagrama de secuencia de la figura 4.7.

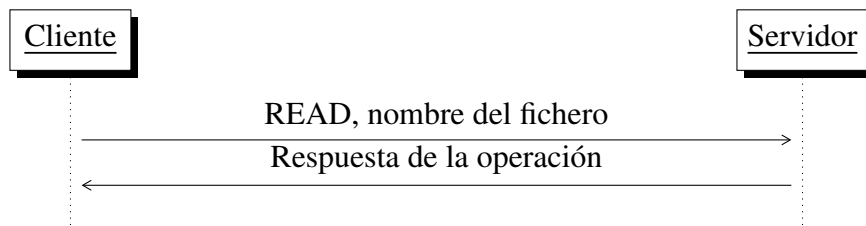


Fig. 4.7. Diagrama de flujo de READ

4.5.3. Escribir un fichero

La tercera operación a definir es la de crear un fichero, identificada por la palabra clave “WRITE”. El cliente sólo tendrá que mandar el nombre del fichero a escribir y el número de bytes que quiere escribir al servidor. El flujo de la operación se visualiza en el diagrama de secuencia de la figura 4.8.

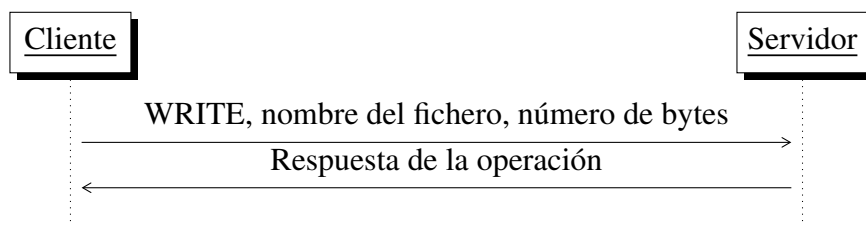


Fig. 4.8. Diagrama de flujo de WRITE

4.5.4. Eliminar un fichero

La cuarta operación a definir es la de eliminar un fichero, identificada por la palabra clave “DELETE”. El cliente sólo tendrá que mandar el nombre del fichero a eliminar al servidor. El flujo de la operación se visualiza en el diagrama de secuencia de la figura 4.9.

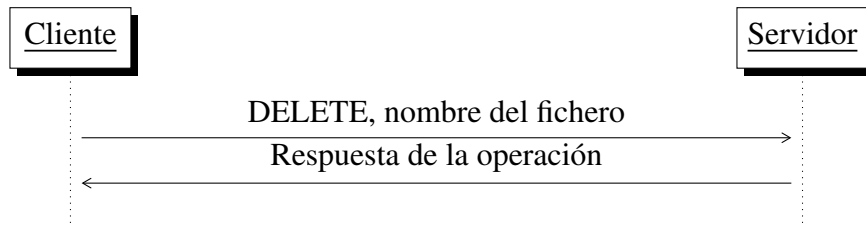


Fig. 4.9. Diagrama de flujo de DELETE

4.5.5. Simular cómputo

La última operación a definir es la de simular un tiempo de cómputo, identificada por las palabras clave “COMPUTE_TIME”. El cliente sólo tendrá que mandar el tiempo que durará la tarea de cómputo del servidor. El flujo de la operación se visualiza en el diagrama de secuencia de la figura 4.10.

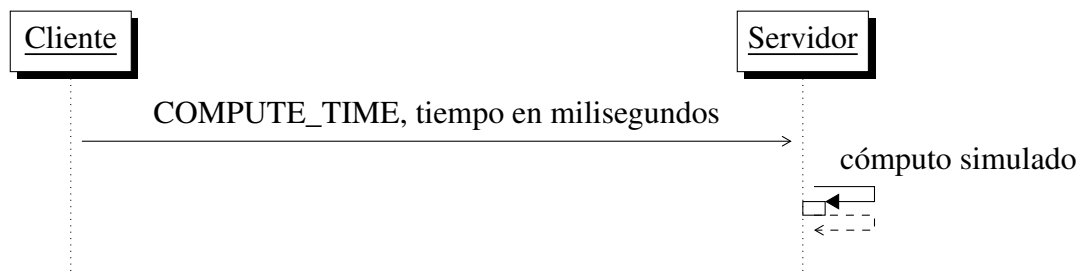


Fig. 4.10. Diagrama de flujo de COMPUTE_TIME

4.5.6. Ficheros de operaciones

Dada la compleja legibilidad de los ficheros de despliegue, y por razones de escalabilidad, las operaciones se definirán en ficheros. Estos ficheros podrán ser asignados a uno o varios clientes, y contendrán tanto las operaciones a realizar como la política de acceso que los clientes usarán en la comunicación.

4.6. Interfaces

Como pequeño resumen, los ficheros de la simulación son: los ficheros de plataforma, los ficheros de despliegue y los ficheros de operaciones. Ya que escribir todos estos ficheros a mano sería una tarea muy lenta, y de acuerdo con el [Requisito RS_FN_01](#) y con el [Requisito RS_FN_08](#), en este proyecto se deberá implementar una serie de interfaces que permitan la modificación controlada de estos ficheros.

Debido a la limitación temporal de este proyecto, las interfaces serán interfaces de líneas de comandos. Estas interfaces serán implementadas en *Python*, con ayuda de la librería *inquirer* [51], que sirve para pedir input de los usuarios de una manera más refinada y ágil. Adicionalmente, y de acuerdo con el [Requisito RS_NF_05](#) y con el [Requisito RS_NF_06](#), las interfaces de usuario deberán seguir los principios de diseño y las heurísticas de Nielsen para mejorar la experiencia de usuario.

5. IMPLEMENTACIÓN Y DESPLIEGUE

Tras explicar las decisiones de diseño que han formado parte del proyecto, es momento de explicar la implementación y el despliegue del simulador. Para ello, primero se expondrá la estructura del proyecto, tras lo que se explicará detalladamente cómo funciona la simulación pasando por los servidores y por los clientes. Tras esto, se hablará de cómo se ha integrado *Gnuplot* dentro de este simulador, prosiguiendo con una explicación de las interfaces y finalizando este apartado con una guía para desplegar o ejecutar el proyecto.

5.1. Estructura del proyecto

Lo primero para entender la implementación del simulador es entender cuál es la estructura del proyecto. Así, se muestra el resultado de ejecutar el comando *tree* sobre la raíz del proyecto:

```
1  .
2  |-- README.md
3  |-- docs
4  \-- src
5      |-- CLI.py
6      |-- CLIs
7          |-- clusterized_CLI.py
8          |-- small_CLI.py
9          \-- templates_CLI.py
10     |-- CMakeLists.txt
11     |-- build
12     |-- cmake
13     |-- cmake_install.cmake
14     |-- functions
15         |-- replication.cpp
16         |-- replication.h
17         |-- sequential.cpp
18         |-- sequential.h
19         |-- server.cpp
20         \-- server.h
21     |-- main.cpp
22     |-- requirements.txt
23     |-- scripts
24         |-- compile.sh
25         |-- run.sh
26     |-- simulation
27         |-- clusterized_config.json
28         |-- clusterized_deployment.xml
29         |-- clusterized_platform.xml
```

```
30 | |-- small_config.json
31 | |-- small_deployment.xml
32 | \-- small_platform.xml
33 \-- templates
34     |-- clusterized
35     \-- small
```

En el anterior código, se pueden distinguir varios directorios. En el siguiente listado se explica la utilidad de cada uno de ellos:

- **docs:** este directorio almacena la documentación del proyecto, como la memoria o algunos documentos de la bibliografía.
- **src:** este directorio almacena todo el código del proyecto.
- **CLIs:** este directorio almacena el código de las interfaces. Como se puede observar, hay tres ficheros dentro de esta carpeta, y un fichero exterior denominado “CLI.py”. Esta organización del código de las interfaces se explicará en profundidad en la sección [5.5. Interfaces](#).
- **build:** almacena los ejecutables una vez ya han sido compilados. En el proyecto final, el código fuente y este directorio se borrarían, dejando solamente los ejecutables.
- **functions:** este directorio almacena varios ficheros de C++ con funciones relacionadas con el servidor o con las políticas de acceso de los clientes.
- **scripts:** este directorio almacena los scripts necesarios para compilar y ejecutar el simulador.
- **simulation:** este directorio almacena los ficheros de información de la simulación. Como se puede observar, se define un fichero de plataforma, uno de despliegue y uno de configuración tanto para las simulaciones pequeñas como para las clusterizadas.
- **templates:** este directorio almacena los ficheros que listan las operaciones y las políticas usadas por los clientes. Aquí, de nuevo, se hace una distinción entre los ficheros para una simulación pequeña y aquellos de las simulaciones clusterizadas.

El resto de ficheros no explicados generalmente sirven para compilar, como es el caso de *CMakeLists.txt* o de *cmake*; o sirven para informar al usuario del proyecto, como es el caso de *README.md*. Una vez que se tiene esta distribución de los ficheros del proyecto en mente, se puede comenzar a describir cómo funciona la simulación.

5.2. Servidores

Entender cómo funcionan los servidores de la simulación es el primer paso para entender cómo funciona la simulación. Para entender su funcionamiento, se seguirán los pasos necesarios para incluir un servidor en los ficheros de plataforma y despliegue, tras lo que se explicará cómo funciona el código de los servidores. Una vez explicado esto, se detallará cómo se han implementado las operaciones CRUD y de cómputo en el servidor, finalizando esta sección con una explicación de cómo se mide el tiempo y la energía consumida en cada servidor.

5.2.1. Servidor en los ficheros

El primer paso para cualquier simulación que se quiera llevar a cabo es definir la plataforma y el despliegue de los actores. Hay que hacer un inciso, ya que el servidor se define de manera distinta dependiendo de si la simulación es pequeña y clusterizada, así que se presentará siempre un ejemplo por cada tipo de simulación.

Un servidor se define en la plataforma para una simulación pequeña y clusterizada, respectivamente, de ls siguientes formas:

```

1  <!--Simulación pequeña-->
2  <host id="Servidor_1" speed="100Mf">
3    <prop id="Disco" value='{ "type": "SSD", "read_speed": "250MB/s", "write_speed":
      "230MB/s", "cache_data": "10%", "cache_metadata": "10%" }' />
4    <prop id="wattage_per_state" value="100.0:120.0"/>
5    <prop id="wattage_off" value="10.0"/>
6  </host>
7
8  <!--Simulación clusterizada-->
9  <cluster id="Cluster_Servidores_1" prefix="Servidor_" radical="1-10" suffix=" (
      Cluster_Servidores_1)" speed="1Gf" bw="125MBps" lat="50us" bb_bw="2GBps"
      bb_lat="500us">
10    <prop id="Disco" value='{ "type": "SSD", "read_speed": "250MB/s", "
      write_speed": "230MB/s", "cache_data": "10%", "cache_metadata": "10%" }',
      />
11    <prop id="wattage_per_state" value="100.0:120.0"/>
12    <prop id="wattage_off" value="10.0"/>
13  </cluster>

```

Como se puede observar, los servidores contienen mucha más información que en el ejemplo presentado en el apartado [4.3. Plataforma y acciones en la simulación](#). Esto se debe a que, para representar en la simulación el acceso a un disco y para modelar el consumo de la energía en los servidores, es necesario definir una serie de *props* adicionales. Mientras que los *props* con ID “wattage_per_state” y “wattage_off” tienen su origen en un plug-in que *SimGrid* ofrece para medir la energía consumida, el *prop* con ID “Disco”

ha sido definido para este proyecto para modelar una unidad de almacenamiento secundario. En la tabla 5.1 se puede consultar el significado de cada uno de los parámetros de cada *prop*.

TABLA 5.1.
ELEMENTOS COMUNES DE UN SERVIDOR EN LA
PLATAFORMA

Nombre	<i>Prop</i>	Descripción	Valores
type	Disco	Representa el tipo de disco que usa el servidor en la simulación	SSD, HDD, SSHD
read_speed	Disco	Representa la velocidad en <i>MB/s</i> de lectura del disco	Entero >0
write_speed	Disco	Representa la velocidad en <i>MB/s</i> de escritura del disco	Entero >0
cache_data	Disco	Representa la tasa de fallos a la caché de datos	100 >Entero >0
cache_metadata	Disco	Representa la tasa de fallos a la caché de metadatos	100 >Entero >0
value	wattage_per_state	Representa el consumo en julios de un servidor en dos estados: <i>idle</i> y máximo rendimiento	Dos números flotantes >0
value	wattage_off	Representa el consumo en julios de un servidor cuando está apagado	Número flotante >0

Mientras que el contenido de cada servidor es igual, las definiciones de las etiquetas *host* y de *cluster* son radicalmente distintas. En las simulaciones pequeñas, sólo hay que definir la velocidad del host en *MFlops*, mientras que un cluster tiene muchos más parámetros. En la tabla 5.2 se definen los atributos específicos a un cluster de servidores.

TABLA 5.2.
ELEMENTOS DE UN CLUSTER DE SERVIDORES EN LA
PLATAFORMA

Nombre	Descripción	Valores
prefix	Define el prefijo que todos los servidores del cluster llevarán en su <i>ID</i>	Siempre será “Servidor_”
radical	Define el número de servidores del cluster. Además, el valor numérico asignado a cada servidor se colocará entre el prefijo y el sufijo para formar su <i>ID</i>	Dos enteros >0 ó 1
sufix	Define el sufijo de todos los servidores del cluster llevarán en su <i>ID</i>	Siempre coincidirá con el <i>ID</i> del cluster
speed	Define la velocidad del cluster en <i>GFlops</i>	Entero >0
bw	Define la banda ancha de los cables que conectan cada servidor con el bus del cluster (4.3) en <i>MBps</i>	Entero >0
lat	Define la latencia de los cables que conectan cada servidor con el bus del cluster (4.3) en <i>us</i>	Entero >0
bb_bw	Define la banda ancha del bus del cluster (4.3)	Entero >0
bb_lat	Define la banda ancha del bus del cluster (4.3)	Entero >0

Una vez que se ha explicado cómo se representan los servidores en la plataforma, es necesario saber cómo se representan en el fichero de despliegue. Para ello, se muestra el siguiente *XML* de ejemplo:

```

1  <!--Simulación pequeña-->
2  <actor host="Servidor_1" function="servidor">
3      <argument value="Servidor_1"/>
4  </actor>
5
6  <!--Simulación clusterizada-->
7  <actor host="Servidor_1 (Cluster_Servidores_1)" function="servidor">
8      <argument value="Servidor_1 (Cluster_Servidores_1)"/>
9  </actor>
10 <actor host="Servidor_2 (Cluster_Servidores_1)" function="servidor">
11     <argument value="Servidor_2 (Cluster_Servidores_1)"/>
12 </actor>
13 <actor host="Servidor_3 (Cluster_Servidores_1)" function="servidor">
14     <argument value="Servidor_3 (Cluster_Servidores_1)"/>
15 ...

```

Como se puede observar, la única diferencia entre los servidores de una simulación pequeña y los de una simulación clusterizada es que, en la clusterizada, hay tantos actores como indique el rango especificado en el cluster. Ambos actores se despliegan sobre el host adecuado y ambos tienen un parámetro *function*, que simplemente indica el nombre de la función del código principal que define el comportamiento de estos actores.

Ambas definiciones de los servidores reciben, también, un argumento. Este argumento es el nombre del *mailbox* que utilizarán. Los *mailboxes* son objetos de *SimGrid* que permiten que los servidores y los clientes se comuniquen, ya que cada uno podrá escribir los mensajes que quieran en el *mailbox* que quieran, para lo que sólo necesitan saber el nombre que tiene dicho *mailbox*.

5.2.2. Núcleo del servidor

Una vez se ha explicado cómo se representan los servidores en los ficheros, se puede explicar qué funciones ejecutan los servidores dentro de la simulación. Como se ha explicado en el apartado anterior, los servidores reciben un parámetro, *function*, que determina el nombre de la función que deben ejecutar. Esta función se describe dentro del fichero *main.cpp* con la siguiente cabecera:

```
1  static void servidor(int argc, char** argv) {  
2      /*  
3      Función principal del servidor que maneja las peticiones de conexión  
4  
5      :param 1: (argc) número de argumentos  
6      :param 2: (argv[1]) nombre del mailbox del servidor  
7      :return: void  
8      */  
9  }
```

Esta función sirve como el punto de inicio para la comunicación con los clientes, ya que el servidor abre su *mailbox* con el nombre que recibe como argumento y espera a que los clientes escriban a ese *mailbox* para iniciar una sesión con ellos. Para hacer esto de manera paralela, cuando el servidor recibe un mensaje de conexión, lanza un hilo que se encarga de procesar y responder las operaciones de esta sesión con el cliente y le pasa como argumento el primer mensaje que ha recibido del cliente, que siempre será el nombre del *mailbox* del cliente. El hilo finalizará cuando reciba la operación especial “END” del cliente, que se la mandará cuando haya terminado de enviarle todas sus operaciones. Se presenta la figura 5.1 como una explicación visual.

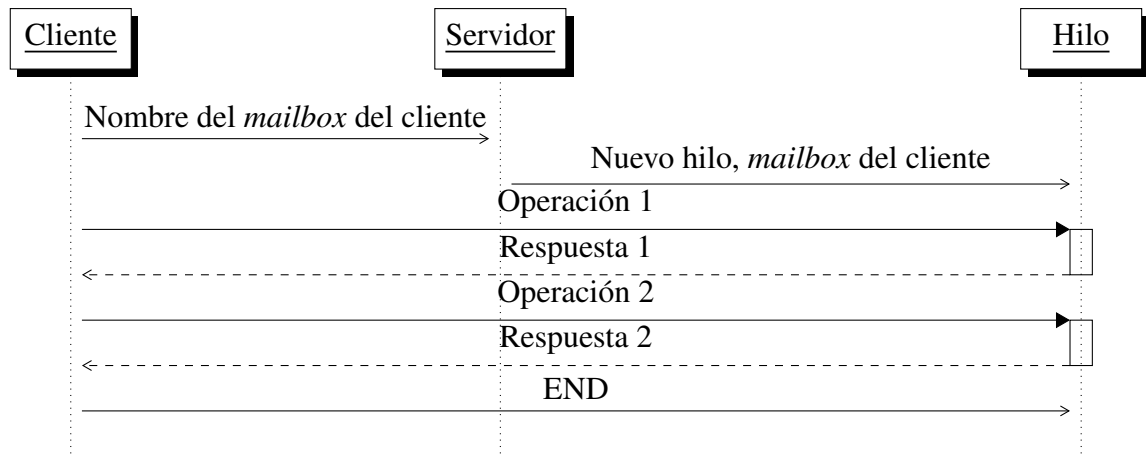


Fig. 5.1. Diagrama de flujo del servidor

El comportamiento del hilo se define en la función *process_operations*, que tiene la siguiente cabecera:

```

1  static void process_operations(std::string client_mailbox) {
2      /*
3      Cliente que procesa la comunicación hilo-cliente
4
5      :param 1: nombre del mailbox del cliente
6      :return: void
7      */
8  }
```

Esta función se encarga de: abrir el *mailbox* del cliente usando el nombre que se le ha pasado como parámetro, conseguir los parámetros de los servidores, explicados en el apartado 5.2.1. *Servidor en los ficheros*; e iniciar un bucle infinito que recibe las operaciones del cliente y las procesa llamando a las funciones adecuadas. Además, cada vez que realiza una operación, actualiza el tiempo consumido por el servidor modificando un mapa definido como una variable global, donde las claves son los nombres de los servidores y cada valor es otro mapa donde se guarda el tiempo consumido.

El servidor estará corriendo hasta que reciba un mensaje de un actor especial que está presente en todas las simulaciones le mande un mensaje para parar. Este actor especial se denomina *Finisher*, ya que su única acción es mandar un mensaje de finalización a todos los servidores de la simulación. El *Finisher* tiene una función asignada en el fichero de despliegue llamada *finalizador*, y cuenta con la siguiente definición:

```

1  static void finalizador(int argc, char **argv) {
2      /*
3      Función que se encarga de acabar con el servidor una vez se hayan comunicado
4      todos los clientes
5
6      :param 1: (argc) número de argumentos, en este caso, 0
7      :return: void
8  }
```

```

7      */
8  }

```

Esta función se asegura de que el *Finisher* se ejecute siempre cuando acaben todos los clientes empleando un *mútex* y una *variable de condición* para asegurar este orden. El *Finisher* siempre esperará a que el número de clientes llegue a cero y, mientras este número sea mayor que cero, esperará bloqueando la variable de condición. Cada vez que un cliente acabe, llamará a la siguiente función, que se encarga de disminuir el contador de clientes por ejecutarse:

```

1  static void finalizar_actor() {
2      /*
3      Función auxiliar que se encarga de reducir en 1 el número de actores cuando
4      finaliza un cliente
5
6      :return: void
7      */
8      mutex_finisher->lock();
9      num_actors--;
10     cv_finisher->notify_one();
11     mutex_finisher->unlock();
12 }

```

El número de clientes a ejecutar se inicializa a un valor presente en los ficheros de configuración de la simulación (*small_config.json* y *clusterized_config.json*), que varía cada vez que se realiza un cambio a los ficheros de despliegue. Cuando se hayan ejecutado todos los clientes, el *Finisher* enviará el mensaje “STOP_SERVER” a todos los servidores.

5.2.3. Operaciones

Las operaciones de los servidores se definen en el fichero *functions/server.cpp*. El objetivo de definir las operaciones en un fichero distinto es proporcionar una especie de interfaz, de manera que el servidor sólo tiene que llamar a las funciones de dicha interfaz cuando reciba una operación del cliente. Esto se visualiza en la figura 5.2.

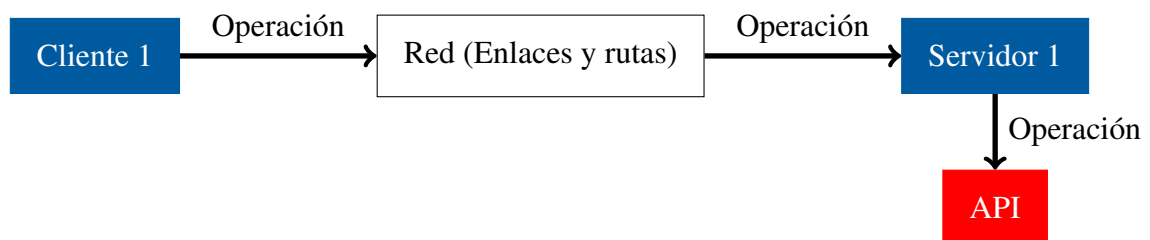


Fig. 5.2. Llamadas a la API de operaciones

A cada operación siempre se le pasa: el *mailbox* del cliente para que escriba directamente la respuesta en él, la velocidad del servidor para simular el tiempo de acceso a la caché, que se explicará detalladamente en el siguiente apartado; el path a un fichero temporal, el cuál se crea durante la simulación para guardar los ficheros (y su tamaño) que cada cliente ha ido creando en cada servidor, simulando efectivamente un sistema de ficheros; y la información del disco. Todas las operaciones se definen en el fichero *functions/server.h* con el siguiente código:

```

1  unsigned long int server_create(sg4::Mailbox* c_mailbox, unsigned long int speed,
    std::string control_file_path, json disk_info, sg4::MutexPtr mutex);
2  unsigned long int server_read(sg4::Mailbox* c_mailbox, unsigned long int speed,
    std::string control_file_path, json disk_info, sg4::MutexPtr mutex);
3  unsigned long int server_write(sg4::Mailbox* c_mailbox, unsigned long int speed,
    std::string control_file_path, json disk_info, sg4::MutexPtr mutex);
4  unsigned long int server_delete(sg4::Mailbox* c_mailbox, unsigned long int speed,
    std::string control_file_path, json disk_info, sg4::MutexPtr mutex);
5  unsigned long int server_compute_time(sg4::Mailbox* c_mailbox, unsigned long int
    speed);

```

Ahora, se procede a explicar la implementación de cada operación mediante la descripción de su algoritmo.

Algoritmo de creación de ficheros (CREATE)

1. Consigue los datos del disco.
2. Consigue el path del fichero leyendo el mensaje que le ha pasado el cliente.
3. Abre el fichero de control del servidor y lee los metadatos.
 - a) Si el fichero no se ha abierto bien, devuelve un error.
 - b) Si el fichero se ha abierto bien, simula un tiempo de lectura de los metadatos *.(5.2.4. Medición de tiempo y energía)* y continúa.
4. Comprueba si el fichero existe.
 - a) Si el fichero existe, devuelve un error.
 - b) Si el fichero no existe, simula un tiempo de escritura de los metadatos del fichero *.(5.2.4. Medición de tiempo y energía)* y continúa.
5. Escribe los datos en el fichero de control y devuelve un mensaje de éxito.

Algoritmo de lectura de ficheros (READ)

1. Consigue los datos del disco.

2. Consigue el path del fichero leyendo el mensaje que le ha pasado el cliente.
3. Abre el fichero de control del servidor y lee los metadatos.
 - a) Si el fichero no se ha abierto bien, devuelve un error.
 - b) Si el fichero se ha abierto bien, simula un tiempo de lectura de los metadatos.(5.2.4. *Medición de tiempo y energía*) y continúa.
4. Comprueba si el fichero existe.
 - a) Si el fichero no existe, devuelve un error.
 - b) Si el fichero existe, simula un tiempo de lectura de los datos del fichero .(5.2.4. *Medición de tiempo y energía*) y continúa.
5. Devuelve un mensaje de éxito.

Algoritmo de escritura de ficheros (WRITE)

1. Consigue los datos del disco.
2. Consigue el path del fichero y la longitud a escribir en bytes leyendo el mensaje que le ha pasado el cliente.
3. Abre el fichero de control del servidor y lee los metadatos.
 - a) Si el fichero no se ha abierto bien, devuelve un error.
 - b) Si el fichero se ha abierto bien, simula un tiempo de lectura de los metadatos .(5.2.4. *Medición de tiempo y energía*) y continúa.
4. Comprueba si el fichero existe.
 - a) Si el fichero no existe, devuelve un error.
 - b) Si el fichero existe, simula un tiempo de escritura de los datos (5.2.4. *Medición de tiempo y energía*) y continúa.
5. Escribe los datos en el fichero de control y devuelve un mensaje de éxito.

Algoritmo de borrado de ficheros (DELETE)

1. Consigue los datos del disco.
2. Consigue el path del fichero leyendo el mensaje que le ha pasado el cliente.
3. Abre el fichero de control del servidor y lee los metadatos.
 - a) Si el fichero no se ha abierto bien, devuelve un error.

- b) Si el fichero se ha abierto bien, simula un tiempo de lectura de los metadatos.(5.2.4. *Medición de tiempo y energía*) y continúa.
- 4. Comprueba si el fichero existe.
 - a) Si el fichero no existe, devuelve un error.
 - b) Si el fichero existe, simula un tiempo de escritura de los metadatos (5.2.4. *Medición de tiempo y energía*) y continúa.
- 5. Borra los datos del fichero de control y devuelve un mensaje de éxito.

Algoritmo de simulación de cómputo (COMPUTE_TIME)

1. Consigue el tiempo que tiene que computar leyendo el mensaje que le ha pasado el cliente.
2. Simula una tarea de cómputo haciendo los cálculos adecuados 5.2.4. *Medición de tiempo y energía*
3. Devuelve un mensaje de éxito.

5.2.4. Medición de tiempo y energía

Un objetivo de la simulación mencionado en repetidas ocasiones es obtener unas medidas de consumo razonablemente precisas, pero cuya complejidad de medición no sea lo suficientemente grande como para ralentizar la simulación. Este apartado explica cómo se obtiene dichas medidas de una manera eficiente y precisa, empezando por el tiempo y continuando por la energía.

Medición de tiempo

Como se ha mencionado al hablar del núcleo del código de los servidores, 5.2.2. *Núcleo del servidor* y al describir las operaciones en el apartado 5.2.3. *Operaciones*, cada vez que se realiza una operación se simula el tiempo de la operación y se actualiza en una variable global. Lo que interesa saber es cómo se simula el tiempo y, para ello, hay que recordar varios aspectos de cómo se han definido los servidores en este simulador:

1. Los servidores tienen parámetros relacionados con el disco 5.2.1. *Servidor en los ficheros*
2. El procesamiento de las operaciones se realiza en un hilo 5.2.2. *Núcleo del servidor*
3. Los servidores cuentan con ficheros temporales que guardan la información de los ficheros simulados 5.2.3. *Operaciones*

Con estos tres conceptos en mente, saber cuánto tiempo dura cada operación y simular dicho tiempo se reduce a dos procesos, el cálculo de la duración de la operación y el pausado del hilo durante ese tiempo. El cálculo del tiempo es simple, con:

- t : Tiempo de la operación
- s : Tamaño del bloque a leer/escribir
- f_c : Tasa de fallos a la respectiva caché
- v_c : Velocidad de lectura/escritura de la respectiva caché
- v_d : Velocidad de lectura/escritura del disco

$$t = (1 - f_c) * \left(\frac{s}{v_c}\right) + f_c * \left(\frac{s}{v_d}\right)$$

De esta fórmula se han extraído aspectos como la conversión de las variables a sus respectivas unidades por que esta conversión se hace mediante funciones externas en el código del simulador. Una vez que se tiene el tiempo calculado, el resto es una cuestión de usar la función `sg4::this_actor::execute()` de la interfaz de *SimGrid*, que permite simular un tiempo de uso de CPU del servidor. El único problema que tiene esta función es que no recibe un tiempo a simular, sino el tamaño de una tarea en número de operaciones. Por ello, la velocidad del servidor en *MFlops* se usa para calcular el tamaño de la tarea mediante la siguiente fórmula:

$$Size = Time * Speed$$

Como se ha podido intuir, la operación “COMPUTE_TIME” es más simple ya que recibe el tiempo a simular, por lo que solo necesita usar la última fórmula y la función `sg4::this_actor::execute()` para ser simulada. Este modelo de medición de tiempo se toma un par de licencias para simplificar la complejidad del simulador: la velocidad de lectura y escritura de la caché se aproxima a la velocidad del servidor, que debido a la magnitud de cada dato, no es una aproximación imprecisa; y las lecturas y escrituras a las cachés de metadatos asumen un tamaño de bloque de 1024 Bytes.

Medición de energía

La medición de la energía es muy simple, ya que se usa el *plug-in* de *SimGrid* [52] para calcularla. Para ello, sólo se necesitan los parámetros energéticos definidos en la tabla 5.1 y *SimGrid* se encargará de calcular el consumo energético de cada servidor. Cuando el servidor recibe la operación “STOP_SERVER” del *Finisher*, obtiene la energía consumida llamando a la función `sg_host_get_consumed_energy()` que recibe como parámetro el host del que se quiere obtener la energía. Luego, simplemente actualiza la energía en el mapa global anteriormente mencionado.

5.3. Clientes

Una vez se han explicado los servidores, es momento de hablar sobre la implementación de los clientes. Al igual que con los servidores, para entender su funcionamiento se explica la representación de los clientes en los ficheros de *SimGrid*, luego se detalla cómo funciona el código del cliente dentro de la simulación y se finaliza esta sección hablando sobre las políticas de acceso implementadas.

5.3.1. Cliente en los ficheros

Como se ha mencionado anteriormente, el primer paso de las simulaciones es la definición de la plataforma y del despliegue. En este caso, la complejidad de las representaciones de los clientes es inversa a la de los servidores, ya que es mucho más complejo representar un cliente en el fichero de despliegue que en el fichero de plataforma.

Para representar un cliente en un fichero de plataforma, para una simulación pequeña y una clusterizada respectivamente, se puede usar un algo parecido al siguiente *XML*:

```

1  <!--Simulación pequeña-->
2  <host id="Cliente_1" speed="200Mf"/>
3
4  <!--Simulación clusterizada-->
5  <cluster id="Cluster_Clientes_1" prefix="Cliente_" radical="1-10" suffix=" (
    Cluster_Clientes_1)" speed="200Mf" bw="200MBps" lat="10us" bb_bw="200MBps"
    bb_lat="10us"/>

```

Como se puede intuir, cada atributo de cada etiqueta del cluster de clientes significa lo mismo que los atributos de los servidores, explicados en la tabla 5.2, además del ya común *speed*. Por este motivo, la representación de los clientes en el fichero de la plataforma es simple, algo que no pasa en los ficheros de despliegue, ejemplificado en el siguiente código:

```

1  <!--Simulación pequeña-->
2  <actor host="Cliente_1" function="cliente">
3    <argument value="Cliente_1"/>
4    <argument value="MANUAL"/>
5    <argument value="SEQUENTIAL"/>
6    <argument value="Servidor_1"/>
7  </actor>
8  <actor host="Cliente_2" function="cliente">
9    <argument value="Cliente_2"/>
10   <argument value="FILE"/>
11   <argument value="./templates/small/prueba1"/>
12 </actor>
13
14 <!--Simulación clusterizada-->

```

```
15 <actor host="Cliente_1 (Cluster_Clientes_1)" function="cliente">
16   <argument value="Cliente_1 (Cluster_Clientes_1)"/>
17   <argument value="FILE"/>
18   <argument value="./templates/clusterized/expand"/>
19 </actor>
20
21 <actor host="Cliente_2 (Cluster_Clientes_1)" function="cliente">
22   <argument value="Cliente_2 (Cluster_Clientes_1)"/>
23   <argument value="FILE"/>
24   <argument value="./templates/clusterized/expand"/>
25 </actor>
26
27 <actor host="Cliente_3 (Cluster_Clientes_1)" function="cliente">
28   ...
```

En este ejemplo hay varios aspectos que mencionar. El primero de ellos es que, como se puede intuir, el parámetro *function* de los actores pasa a ser “cliente”, una función nueva con el código que define el comportamiento de los clientes. El segundo aspecto a comentar son los argumentos que reciben los clientes, que son drásticamente distintos a los que reciben los servidores.

El primer argumento es el nombre del *mailbox* que usará el cliente para sus comunicaciones con los distintos servidores y que le pasará siempre como primer mensaje al servidor correspondiente. El segundo argumento indica si las operaciones que recibe están indicadas en el propio fichero de despliegue (“MANUAL”) o si, en cambio, están definidas en un fichero de operaciones (“FILE”).

El resto de argumentos depende, por lo tanto, del modo de definición de las operaciones. Para las operaciones definidas por un fichero, el tercer y último argumento es el path relativo a dicho fichero de operaciones. En cambio, para las operaciones manuales, el tercer argumento es la política de acceso que el cliente quiere emplear en sus comunicaciones. El orden del resto de argumentos depende completamente de la política elegida. A modo de resumen:

- **Secuencial:** el cuarto argumento es el servidor al que se quieren mandar las operaciones, el resto de argumentos son las operaciones a mandar con sus respectivos argumentos.
- **Distribuida:** el cuarto argumento es el factor de replicación, el resto de argumentos son las operaciones a mandar con sus respectivos argumentos.
- **Expand:** el cuarto argumento es la estrategia de distribución que, a su vez, afecta al orden del resto de argumentos:
 - **Distribución cíclica:** el quinto argumento es el factor de partición, el resto de argumentos son las operaciones a mandar con sus respectivos argumentos.

- **Distribución aleatoria:** el quinto argumento es el factor de partición, el resto de argumentos son las operaciones a mandar con sus respectivos argumentos.
- **Distribución selectiva:** el quinto argumento es el factor de partición, el resto de argumentos es, primero, una lista de servidores en el orden definido por el usuario; y segundo, las operaciones a mandar con sus respectivos argumentos.

El significado de los argumentos y de las políticas se explican en el subapartado [5.3.3. Políticas](#). La opción de definición de operaciones manualmente, pese a ser más cómoda al poder definir las operaciones de manera rápida a través de la interfaz, es menos escalable, como se puede deducir debido al alto número de argumentos y a la variabilidad de los mismos. Para reducir la complejidad de los XML, las operaciones manualmente definidas sólo están disponibles en las simulaciones pequeñas, mientras que la definición mediante ficheros de operación será soportada en ambos tipos de simulaciones.

5.3.2. Núcleo del cliente

Una vez que se ha explicado la representación de los clientes en los ficheros de *SimGrid*, es momento de explicar cómo funciona el código de los clientes. Tal y cómo se ha mencionado en el apartado anterior, la función que define el comportamiento de los clientes se llama *cliente*. Esta función, escrita en el fichero *main.cpp*, tiene la siguiente firma:

```
1 static void cliente(int argc, char** argv) {  
2     /*  
3     Función que describe la comunicación del cliente con el servidor  
4  
5     :param 1: número de argumentos que recibe el cliente  
6     :param 2 (argv[1]): nombre del mailbox del cliente  
7     :param 3 (argv[2]): modo de operación (MANUAL o FILE)  
8     :param 4 (argv[3]):  
9         - (argv[3] a argv[argc-1]): política, operaciones y sus argumentos en  
10        caso de que sea MANUAL  
11        - (argv[3]): fichero donde encontrar la política y las operaciones en  
12        caso de que sea FILE  
13    :return: void  
14    */  
15 }
```

Esta función se encarga de: parsear los argumentos recibidos, ya estén en la variable *argv* o en el fichero de operaciones correspondiente; comprobar que los argumentos recibidos son suficientes y llamar a la función del cliente que implemente la política adecuada. Como se ha explicado en el apartado [5.1. Estructura del proyecto](#), la carpeta *functions* contiene varios ficheros relacionados con las políticas de los clientes. Estos ficheros implementan unos métodos que definen la manera en la que el cliente se tiene que comportar

dependiendo de la política que use, por lo que el cliente definido en *main.cpp* sólo tiene que llamar a los métodos del fichero adecuado.

Todos estos ficheros contienen un método principal, que consiste en un bucle que lee las operaciones del cliente y las manda a los servidores siguiendo la manera definida por la política, y este es el método al que tiene que llamar el cliente definido en el fichero *main.cpp*. Esta organización se puede observar en la figura 5.3.

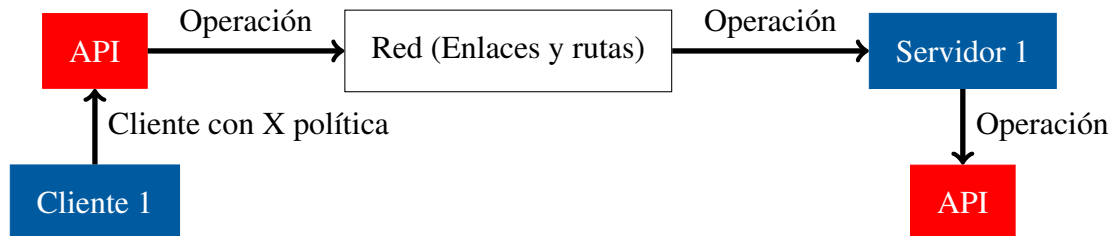


Fig. 5.3. Llamadas a las políticas de acceso

Las implementaciones de los métodos de cada fichero son propias a cada política de acceso, por lo que se explicarán en profundidad en el siguiente apartado. Una vez que el cliente termine de mandar las operaciones y reciba la última respuesta, enviará la operación “END” para finalizar el hilo del servidor y llamará a la función *finalizar_actor* para contribuir al fin de la simulación (5.2.2. *Núcleo del servidor*).

5.3.3. Políticas

Como se ha explicado en el apartado anterior, el comportamiento del cliente según la política de acceso se definidas en ficheros distintos. Pese a que cada implementación es distinta, los métodos son los mismos entre todos los ficheros: un método que es llamado por el cliente y un método por cada operación que implementa la manera en la que se envía dicha operación. El primer método siempre sigue la misma estructura entre todos los ficheros:

1. Comprueba si la simulación es pequeña o clusterizada.
2. La implementación de este paso cambia drásticamente dependiendo de la política y del tipo de simulación, pero siempre obtiene los nombres de los servidores con los que tiene que comunicarse.
3. Abre los *mailboxes* del cliente y de los servidores.
4. Entra en un bucle para mandar las operaciones a los servidores, llamando al método adecuado que implementa el envío de la operación.

Dado que el paso número 2 depende altamente de la política, y los métodos de envío de operaciones son también variables, se proporciona el siguiente resumen para saber cómo están implementados los métodos de cada fichero.

Métodos de la política secuencial

El método principal simplemente recibe el nombre del servidor al que mandar las operaciones en caso de que sea una simulación pequeña, por lo que la obtención del nombre del *mailbox* (que siempre coincide con el nombre del servidor) es directa. En el caso de que la simulación sea clusterizada, recibirá el nombre del cluster de servidores al que se quiere mandar la operación, y elige el número del servidor mediante una sencilla fórmula determinista:

$$Index_{server} = (Index_{client} \% Num_{servers}) + 1$$

Una vez que sabe a que servidor mandar las operaciones, entra en el bucle que llama a los métodos de envío de la operación respectiva. Estos métodos son muy sencillos, ya que el código es igual entre todos, exceptuando el de la función “WRITE”, que debe enviar un argumento más que el resto de operaciones. Este código se puede generalizar en las siguientes líneas:

```
1 void client_sequential_operation(sg4::Mailbox* o_mailbox, const std::string
   argument) {
2     o_mailbox->put(new std::string("OPERATION_NAME"), 16);
3     o_mailbox->put(new std::string(argument), argument.size());
4 }
```

Métodos de la política de replicación

En este caso, el método principal no recibe el nombre del servidor, simplemente recibe un factor de replicación, que es, esencialmente, el número de servidores al que se quiere enviar cada operación. Es por ello que el método de elección del servidor tiene que ser semialeatorio, para evitar generar siempre los mismos servidores que verán su carga de operaciones aumentada; pero determinista para hacer que la simulación se pueda repetir logrando los mismos resultados en distintas ejecuciones de la misma configuración. Para ello, se ha ideado un algoritmo que se resume en los siguientes pasos:

1. Pasar a valores decimales ascii todos los caracteres del nombre del servidor y guardar la suma en una variable.
2. Hacer el módulo de esta variable con el número de servidores y guardarlo en una variable. Matemáticamente:

$$Base_{Index} = Sum_{chars} \% Num_{servers}$$

3. Si la simulación es pequeña, pasar al paso 5; si la simulación es clusterizada, pasar al paso 4.

4. Elegir el cluster de servidores con el índice conseguido en el paso 2 y elegir un servidor de ese cluster con un generador aleatorio con una semilla fijada. No pasar al paso 4.
5. Elegir el servidor con el índice conseguido en el paso 2.

Una vez que se ha conseguido el servidor base al que mandar las operaciones, el resto de servidores se eligen de manera cíclica, ver figura 4.4, hasta que el número de servidores sea igual al $\min(Num_{servers}, Replication_{Factor})$. Una vez que se abren los mailboxes y se entre al bucle de envío de operaciones, el cliente llamará a las funciones encargadas de enviar la operación adecuada.

De la misma manera que en la política secuencial, estas operaciones tienen una misma estructura. Lo único en lo que difieren de los métodos anteriores es que ahora las operaciones se envían a varios servidores. Los métodos siguen el siguiente ejemplo de código genérico:

```

1 void client_replication_operation(std::vector<sg4::Mailbox*> o_mailboxes, const
  std::string argument) {
2   for (int i = 0; i < o_mailboxes.size(); i++) {
3     o_mailboxes[i]->put(new std::string("OPERATION_NAME"), 16);
4     o_mailboxes[i]->put(new std::string(argument), argument.size());
5   }
6 }

```

Métodos de la política Expand

La política Expand es la más complicada de las tres, ya que todos sus métodos dependen también de la estrategia de distribución que siga. Para explicar los argumentos que se reciben en función de cada estrategia de distribución se ha elaborado la tabla 5.3.

TABLA 5.3.
ARGUMENTOS DE LA POLÍTICA EXPAND

Estrategia de distribución	Argumentos
Cíclica	Factor de partición: cuantos bytes tiene cada sub-fichero
Aleatoria	Factor de partición: cuantos bytes tiene cada sub-fichero
Selectiva	Factor de partición: cuantos bytes tiene cada sub-fichero Servidores: servidores en los que el usuario elige que se distribuyan los sub-ficheros

El método principal, por lo tanto, tiene que elegir una serie de servidores en función de la estrategia de distribución que siga el cliente. En el caso de la estrategia selectiva, los servidores y su orden son elegidos por el usuario, así que simplemente se abrirán los *mailboxes* de dichos servidores.

Para la distribución aleatoria, simplemente se abren los *mailboxes* de todos los servidores en caso de que la simulación sea pequeña. En caso de que la simulación sea clusterizada, se genera un cluster aleatorio teniendo en cuenta una semilla fija a la que se suma el índice del cliente, para así generar un cluster distinto en cada cliente que sea el mismo entre ejecuciones. Luego, se abren los *mailboxes* de todos los servidores de dicho cluster. Adicionalmente, en cada operación se genera un orden aleatorio pero fijando una semilla que determina a que servidor enviar la operación.

Para la distribución cíclica, el único problema que hay que solucionar es la elección del primer servidor. Esta selección es prácticamente igual que en la política de replicación, donde se elige el índice del servidor base haciendo el módulo de la suma de los caracteres del nombre del cliente, cogiendo su valor decimal ascii, con el número de servidores. En el caso de una simulación clusterizada, se genera tanto el cluster base como el servidor base de dicho cluster con esta fórmula, cambiando el número de servidores por, primero, el número de clusters, y luego por el número de servidores del cluster seleccionado. Tras esto, se abren todos los *mailboxes* de todos los servidores (o de todos los servidores del cluster elegido) en un orden cíclico.

Una vez que se ha elegido el orden de los servidores, el cliente entra en el bucle de envío de operaciones, donde llama a los métodos encargados de enviar las distintas operaciones. La diferencia entre esta política y el resto es que los métodos no siguen una estructura general, por lo que es difícil resumirlos sin una explicación profunda. A pesar de ello, a continuación se adjuntan los algoritmos genéricos de cada método:

Método CREATE

- Si la estrategia de distribución es selectiva o cíclica, se envía la operación y se recibe la respuesta.
- Si la estrategia de distribución es aleatoria, se genera una semilla teniendo en cuenta un número fijo y el índice del cliente, manda la operación a un servidor aleatorio y recibe la respuesta.

Método READ

- Si la estrategia de distribución es selectiva o cíclica, se envía la operación para leer el fichero de metadatos y se recibe la respuesta.
- Si la estrategia de distribución es aleatoria, se genera una semilla teniendo en cuenta un número fijo y el índice del cliente, manda la operación de lectura del fichero de metadatos al servidor generado aleatoriamente (el mismo entre operaciones y ejecuciones al fijar la semilla) y recibe la respuesta.

Tras esto, independientemente de la política:

1. Abre un fichero auxiliar de Expand (que simula el fichero de metadatos de Expand) que guarda el orden de los servidores donde está cada sub-fichero.
2. Manda una operación de lectura por cada sub-fichero al servidor correspondiente y recibe la respuesta de cada envío .

Método WRITE

1. Calcula como se divide el fichero a escribir en función del factor de partición.
2. Elige a qué servidor enviar cada sub-fichero dependiendo de la estrategia de distribución.
3. Abre el fichero auxiliar de Expand (que simula el fichero de metadatos de Expand) que guarda el orden de los servidores donde está cada sub-fichero.
4. Guarda la información del orden de los servidores.
5. Envía una operación de creación y una de escritura por cada sub-fichero al servidor correspondiente y recibe la respuesta de cada envío.

Método DELETE

1. Si la estrategia de distribución es selectiva o cíclica, se envía la operación de eliminación del fichero de metadatos de Expand y se recibe la respuesta.
2. Si la estrategia de distribución es aleatoria, se genera una semilla teniendo en cuenta un número fijo y el índice del cliente, manda la operación de eliminación del fichero de metadatos al servidor generado aleatoriamente (el mismo entre operaciones y ejecuciones al fijar la semilla) y recibe la respuesta.

Tras esto:

1. Abre el fichero auxiliar de Expand (que simula el fichero de metadatos de Expand) que guarda el orden de los servidores donde está cada sub-fichero.
2. Manda una operación de lectura del fichero de metadatos y una operación de eliminación por cada sub-fichero al servidor correspondiente, recibe la respuesta de cada envío.
3. Borra el fichero y sus sub-ficheros correspondientes del fichero simulado de metadatos de Expand.

Método COMPUTE_TIME

Este método simplemente envía una operación de cómputo a todos los servidores cuyo *mailbox* esté abierto.

5.4. Integración con Gnuplot

Integrar el resultado de la simulación para generar gráficas con *Gnuplot* es realmente simple. Lo primero que hay que hacer es sacar la información del consumo temporal y energético de los servidores por pantalla. Esto se hace en la simulación mediante la función *print_summary*, que tiene la siguiente definición:

```
1 void print_summary() {
2     /*
3     Función que imprime un resumen de la simulación
4
5     :return: void
6     */
7 }
```

Esta función simplemente recorre la variable global donde se guarda el consumo de cada servidor e imprime el resultado por pantalla. Esta salida es redirigida a un archivo en un directorio de resultados bajo el nombre *output.txt*. Tras esto, se parsea la salida mediante un script de *Python* llamado *parse_output.py* que se encarga de generar tres ficheros de extensión *.dat*.

Estos ficheros son usados por los scripts de *Gnuplot* (ficheros *.plt*), que son ejecutados automáticamente tras finalizar la simulación. Los scripts de *Gnuplot* son realmente simples y siguen todos la misma lógica general que se resume en el siguiente código.

```
1 // Generar un png de 800x600 píxeles
2 set terminal pngcairo size 800,600 enhanced
3 set output 'results/graphs/graph_name.png'
4
5 // Estilo del gráfico
6 set style data histogram
7 set style histogram rowstacked // Sólo en aquellos que tengan esta forma
8 set style fill solid border -1
9 set boxwidth 0.75
10
11 // Título y etiquetas de los ejes
12 set title "Título"
13 set xlabel "X-Label"
14 set ylabel "Y-Label"
15
16 set key outside
17 set xtics rotate by -45
```

Con todo esto, y tras una simulación pequeña sencilla, se generan los gráficos de las figuras 5.4, 5.5 y 5.6; tal y cómo se había planteado en el [Requisito RS_FN_19](#), el [Requisito RS_FN_20](#) y el [Requisito RS_FN_21](#):

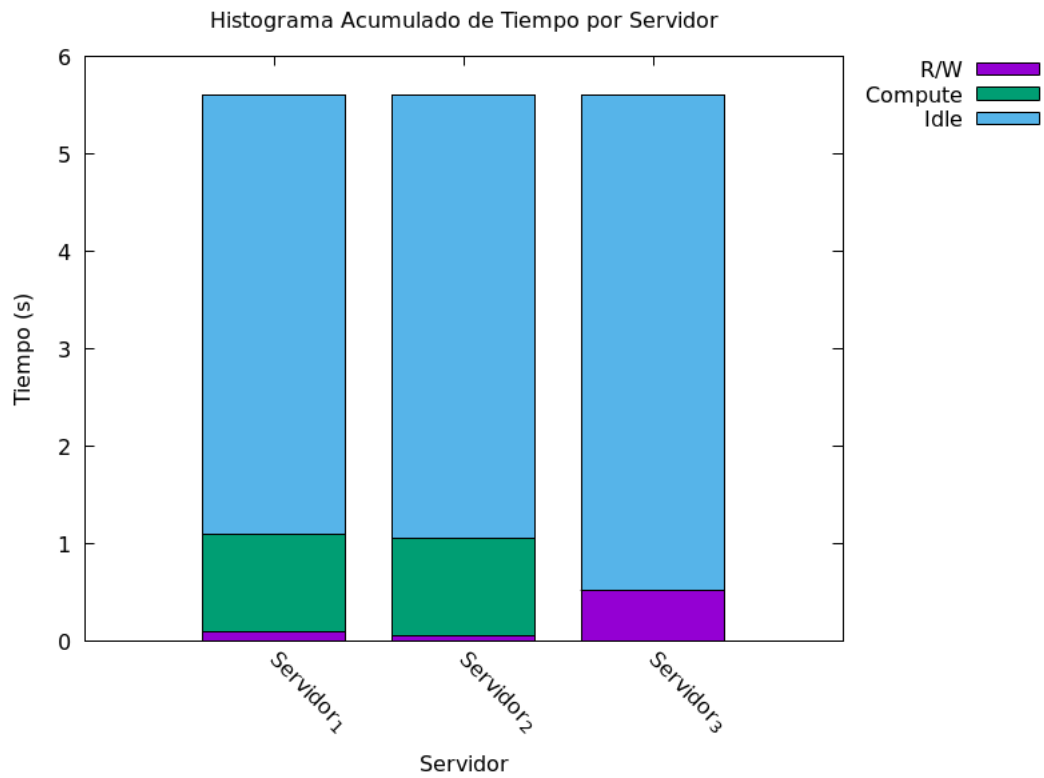


Fig. 5.4. Ejemplo de tiempo por servidor

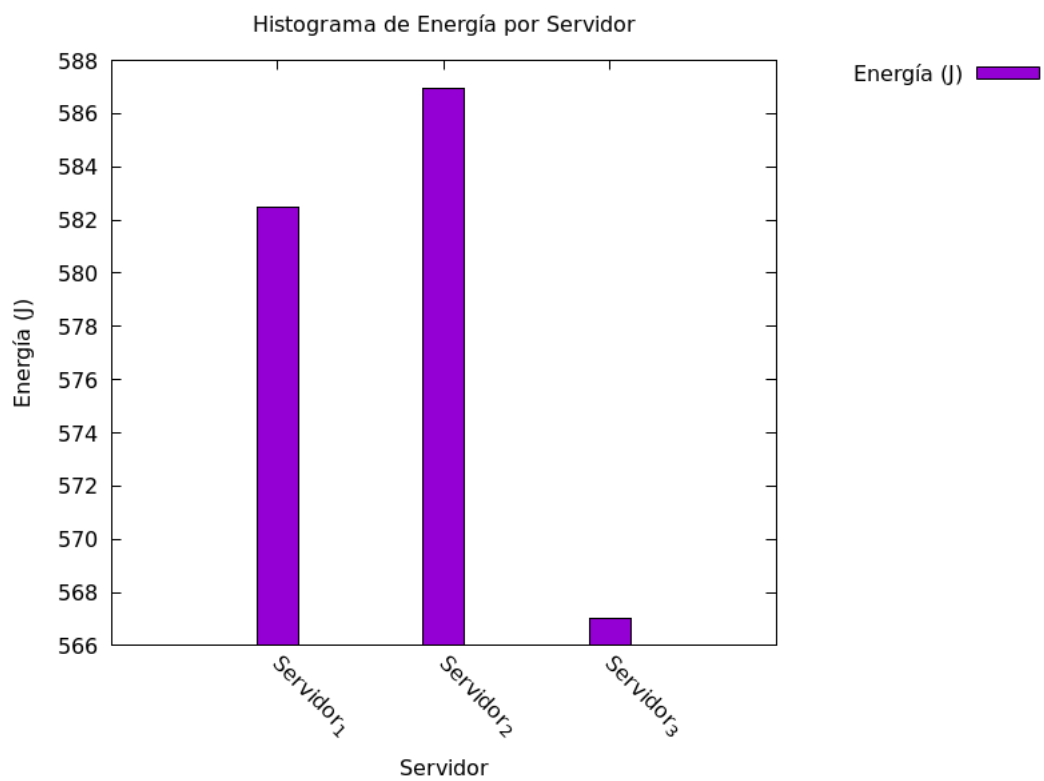


Fig. 5.5. Ejemplo de energía por servidor

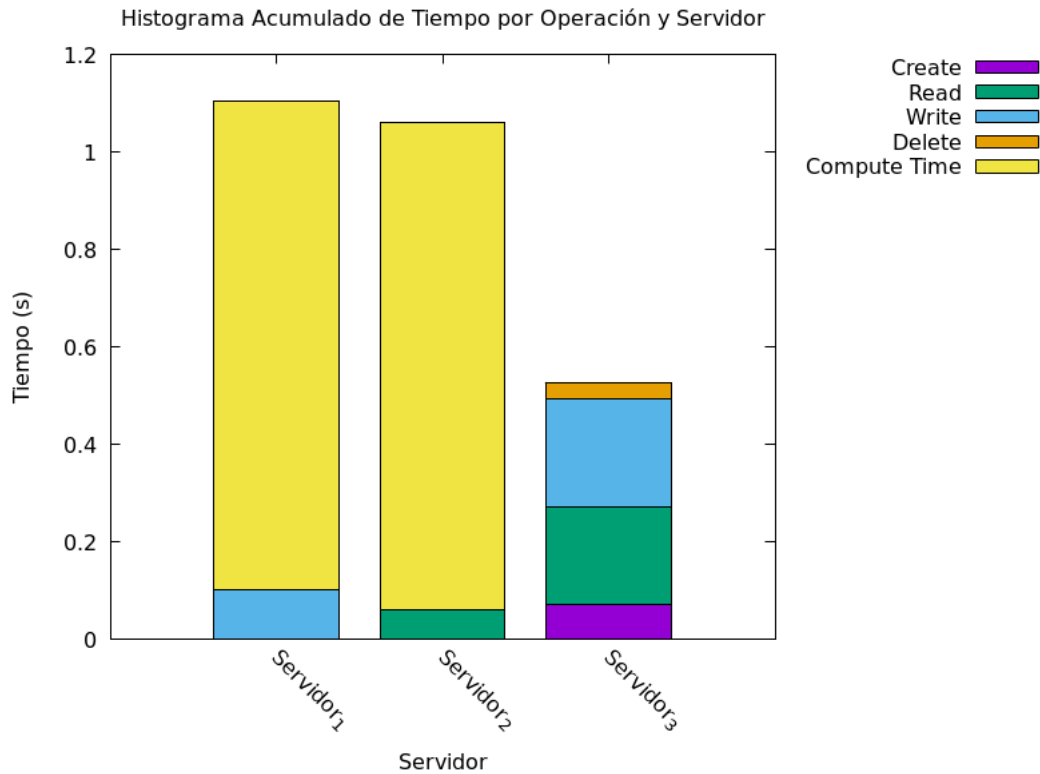


Fig. 5.6. Ejemplo de tiempo por operaciones por servidor

5.5. Interfaces

El último punto a explicar de la implementación es el desarrollo de las interfaces de líneas de comandos. En este punto se explica: qué interfaces se han desarrollado, cómo se han desarrollado y el diseño detrás de las mismas.

5.5.1. Interfaces finales

Tal y cómo se ha especificado en el [Requisito RU_RE_02](#) y el [Requisito RU_RE_03](#), el usuario debe contar con una interfaz que controle la modificación de la plataforma de la simulación y con otra que controle las acciones de los actores, que, en el ámbito de este proyecto, es el fichero de despliegue. Adicionalmente, a lo largo del desarrollo del proyecto, se vió la necesidad de imlementar una interfaz adicional que agilizase y automatizase la escritura de los ficheros de las operaciones. Es por tanto que se han implementado las interfaces expuestas en la tabla [5.4](#).

TABLA 5.4.
INTERFACES IMPLEMENTADAS

Nombre	Fichero	Descripción
Interfaz de simulaciones pequeñas	<i>small_CLI.py</i>	Interfaz que controla la plataforma y el despliegue de las simulaciones pequeñas
Interfaz de simulaciones clusterizadas	<i>clusterized_CLI.py</i>	Interfaz que controla la plataforma y el despliegue de las simulaciones clusterizadas
Interfaz de operaciones	<i>templates_CLI.py</i>	Interfaz que controla la generación de ficheros de operaciones para simulaciones pequeñas y clusterizadas

A pesar de que esta no es la organización inicial planteada para las interfaces, esta nueva distribución de ficheros cumple con los requisitos planteados y, además, agrupa las funciones de cada tipo de simulación en un fichero. Estos tres ficheros contienen distintas funciones que controlan cada tipo de interfaz y, para emplear estas funciones, se ha creado el fichero *CLI.py*, que actúa como un ejecutable que une el acceso a las tres interfaces en un punto.

5.5.2. Programación de las interfaces

Para programar las interfaces de *Python*, se ha usado la librería *inquirer* [51], que contiene un set de funciones que permiten mostrar un menú de opciones seleccionables en una línea de comandos. La programación de las interfaces usando esta librería es rápida, fácil e intuitiva, pese a que no ofrece opciones para personalizar tanto las interfaces como otras librerías.

La estructura de los métodos implementados en cada fichero de interfaces siguen una estructura similar, ya que existen dos tipos de métodos principales: los que implementan bucles infinitos para preguntar lo que el usuario quiere hacer y los métodos o funciones finales, que implementan la lógica para agregar contenido al fichero adecuado. Por esto, es conveniente explicar ambos tipos de métodos es sus respectivos subapartados.

Funciones de bucles

Estas funciones implementan un bucle infinito que pregunta al usuario que acción quiere tomar y llama al método conveniente una vez que el usuario se haya decidido. A pesar de que la implementación puede variar dependiendo del punto de la interfaz en el que se encuentre el usuario, se puede generalizar un código que muestra la implementación de estos métodos.

```
1 def loop_function():
2     """
3     Función que define el loop principal de cierta parte de la interfaz
4
5     :return: (None)
6     """
7
8     while (1):
9
10        option = inquirer.prompt(options)
11
12        if "option_1" in first_option["options"]:
13            option_1()
14        elif "option_2" in first_option["options"]:
15            option_2()
16        elif "option_3" in first_option["options"]:
17            option_3()
18        else:
19            # Limpiar la pantalla
20            os.system(CLEANING_COMMAND)
21
22        return
```

Las funciones usan la función *prompt* de la librería *inquirer* para preguntar por el paso que quiere tomar el usuario. La manera de responder a las preguntas puede variar, ya que el usuario puede escribir texto, seleccionar una opción de una lista, marcar una serie de *checkboxes*...

Adicionalmente, este tipo de métodos puede llamar a más funciones cíclicas o a una función finales. En ese sentido, las interfaces se pueden estructurar como un árbol, donde la raíz y los nodos padres son las funciones de bucles, y los nodos hoja son las funciones finales, que añaden contenido a los ficheros adecuados en vez de ofrecer una ramificación de opciones al usuario.

Funciones finales

Las funciones finales, pese a compartir los mismos pasos generalmente, pueden diferir extensamente en su implementación. Debido al alto número de métodos finales, que se encargan de añadir, modificar o eliminar: servidores, clientes, operaciones, ficheros, enlaces, redes... no se cree conveniente hacer una explicación de cada una de las funciones.

Es por ello que, para resumir el comportamiento de estos métodos, se presenta el siguiente algoritmo genérico que todas las funciones finales siguen a un nivel fundamental, si bien el código real puede cambiar mucho entre cada método. Este algoritmo de aplica sólo a las interfaces pequeñas y clusterizadas porque trabajan con ficheros *XML*, es decir, a las interfaces de modificación de los ficheros de plataforma y despliegue:

1. Se abre el fichero correspondiente y se consigue el nodo root que contiene el resto de nodos del fichero XML.
2. Se encuentran todos los nodos relacionados con la función. Por ejemplo, si la función tiene que ver con la modificación de los servidores, se consiguen todos los nodos relacionados con los servidores del fichero.
3. Dependiendo del tipo de función, pregunta por unos argumentos o por otros. Granularmente:
 - a) Si la función consiste en añadir información, preguntará por los argumentos relacionados del nodo a añadir. Por ejemplo, si se quiere añadir un servidor, se preguntará por todos los elementos de la tabla [5.1](#).
 - b) Si la función consiste en modificar información, preguntará qué nodo quiere el usuario modificar, y luego preguntará por los nuevos argumentos de dicho nodo.
 - c) Si la función consiste en eliminar información, simplemente preguntará por el nodo que el usuario quiere borrar.
4. Si la función consiste en añadir información, crea los elementos a añadir y busca el lugar del fichero donde añadir dichos elementos. Si la función no consiste en añadir información, pasar al paso 5.
5. Si la función consiste en modificar información, crea los elementos a añadir, elimina los antiguos y busca el lugar del fichero donde añadir los nuevos elementos. Si la función no consiste en modificar información, pasar al paso 6.
6. Si la función consiste en eliminar información, simplemente se elimina el elemento elegido por el usuario. Si la función no consiste en eliminar información, pasar al paso 7.
7. Se actualiza el fichero con la nueva información.

Las funciones finales de la interfaz de modificación de operaciones varían mucho, pero se pueden clasificar en estos tipos: modificación de las operaciones, modificación de la política, creación de los ficheros y eliminación de los ficheros. Pese a no ser muchos tipos, las funciones son altamente complejas, por lo que describirlas una a una puede ser una tarea demasiado extensa para el alcance de esta memoria.

Por ello, sólo se necesita saber que los principios de adición, modificación y eliminación de contenido son muy similares a los explicados en el algoritmo anterior, si bien, al modificar un texto plano en vez de un *XML*, la manera de leer y escribir este fichero se hace línea a línea, en vez de nodo a nodo.

5.5.3. Diseño de Interfaces

Las interfaces han sido diseñadas intentando mejorar la experiencia de usuario y con el objetivo de optimizar la velocidad de modificación de los ficheros. Algunas de las características de las interfaces que ayudan con estas tareas y que no han podido ser mencionadas con anterioridad son:

- Si un cliente o un servidor se elimina, el índice del resto de clientes o servidores se actualiza para no dejar huecos en los índices. Por ejemplo, en una plataforma con tres clientes, si el usuario decide eliminar el segundo cliente, el tercer cliente pasará a llamarse “Cliente_2”, y la información se actualizará en las rutas y en el fichero de despliegue.
- Si ya existe una ruta entre un cliente y un servidor, no se muestra la opción de añadir la ruta para evitar duplicaciones de rutas.
- Si se decide modificar un nodo, la información antigua de ese nodo aparecerá entre paréntesis para tener una referencia del valor anterior y, en caso de que se pueda, aparecerá como el valor predefinido en la respuesta del usuario.
- El usuario podrá seleccionar la opción “Salir” para volver al estado previo sin realizar cambios.
- En el caso de que el usuario tenga que introducir un valor por texto, se comprueba en todo momento que este valor sea adecuado. En caso de que no sea un valor adecuado, se notifica adecuadamente al usuario.
- El usuario solo debe introducir la parte numérica de aquellos valores que se escriban con su unidad en los ficheros. Por ejemplo, cuando se pregunta al usuario por la velocidad de un cliente, el usuario no tiene que introducir el texto “100Mf”, sólo tendrá que introducir el texto “100”.
- La interfaz de los ficheros de las operaciones incluye bucles para repetir una serie de operaciones un número deseado de veces.
- Las interfaces añaden la nueva información de los ficheros de manera ordenada, por ejemplo, un nuevo servidor será introducido después del último servidor actual.

Estas son las principales características de la interfaz, si bien hay más que no se han explicado debido a la limitación de la longitud de la memoria.

Heurísticas de Nielsen

Tal y como se ha mencionado en el [Requisito RS_NF_05](#), las interfaces deben seguir, en medida, de lo posible las heurísticas de Nielsen. Estas heurísticas describen una serie de características que una buena interfaz debe tener.

A modo de resumen, se presenta la tabla 5.5 que, por cada heurística de Nielsen [53], explica cómo se sigue en las interfaces.

TABLA 5.5.
HEURÍSTICAS DE NIELSEN EN LAS INTERFACES

Heurística	Uso en las interfaces
Visibilidad del estado del sistema	- Cada vez que se modifica un fichero, se notifica al usuario mediante un mensaje. - Cada vez que se produce un error, se notifica al usuario mediante un mensaje.
Coincidencia entre el sistema y el mundo real	- Los mensajes de error están escritos para que el usuario entienda qué ha pasado. - Los conceptos como enlaces y rutas se corresponden con su significado en el mundo de las redes.
Control y libertad	- El usuario puede deshacer las decisiones seleccionando la opción de “Salir” o pulsando <i>CTRL + C</i>.
Consistencia y estándares	- La opción de “Salir” siempre se encuentra situada como la última opción. - El orden para la selección de la modificación de la plataforma y del despliegue es el mismo entre la interfaz pequeña y la clusterizada. - Las opciones dentro de la interfaz pequeña y de la clusterizada son las mismas y siguen el mismo orden.
Prevención de errores	- El texto introducido por el cliente se comprueba antes de incluirlo en el fichero, dándole la opción al usuario de volver a introducirlo si es incorrecto. - La selección de opciones como los nodos a modificar o el fichero de operaciones a modificar se restringe a una lista sobre la que el usuario tiene que seleccionar un valor.
Reconocer en lugar de recordar	- Existe texto que pregunta al usuario qué quiere hacer, y cada opción es autodescriptiva.
Flexibilidad y eficiencia de uso	- Al modificar información de un fichero, la información anterior aparece por defecto para que el usuario no tenga que introducirla de nuevo si quiere sólo cambiar una parte.
Diseño minimalista	- La pantalla se limpia cada vez que el usuario cambia de contexto y se restringe a tener sólo la operación actual.
Ayuda con los errores	- Los mensajes de error indican qué ha ido mal al realizar la operación.
Documentación	- Existe una guía de uso en el <i>README.md</i> del proyecto.

Principios de diseño

Tal y cómo se ha mencionado en el [Requisito RS_NF_06](#), las interfaces deben seguir los principios de diseño. Estos son algunos principios que se usan para diseñar interfaces ya que resuelven problemas recurrentes [54]. Dado que la cantidad de principios de diseño es muy grande, la tabla 5.6 resume los más importantes o aquellas cuya visibilidad en las interfaces es mayor.

TABLA 5.6.
PRINCIPIOS DE DISEÑO EN LAS INTERFACES

Principio	Uso en las interfaces
B3 - Organización jerárquica	- Los métodos para modificar los ficheros se dividen en varios pasos jerárquicos. Por ejemplo, para añadir un servidor en la simulación pequeña hay que navegar a través de la interfaz siguiendo estos pasos: Simulación Pequeña ->Plataforma ->Gestionar los servidores ->Añadir los servidores
B5 - Organización alfabética	- Cuando se muestra un listado de opciones, estas opciones se ordenan alfabéticamente.
K6 - Menú de migas de pan	- Las interfaces tienen un menú de migas de pan localizado en la parte superior de la pantalla que permite saber dónde se encuentra el usuario.
K11 - Lenguaje familiar	- Términos como velocidad de lectura/escritura, servidores, caché... son palabras familiares para usuarios experimentados en el ámbito de la informática
K12 - Prevención de errores	- El texto introducido por el cliente se comprueba antes de incluirlo en el fichero, dándole la opción al usuario de volver a introducirlo si es incorrecto. - La selección de opciones como los nodos a modificar o el fichero de operaciones a modificar se restringe a una lista sobre la que el usuario tiene que seleccionar un valor.
K13 - Mensajes de error significativos	- Los mensajes de error indican qué ha ido mal al realizar la operación de manera concreta.

Si bien estos principios de diseño son generalmente aplicados a interfaces web, algunos principios se pueden aplicar también a las interfaces de líneas de comandos.

5.6. Despliegue

El despliegue de la aplicación es sencillo, y se detalla en el fichero *README.md*. De todas maneras, se resume aquí la manera de ejecutar los distintos scripts y ejecutables.

5.6.1. Dependencias

Lo primero que hay que hacer es instalar las dependencias. Para *Python* se proporciona un fichero *requirements.txt* que contiene todas las dependencias de las interfaces. Para *C++*, hay que tener instalado *cmake* y *make*, que son las herramientas usadas para compilar la simulación.

También, hay que tener instalada la versión 3.31 de *SimGrid*. Para ello, se recomienda descargarse el código fuente y compilarlo manualmente, para lo que se adjunta los siguientes comandos de *bash*:

```
1 git clone https://framagit.org/simgrid/simgrid.git
2 cd simgrid
3 git checkout v3.31
4 cmake -DCMAKE_INSTALL_PREFIX=/opt/simgrid .
5 make
6 make install
```

5.6.2. Ejecución

Para ejecutar las interfaces, sólo hace falta ejecutar el fichero de *Python CLI.py*. En cambio, para ejecutar la simulación hace falta ejecutar dos scripts, primero el de compilado y luego el de ejecución.

Adicionalmente, el script de ejecución recibe un parámetro, el valor “0” o el valor “1”, dependiendo de si se quiere ejecutar una simulación pequeña o una clusterizada, respectivamente. Los resultados de la simulación se mostrarán por pantalla y los gráficos generados se guardarán en el directorio */results/graphs*.

6. VALIDACIÓN Y VERIFICACIÓN

Este capítulo habla de las pruebas realizadas para comprobar que el sistema cumple con lo que se había especificado en los requisitos (3.2. *Requisitos*). Para ello se han diseñado unas pruebas de verificación, que comprueban el cumplimiento de los requisitos de sistema; y unas pruebas de validación, que demuestran como un usuario podría interactuar con el sistema para llevar a cabo los distintos casos de uso teniendo en cuenta sus expectativas sobre dicho sistema, representado mediante los requisitos de usuario.

Antes de presentar las pruebas, se define la tabla 6.1, que servirá a modo de plantilla para las pruebas a realizar. En ella se pueden observar distintos atributos que son esenciales para la descripción de cada prueba. También define un *ID*, en el cual el valor “XXX” será sustituido por la cadena “VER” si es una prueba de verificación o “VAL” si es una prueba de validación. De manera similar a los requisitos, la cadena “YY” del *ID* se sustituirá por un contador autoincremental independiente entre los tipos de las pruebas.

TABLA 6.1.
PLANTILLA PARA LAS PRUEBAS

ID: XXX-YY	
Descripción	Descripción de la prueba
Procedimiento	Pasos para realizar la prueba
Pre-condición	Las condiciones que deben darse antes de la prueba
Post-condiciones	Las condiciones que deben darse después de la prueba
Origen	Qué requisitos han impulsado el desarrollo de la prueba
Evaluación	Ha pasado el sistema la prueba

6.1. Pruebas de verificación

Este apartado contiene una elicitación de las pruebas que comprueban que los requisitos de sistema se cumplen. Antes de mostrar este listado, es necesario mencionar los requisitos para los que no hacen falta pruebas ya que tratan temas como los lenguajes de programación que se deben usar en el proyecto o la manera en la que hay que desarrollar el sistema. Estos requisitos se enumeran en la tabla 6.2.

TABLA 6.2.
REQUISITOS DE SISTEMA EXCLUIDOS DE LAS PRUEBAS DE
VERIFICACIÓN

Requisito	Motivo de la exclusión
<i>RS_FN_01</i>	Las propias pruebas demuestran la existencia de una interfaz
<i>RS_FN_06</i>	El estado está definido, sólo hay que comprobar que se pueda recuperar
<i>RS_FN_08</i>	Las propias pruebas demuestran la existencia de una interfaz
<i>RS_FN_11</i>	El estado está definido, sólo hay que comprobar que se pueda recuperar
<i>RS_FN_13</i>	La comunicación se realiza mediante los <i>mailboxes</i> y las pruebas que ejecuten la simulación demuestran la presencia de esta comunicación
<i>RS_FN_15</i>	La medición del tiempo se explica en el apartado 5.2.4. Medición de tiempo , y la precisión se debería comprobar con un sistema real
<i>RS_FN_16</i>	La medición de la energía se explica en el apartado 5.2.4. Medición de energía , y la precisión se debería comprobar con un sistema real
<i>RS_NF_01</i>	La simulación usa la interfaz <i>S4U</i> de <i>SimGrid</i>
<i>RS_NF_02</i>	Las interfaces están implementadas en <i>Python</i> usando <i>inquirer</i>
<i>RS_NF_03</i>	Se usa <i>Gnuplot</i> para generar los gráficos
<i>RS_NF_04</i>	El sistema de logs usa <i>XBT</i>
<i>RS_NF_05</i>	Las heurísticas de Nielsen se explican en el apartado 5.5.3. Heurísticas de Nielsen
<i>RS_NF_06</i>	Los patrones de diseño se explican en el apartado 5.5.3. Principios de diseño
<i>RS_NF_11</i>	Se prueba en el resto de pruebas relacionadas con las interfaces

Una vez se han explicitado los requisitos que no se deben tener en cuenta para las pruebas de verificación, se adjuntan todas las pruebas ideadas.

TABLA 6.3.
PRUEBA VER-01

ID: VER-01	
Descripción	Gestión de los servidores
Procedimiento	1. Crea un nuevo fichero de plataforma de una simulación pequeña. 2. Accede al la interfaz de gestión de servidores de la simulación pequeña. 3. Añade 1 servidor. 4. Modifica un servidor. 5. Elimina el servidor no modificado. 6. Comprueba el fichero de la plataforma.
Pre-condición	Ejecutar la interfaz general
Post-condiciones	El fichero de la plataforma tendrá solamente un servidor que ha sido modificado
Origen	<i>RS_FN_02, RS_FN_07</i>
Evaluación	Ejecución correcta

TABLA 6.4.
PRUEBA VER-02

ID: VER-02	
Descripción	Gestión de los clientes
Procedimiento	1. Accede al la interfaz de gestión de clientes de una simulación pequeña. 2. Añade 2 clientes. 3. Modifica un cliente. 4. Elimina el cliente no modificado. 5. Comprueba el fichero de la plataforma.
Pre-condición	Realización de la prueba <i>VER-01</i> y ejecución de la interfaz general
Post-condiciones	El fichero de la plataforma tendrá solamente un cliente que ha sido modificado
Origen	<i>RS_FN_03</i>
Evaluación	Ejecución correcta

TABLA 6.5.
PRUEBA VER-03

ID: VER-03	
Descripción	Gestión de los enlaces
Procedimiento	1. Accede al la interfaz de gestión de enlaces de una simulación pequeña. 2. Añade 3 enlaces. 3. Modifica un enlace. 4. Elimina uno de los enlaces no modificados. 5. Comprueba el fichero de la plataforma.
Pre-condición	Realización de la prueba <i>VER-02</i> y ejecución de la interfaz general
Post-condiciones	El fichero de la plataforma tendrá sólomente dos enlaces, uno que ha sido modificado y otro que no
Origen	<i>RS_FN_04</i>
Evaluación	Ejecución correcta

TABLA 6.6.
PRUEBA VER-04

ID: VER-04	
Descripción	Gestión de las rutas
Procedimiento	1. Accede al la interfaz de gestión de rutas de una simulación pequeña. 2. Añade 1 ruta entre el cliente-1 y el servidor-1 que use un enlace. 3. Modifica la ruta para que use el otro enlace. 4. Elimina la ruta. 5. Comprueba que no hay una ruta entre el cliente-1 y el servidor-1 en el fichero de la plataforma . 6. Ejecuta la interfaz general y accede de nuevo la interfaz de gestión de rutas de una simulación pequeña. 7. Añade una ruta entre el cliente-1 y el servidor-1. 8. Comprueba el fichero de la plataforma.
Pre-condición	Realización de la prueba <i>VER-03</i> y ejecución de la interfaz general
Post-condiciones	El fichero de la plataforma tendrá una ruta entre el cliente-1 y el servidor-1
Origen	<i>RS_FN_05</i>
Evaluación	Ejecución correcta

TABLA 6.7.
PRUEBA VER-05

ID: VER-05	
Descripción	Gestión de las operaciones de los clientes manualmente
Procedimiento	1. Crea un fichero de despliegue nuevo. 2. Accede a la gestión de los clientes en los ficheros de despliegue de una simulación pequeña. 3. Añade el cliente usando operaciones manuales, añade una operación de crear un fichero y una de eliminar el fichero. 4. Modifica la operación de crear un fichero para que sea una de escribir un fichero. 5. Elimina la operación de eliminar el fichero. 6. Comprueba el fichero de despliegue.
Pre-condición	Realización de la prueba <i>VER-04</i> y ejecución de la interfaz general
Post-condiciones	El fichero de despliegue tiene un cliente con una operación de escritura
Origen	<i>RS_FN_09, RS_FN_12</i>
Evaluación	Ejecución correcta

TABLA 6.8.
PRUEBA VER-06

ID: VER-06	
Descripción	Gestión de las operaciones y las políticas de los clientes mediante la interfaz de operaciones
Procedimiento	1. Crea un nuevo fichero de operaciones para una simulación pequeña y elige una de las políticas de acceso. 2. Añade una operación de creación de un fichero, una de escritura de ese fichero, una de lectura de ese fichero, una de eliminación de ese fichero y una de cómputo. 3. Accede a la gestión de los clientes en los ficheros de despliegue de una simulación pequeña. 4. Modifica el cliente manual existente para que use con el fichero recién creado. 5. Comprueba el fichero de despliegue.
Pre-condición	Realización de la prueba <i>VER-05</i> y ejecución de la interfaz general
Post-condiciones	El fichero de despliegue tiene un cliente con unas operaciones definidas por un fichero externo
Origen	<i>RS_FN_09, RS_FN_10</i>
Evaluación	Ejecución correcta

TABLA 6.9.
PRUEBA VER-07

ID: VER-07	
Descripción	Gestión manual de las políticas de los clientes
Procedimiento	1. Accede a la gestión de los clientes en los ficheros de despliegue de una simulación pequeña. 2. Crea un nuevo cliente con una política de acceso secuencial. 3. Modifica el cliente para que use una política de acceso de replicación. 4. Comprueba el fichero de despliegue.
Pre-condición	Tener un cliente en el fichero de plataforma y ejecución de la interfaz general
Post-condiciones	El fichero de despliegue tiene un cliente con una política secuencial
Origen	<i>RS_FN_10</i>
Evaluación	Ejecución correcta

TABLA 6.10.
PRUEBA VER-08

ID: VER-08	
Descripción	Ejecución de la simulación y visualización de los resultados en Linux
Procedimiento	1. Ejecuta la simulación pequeña. 2. Visualiza la pantalla. 3. Accede a la carpeta de resultados y visualiza los gráficos generados.
Pre-condición	Realización de la prueba <i>VER-06</i> y tener la simulación compilada
Post-condiciones	Los gráficos se han generado bien y la información aparece por pantalla
Origen	<i>RS_FN_14</i> , <i>RS_FN_17</i> , <i>RS_FN_18</i> , <i>RS_FN_19</i> , <i>RS_FN_20</i> , <i>RS_FN_21</i> , <i>RS_NF_07</i> , <i>RS_NF_08</i>
Evaluación	Ejecución correcta

TABLA 6.11.
PRUEBA VER-09

ID: VER-09	
Descripción	Prueba con plataforma mal definida
Procedimiento	1. Borra una línea con contenido del fichero de plataforma pequeña. 2. Ejecuta la simulación pequeña.
Pre-condición	Realización de la prueba <i>VER-07</i> y tener la simulación compilada
Post-condiciones	El sistema informa del error y la simulación termina con un error
Origen	<i>RS_FN_22</i> , <i>RS_NF_09</i>
Evaluación	El sistema falla, pero es esperado

TABLA 6.12.
PRUEBA VER-10

ID: VER-10	
Descripción	Prueba con despliegue mal definido
Procedimiento	1. Borra una línea con contenido del fichero de despliegue pequeño. 2. Ejecuta la simulación pequeña.
Pre-condición	Realización de la prueba <i>VER-07</i> y tener la simulación compilada
Post-condiciones	El sistema informa del error y la simulación termina con un error
Origen	<i>RS_FN_22, RS_NF_10</i>
Evaluación	El sistema falla, pero es esperado

TABLA 6.13.
PRUEBA VER-11

ID: VER-11	
Descripción	Ejecución de la simulación en MacOS
Procedimiento	1. Define una plataforma y un despliegue para una simulación clusterizada con un cluster de servidores y uno de clientes, que los clientes tengan una operación de cómputo de tiempo en un fichero de operaciones. 2. Ejecuta la simulación.
Pre-condición	Estar en un ordenador cuyo sistema operativo sea MacOS
Post-condiciones	La simulación se ejecuta correctamente
Origen	<i>RS_FN_14, RS_FN_17, RS_FN_18, RS_FN_19, RS_FN_20, RS_FN_21, RS_NF_07, RS_NF_08</i>
Evaluación	Ejecución correcta

Se proporciona la matriz de trazabilidad para resumir la relación de los requisitos con las pruebas en la tabla [6.14](#).

TABLA 6.14.
MATRIZ DE TRAZABILIDAD DE LAS PRUEBAS DE
VERIFICACIÓN

	<i>VER-01</i>	<i>VER-02</i>	<i>VER-03</i>	<i>VER-04</i>	<i>VER-05</i>	<i>VER-06</i>	<i>VER-07</i>	<i>VER-08</i>	<i>VER-09</i>	<i>VER-10</i>	<i>VER-11</i>
RS_FN_02	✓										
RS_FN_03		✓									
RS_FN_04			✓								
RS_FN_05				✓							
RS_FN_07	✓										
RS_FN_09					✓	✓					
RS_FN_10						✓	✓				
RS_FN_12					✓						
RS_FN_14								✓			✓
RS_FN_17								✓			✓
RS_FN_18								✓			✓
RS_FN_19								✓			✓
RS_FN_20								✓			✓
RS_FN_21								✓			✓
RS_FN_22									✓	✓	
RS_NF_07								✓			✓
RS_NF_08								✓			✓
RS_NF_09									✓		
RS_NF_10										✓	

6.2. Pruebas de validación

A pesar de la gran superposición de estas pruebas con las pruebas de verificación, ya que ambas comprueban el correcto funcionamiento del sistema, estas pruebas difieren en su enfoque, ya que están inspiradas en los casos de uso para representar cómo el usuario interactúa con el sistema. Por lo tanto, se ha decidido desarrollar cuatro pruebas que muestran la interacción del usuario con cada caso de uso.

TABLA 6.15.
PRUEBA VAL-01

ID: VAL-01	
Descripción	Gestión de la plataforma clusterizada
Procedimiento	1. Accede a la interfaz de gestión de plataformas clusterizadas. 2. Crea un nuevo fichero de plataforma. 3. Añade 3 clusters de servidores, elimina uno y modifica otro. 4. Añade 3 clusters de clientes, elimina uno y modifica otro. 5. Añade 5 enlaces, elimina 2 y modifica otros 2. 6. Añade las rutas entre los clientes y los servidores, elimina una, modifica otra, y vuelve a crear la que había borrado.
Pre-condición	Ejecutar la interfaz genérica
Post-condiciones	El fichero de la plataforma clusterizada se ha modificado correctamente
Origen	<i>RU_CA_01, RU_CA_02, RU_CA_03, RU_CA_04,</i> <i>RU_CA_05, RU_CA_06, RU_CA_07, RU_CA_08,</i> <i>RU_CA_09, RU_CA_10, RU_CA_11, RU_CA_12,</i> <i>RU_CA_18, RU_RE_01</i>
Evaluación	Ejecución correcta

TABLA 6.16.
PRUEBA VAL-02

ID: VAL-02	
Descripción	Gestión del despliegue clusterizado
Procedimiento	1. Accede a la interfaz de gestión de las operaciones. 2. Crea 3 ficheros de operaciones distintos, cada uno con una política distinta y con sus operaciones. 3. Accede a la interfaz de gestión del despliegue clusterizado. 4. Crea un nuevo fichero de despliegue. 5. Añade un fichero de operaciones a cada uno de los clientes. 6. Modifica el fichero de operaciones de uno de los clientes. 7. Accede de nuevo a la interfaz de gestión de las operaciones. 8. Modifica las operaciones de algún fichero. 9. Elimina alguna operación de algún fichero. 10. Cambia la política de un fichero.
Pre-condición	Realización de la prueba <i>VAR-01</i> y ejecutar la interfaz gráfica
Post-condiciones	el fichero del despliegue clusterizado se ha modificado correctamente
Origen	<i>RU_CA_13, RU_CA_14, RU_CA_15, RU_CA_16, RU_CA_17, RU_CA_19, RU_RE_02</i>
Evaluación	Ejecución correcta

TABLA 6.17.
PRUEBA VAL-03

ID: VAL-03	
Descripción	Ejecución de la simulación
Procedimiento	1. Ejecuta la simulación clusterizada.
Pre-condición	Realización de la prueba <i>VAR-02</i> y tener la simulación compilada
Post-condiciones	La simulación se ejecuta correctamente
Origen	<i>RU_RE_03, RU_RE_04, RU_RE_05, RU_RE_06</i>
Evaluación	Ejecución correcta

TABLA 6.18.
PRUEBA VAL-04

ID: VAL-04	
Descripción	Acceso a los datos de la simulación
Procedimiento	1. Ejecuta la simulación. 2. Visualiza el resultado de la simulación en la pantalla. 3. Accede a la carpeta de resultados y visualiza los gráficos.
Pre-condición	Realización de la prueba <i>VAR-03</i> y tener la simulación compilada
Post-condiciones	Los gráficos se han generado correctamente y la información de la simulación es visible en la pantalla
Origen	<i>RU_CA_20, RU_CA_21</i>
Evaluación	Ejecución correcta

Al igual que en las pruebas anteriores, se proporciona una matriz de trazabilidad para resumir la conexión de los requisitos con cada prueba en la tabla [6.19](#).

TABLA 6.19.

MATRIZ DE TRAZABILIDAD DE LAS PRUEBAS DE VALIDACIÓN

	VAL-01	VAL-02	VAL-03	VAL-04
RU_CA_01	✓			
RU_CA_02	✓			
RU_CA_03	✓			
RU_CA_04	✓			
RU_CA_05	✓			
RU_CA_06	✓			
RU_CA_07	✓			
RU_CA_08	✓			
RU_CA_09	✓			
RU_CA_10	✓			
RU_CA_11	✓			
RU_CA_12	✓			
RU_CA_13		✓		
RU_CA_14		✓		
RU_CA_15		✓		
RU_CA_16		✓		
RU_CA_17		✓		
RU_CA_18	✓			
RU_CA_19		✓		
RU_CA_20			✓	✓
RU_CA_21			✓	✓
RU_RE_01	✓			
RU_RE_02		✓		
RU_RE_03			✓	
RU_RE_04			✓	
RU_RE_05			✓	
RU_RE_06			✓	

6.3. Prueba completa

Tras haber expuesto las pruebas de validación y verificación del simulador, se cree conveniente mostrar un caso de uso completo que cubra la interacción entera con la aplicación. Para ello, se expondrá un caso que muestre, paso por paso, las acciones que un usuario tomaría para realizar una simulación.

Para lograr una mayor precisión con los distintos valores que los servidores y los clientes toman en relación a las capacidades de las máquinas en las que se alojan, se usará el benchmark *IOR* [55]. Este benchmark permite “medir el rendimiento de sistemas de almacenamiento paralelos mediante el uso de varias interfaces y patrones de acceso” [55].

Con todo esto, esta sección se divide en varios apartados que explican: el caso de uso completo, cómo usar el benchmark *IOR* para este caso de uso completo, qué acciones tomaría el usuario para definir la plataforma y las operaciones de los clientes y una exposición de los resultados obtenidos.

6.3.1. Caso de uso

Para probar la herramienta en su totalidad, se ha ideado un caso de uso que pruebe las funcionalidades descritas en el diagrama de casos de uso (figura 3.1). En este caso se probarán una serie de operaciones que unos clientes mandará a un servidor, para lo que el usuario deberá: describir la plataforma con los clientes y los servidores, asignarle las operaciones a los clientes y ejecutar la simulación. Tras esto, el usuario podrá observar los resultados tanto por la salida del programa como por los gráficos generando, cubriendo así todos los casos de uso del simulador.

6.3.2. Benchmark IOR

El benchmark *IOR* es usado para medir el rendimiento de unos procesos que realizan escrituras y lecturas sobre un fichero de prueba. De la ejecución del benchmark se extraerán datos como la velocidad de escritura o de lectura, que posteriormente pueden ser asignados a los servidores para realizar una prueba con datos realistas. Antes de explicar cómo se ha empleado este benchmark, es necesario listar una serie de términos relacionados con el mismo:

- **Número de segmentos:** este parámetro dictamina en cuantos fragmentos se divide el fichero donde se realizan las pruebas del benchmark.
- **Tamaño de los bloques:** este parámetro indica el tamaño de los bloques que conforman cada uno de los segmentos.

- **Tamaño de la transferencia:** este parámetro indica la cantidad de bytes que se envían en una llamada a una función de escritura o de lectura.
- **Número de tareas:** este parámetro dictamina el número de tareas que tomarán parte en la ejecución del benchmark.

Para usar este benchmark, una serie de tareas serán ejecutadas para realizar diversas escrituras y lecturas sobre un fichero. Para ello, se fijarán parámetros como el número de segmentos, el tamaño de los bloques y el tamaño de la transferencia para realizar pruebas consistentes entre ejecuciones.

Además, por cada ejecución del benchmark se ejecutarán un número fijo de escrituras y lecturas para así sacar un valor medio de velocidad de lectura y de escritura, reduciendo la variabilidad de los resultados. Por último, ya que el benchmark se realiza con ficheros reales, se limpiará la caché del ordenador sobre el que se ejecuta este benchmark para reducir su impacto entre las escrituras y lecturas.

Con todo esto en mente, el fichero de configuración del benchmark tiene la siguiente forma:

```

1  IOR START
2      api=POSIX
3      testFile=testFile
4      repetitions=256
5      readFile=1
6      writeFile=1
7      filePerProc=1
8      segmentCount=1
9      blockSize=256k
10     transferSize=8k
11     numTasks=8
12     collective=1
13  IOR STOP

```

Con este fichero de configuración, al ejecutar el benchmark y analizar la salida, se obtienen los siguientes resultados, que serán usados para describir la plataforma:

TABLA 6.20.
RESULTADOS DEL BENCHMARK *IOR*

	Velocidad Máxima	Velocidad Mínima	Velocidad Media
Escritura	6118.61 MB/s	1147.55 MB/s	5117.07 MB/s
Lectura	29026.33 MB/s	5648.89 MB/s	22685.22 MB/s

6.3.3. Configuración de la simulación

Para configurar la simulación, el usuario tiene que usar las interfaces desarrolladas. Primero, tiene que describir la plataforma, que contará con un servidor y ocho clientes que se comunicarán en una ruta sencilla. Para poder hacer esto, el usuario tendrá que:

1. Ejecutar la interfaz.
2. Seleccionar la interfaz de una simulación simple (se podría hacer con una clusterizada).
3. Crear un nuevo fichero de plataforma.
4. Crear un servidor, usando las velocidades medias calculadas en la tabla 6.20.
5. Crear ocho clientes distintos.
6. Conectar los clientes y los servidores mediante rutas.

Algunos valores, como el consumo energético del ordenador, se ha calculado usando los valores medios de consumo. Tras seguir estos pasos en la interfaz, el fichero de plataforma tiene la siguiente forma:

```

1  <?xml version="1.0"?>
2  <!DOCTYPE platform SYSTEM "https://simgrid.org/simgrid.dtd">
3  <platform version="4.1">
4    <zone id="zone_1" routing="Full">
5      <AS id="AS0" routing="Full">
6
7        <!--Servidores-->
8        <host id="Servidor_1" speed="3800Mf" core="4">
9          <prop id="Disco" value='{ "type": "HDD", "read_speed": "22685MB/s", "
          write_speed": "5117MB/s", "cache_data": "20%", "cache_metadata": "20%"
          }' />
10         <prop id="wattage_per_state" value="4500.0:8400.0"/>
11         <prop id="wattage_off" value="10.0"/>
12       </host>
13
14       <!--Clientes-->
15       <host id="Cliente_1" speed="200Mf"/>
16       <host id="Cliente_2" speed="300Mf"/>
17       <host id="Cliente_3" speed="250Mf"/>
18       <host id="Cliente_4" speed="275Mf"/>
19       <host id="Cliente_5" speed="285Mf"/>
20       <host id="Cliente_6" speed="290Mf"/>
21       <host id="Cliente_7" speed="310Mf"/>
22       <host id="Cliente_8" speed="337Mf"/>
23
24       <!--Finishers-->

```

```

25     <host id="Finalizador" speed="100Mf"/>
26
27     <!--Información de red-->
28     <link id="Enlace_Finisher" bandwidth="100MBps" latency="10us"/>
29     <link id="Enlace_1" bandwidth="125MBps" latency="200us"/>
30     <link id="Enlace_2" bandwidth="150MBps" latency="150us"/>
31     <link id="Enlace_3" bandwidth="130MBps" latency="250us"/>
32     <link id="Enlace_4" bandwidth="300MBps" latency="100us"/>
33     <route symmetrical="YES" src="Servidor_1" dst="Finalizador">
34         <link_ctn id="Enlace_Finisher"/>
35     </route>
36     <route symmetrical="YES" src="Servidor_1" dst="Cliente_1">
37         <link_ctn id="Enlace_1"/>
38     </route>
39     <route symmetrical="YES" src="Servidor_1" dst="Cliente_2">
40         <link_ctn id="Enlace_2"/>
41     </route>
42     <route symmetrical="YES" src="Servidor_1" dst="Cliente_3">
43         <link_ctn id="Enlace_3"/>
44     </route>
45     <route symmetrical="YES" src="Servidor_1" dst="Cliente_4">
46         <link_ctn id="Enlace_4"/>
47     </route>
48     <route symmetrical="YES" src="Servidor_1" dst="Cliente_5">
49         <link_ctn id="Enlace_1"/>
50         <link_ctn id="Enlace_3"/>
51     </route>
52     <route symmetrical="YES" src="Servidor_1" dst="Cliente_6">
53         <link_ctn id="Enlace_2"/>
54         <link_ctn id="Enlace_4"/>
55     </route>
56     <route symmetrical="YES" src="Servidor_1" dst="Cliente_7">
57         <link_ctn id="Enlace_2"/>
58         <link_ctn id="Enlace_3"/>
59         <link_ctn id="Enlace_4"/>
60     </route>
61     <route symmetrical="YES" src="Servidor_1" dst="Cliente_8">
62         <link_ctn id="Enlace_1"/>
63         <link_ctn id="Enlace_4"/>
64     </route>
65 </AS>
66 </zone>
67 </platform>

```

Una vez se ha creado el fichero de plataforma, es necesario crear el fichero de despliegue. A cada cliente se le asigna un fichero de operaciones (creado con la interfaz) que describe la creación de un fichero, la escritura y la lectura del mismo repetida diez mil veces y la eliminación del mismo. Debido a la existencia de un único servidor, la política elegida es la secuencial. Tras la configuración del mismo, el fichero de despliegue toma

la siguiente forma:

```

1  <?xml version="1.0"?>
2  <!DOCTYPE platform SYSTEM "http://simgrid.gforge.inria.fr/simgrid/simgrid.dtd">
3  <platform version="4.1">
4    <!--Clientes-->
5    <actor host="Cliente_1" function="cliente">
6      <argument value="Cliente_1"/>
7      <argument value="FILE"/>
8      <argument value="./templates/small/testCompleto"/>
9    </actor>
10   <actor host="Cliente_2" function="cliente">
11     <argument value="Cliente_2"/>
12     <argument value="FILE"/>
13     <argument value="./templates/small/testCompleto"/>
14   </actor>
15   <actor host="Cliente_3" function="cliente">
16     <argument value="Cliente_3"/>
17     <argument value="FILE"/>
18     <argument value="./templates/small/testCompleto"/>
19   </actor>
20   <actor host="Cliente_4" function="cliente">
21     <argument value="Cliente_4"/>
22     <argument value="FILE"/>
23     <argument value="./templates/small/testCompleto"/>
24   </actor>
25   <actor host="Cliente_5" function="cliente">
26     <argument value="Cliente_5"/>
27     <argument value="FILE"/>
28     <argument value="./templates/small/testCompleto"/>
29   </actor>
30   <actor host="Cliente_6" function="cliente">
31     <argument value="Cliente_6"/>
32     <argument value="FILE"/>
33     <argument value="./templates/small/testCompleto"/>
34   </actor>
35   <actor host="Cliente_7" function="cliente">
36     <argument value="Cliente_7"/>
37     <argument value="FILE"/>
38     <argument value="./templates/small/testCompleto"/>
39   </actor>
40   <actor host="Cliente_8" function="cliente">
41     <argument value="Cliente_8"/>
42     <argument value="FILE"/>
43     <argument value="./templates/small/testCompleto"/>
44   </actor>
45   <!--Finisher-->
46   <actor host="Finalizador" function="finalizador"/>
47   <!--Servidores-->
48   <actor host="Servidor_1" function="servidor">
49     <argument value="Servidor_1"/>

```

```
50     </actor>
51 </platform>
```

Adicionalmente, el fichero de operaciones tiene la siguiente forma:

```
1 SEQUENTIAL Servidor_1
2 CREATE test
3 LOOP 10000
4     WRITE test 256000
5     READ test
6 DONE;
7 DELETE test
```

Con estos ficheros, el usuario puede ejecutar la simulación.

6.3.4. Resultados de la prueba completa

Una vez que el usuario ejecute la simulación, obtendrá los resultados de la misma. Estos resultados se muestran tanto por pantalla como mediante gráficos, los cuales tienen la siguiente forma:

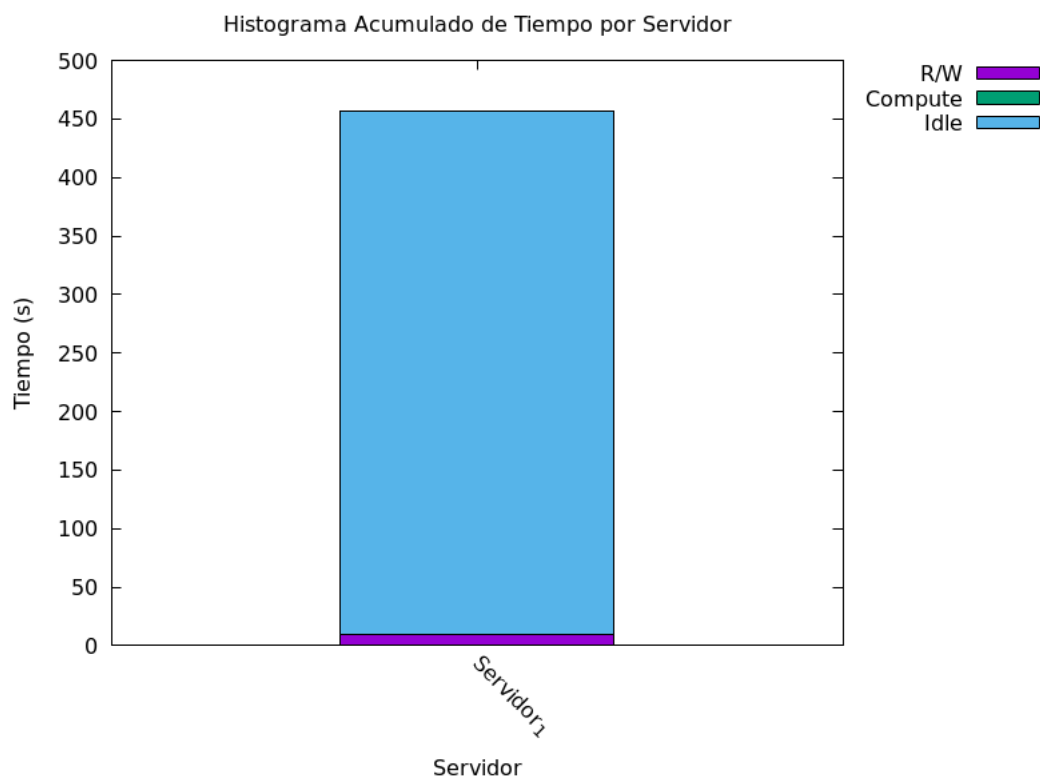
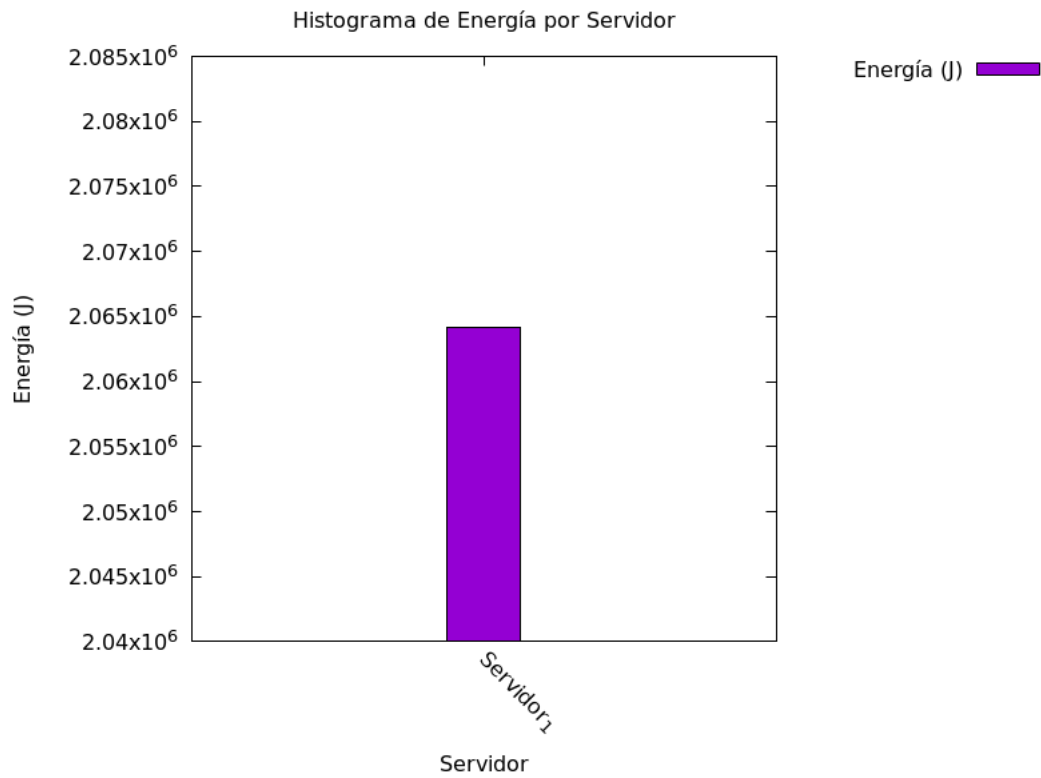
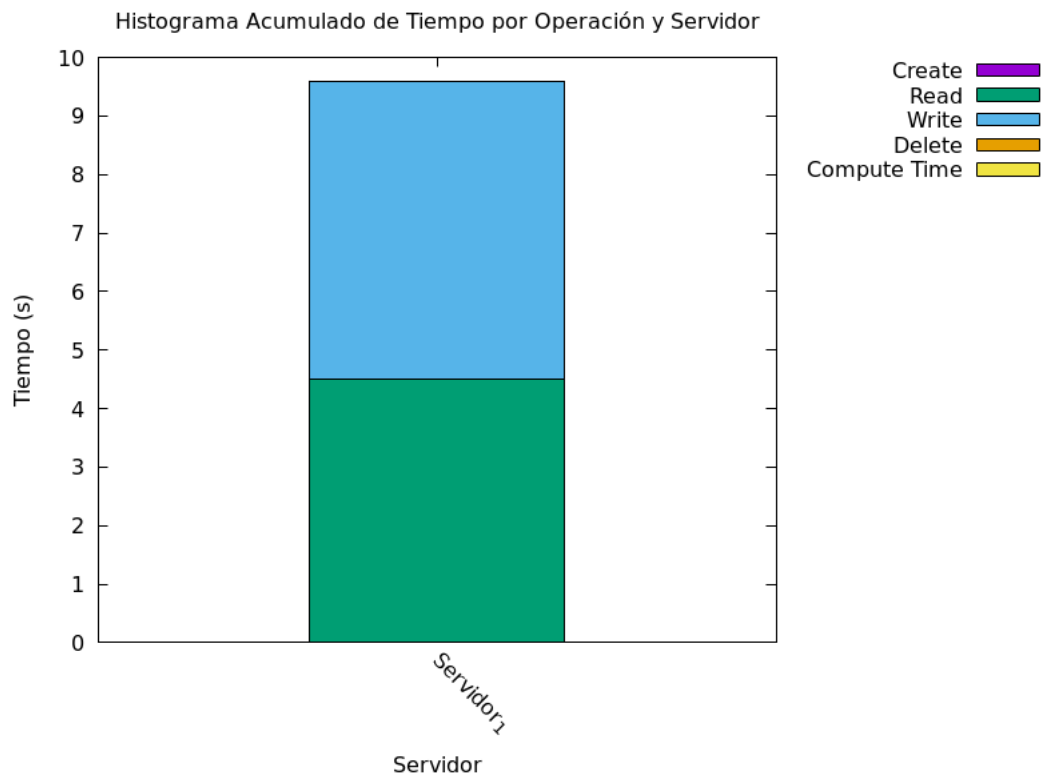


Fig. 6.1. Resultados de tiempo por servidor

**Fig. 6.2.** Resultados de energía por servidor**Fig. 6.3.** Resultados de tiempo por operaciones por servidor

6.3.5. Prueba escalada

A modo de mostrar la visualización de los resultados cuando hay más de un servidor presente en la simulación, se ha decidido escalar la prueba para la existencia de seis servidores. En este caso, la configuración de los parámetros de los servidores viene dada por la tabla 6.20, donde cada servidor tendrá unas velocidades ligeramente superiores o inferiores a las mostradas en dicha tabla para variar los resultados.

Adicionalmente, las operaciones definidas para los clientes son las mismas que en la prueba anterior, cambiando solamente la política secuencial por la de replicación. Asimismo, se define un factor de replicación aleatorio para cada cliente, lo que crea la necesidad de definir un fichero para cada uno. Con todos estos cambios en mente, esta prueba escalada arroja las siguientes gráficas como resultado:

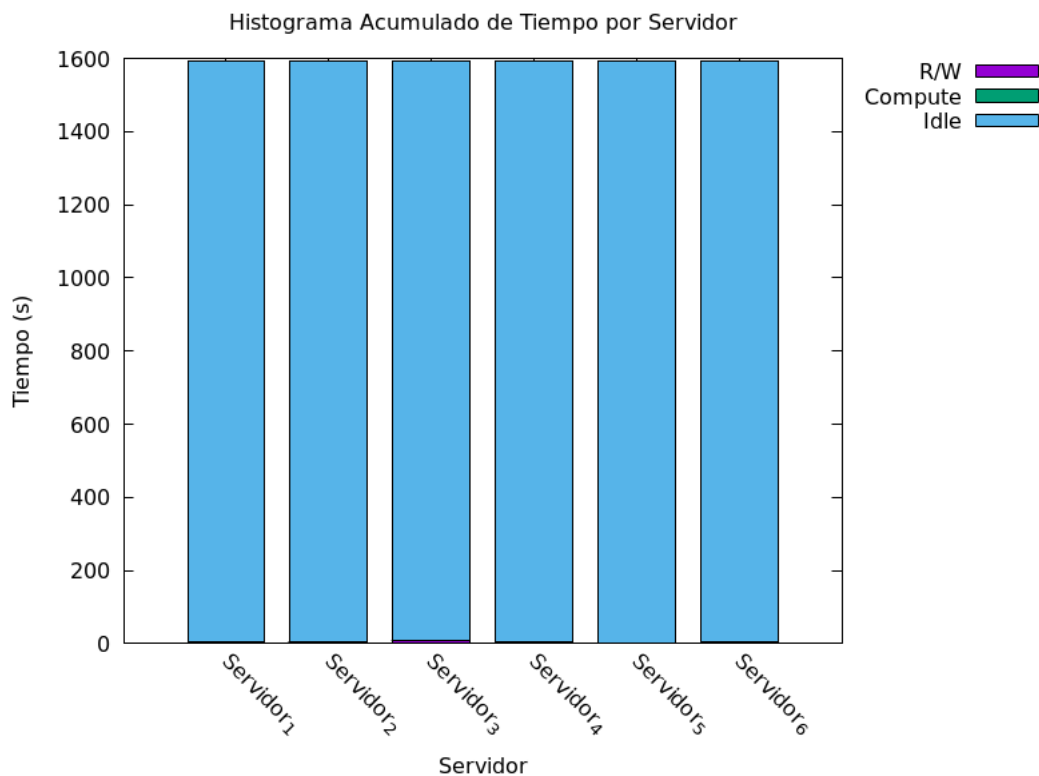


Fig. 6.4. Resultados de tiempo por servidor en la prueba escalada

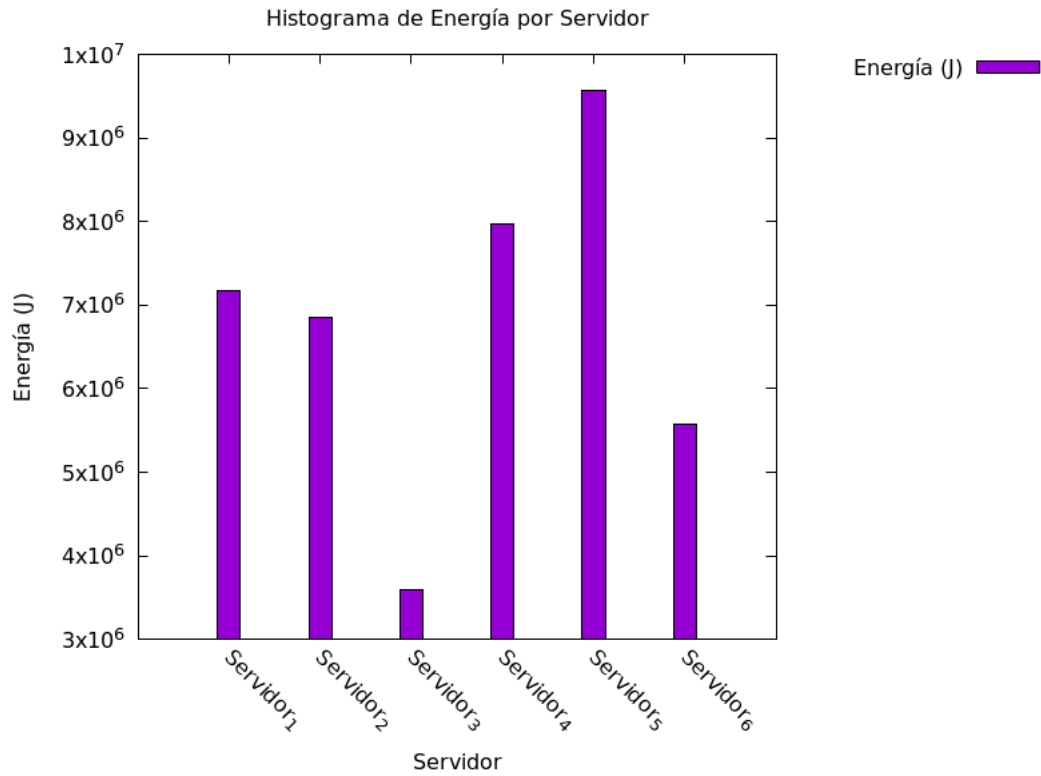


Fig. 6.5. Resultados de energía por servidor en la prueba escalada

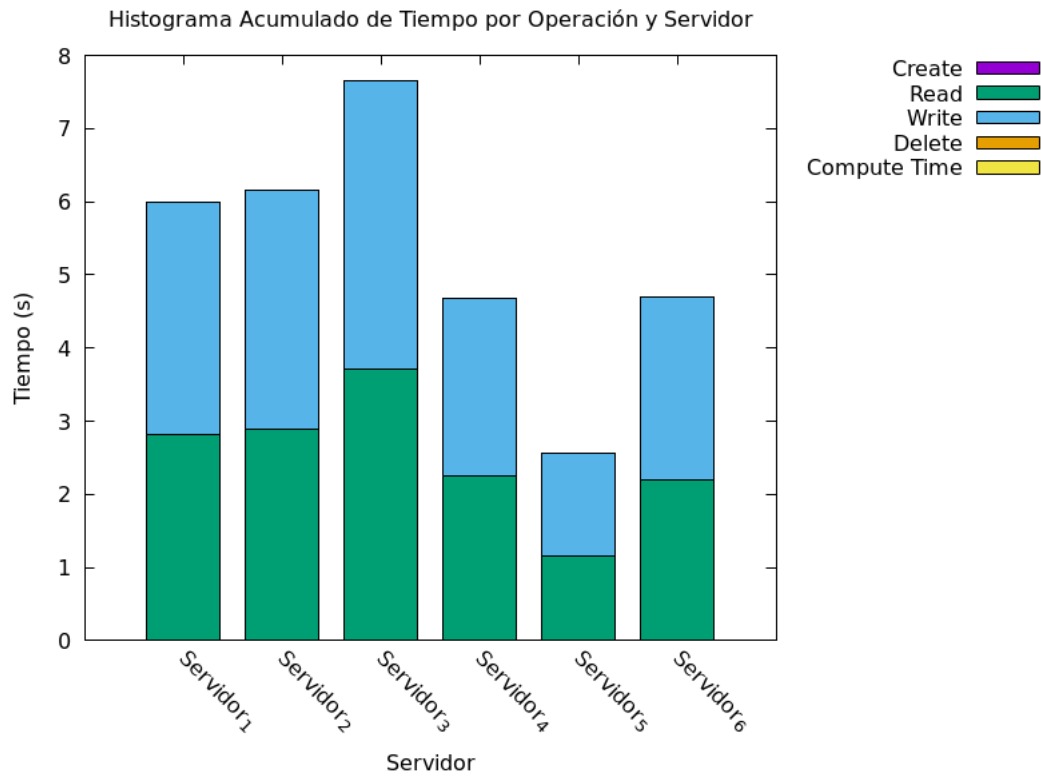


Fig. 6.6. Resultados de tiempo por operaciones por servidor en la prueba escalada

7. PLANIFICACIÓN Y PRESUPUESTO

Este capítulo describe aspectos relacionados con la planificación de este proyecto, como la metodología que se ha seguido, la estimación temporal del proyecto y el presupuesto del mismo. Cada tema se trata en su respectivo apartado, en el orden recién mencionado.

7.1. Metodología

Esta sección discute las metodologías que se podrían haber usado en el proyecto, cerrando con la justificación de la metodología usada. Se proponen las siguientes metodologías de desarrollo de software:

- **Metodología en cascada** [56]: metodología que divide el proyecto en varias fases de manera que una se realiza cuando se acabe la anterior. Esta metodología es aplicable cuando el proyecto tiene un objetivo bien definido, y da una facilidad al seguimiento del proyecto con el que otras metodologías no pueden competir.
- **Metodología en espiral** [57]: es un enfoque del desarrollo de software que intenta arreglar los inconvenientes del método en cascada y que describe el ciclo de vida de un software mediante espirales que se repiten hasta la entrega del producto final.
- **Metodología agile** [58]: es una metodología iterativa, en la cuál se realizan entregas cíclicas y en cada entrega se realizan todas las fases del ciclo, desde los requisitos hasta la entrega.

Dada la naturaleza del proyecto, y debido a que el objetivo del proyecto estaba muy definido desde el principio, se decidió optar por una metodología en cascada, algo más tradicional que sus contrapartes, pero muy aplicable a este proyecto. Para hacer una estimación temporal del proyecto, primero hay que listar las fases del mismo. El modelo en cascada se aplica en el contexto de este proyecto definiendo las siguientes fases:

1. **Análisis**: en esta fase se lleva a cabo el desarrollo de los requisitos y de los casos de uso que permiten definir y delimitar la funcionalidad del sistema final.
2. **Diseño**: en esta fase se lleva a cabo el estudio de herramientas y sus alternativas, así como el desarrollo de la estructura del proyecto y la ideación del modelo cliente-servidor.
3. **Implementación de la simulación**: en esta fase se lleva a cabo la implementación del simulador usando *SimGrid* y las estructuras de comunicación cliente-servidor diseñadas anteriormente, así como la generación de los gráficos con *Gnuplot*.

4. **Implementación de las interfaces:** en esta fase se lleva a cabo el desarrollo de las interfaces que permiten definir los ficheros de plataforma, despliegue y operaciones de manera automática.
5. **Evaluación:** en esta fase se llevan a cabo las prueba de validación y verificación ideadas para garantizar el correcto funcionamiento del sistema de acuerdo con los requisitos.

7.2. Estimación temporal

Una vez que ya se ha elegido la metodología y se ha hecho una clara división en las fases del proyecto, se puede realizar un diagrama de Gantt que estima la duración de cada fase. Este diagrama divide las fases en tareas más pequeñas con el objetivo de describir de manera más precisa la duración de cada tarea. Además, se ha añadido una tarea extra de documentación, presente de manera paralela al desarrollo y la ideación de este sistema. El diagrama se muestra en la figura [7.1](#).

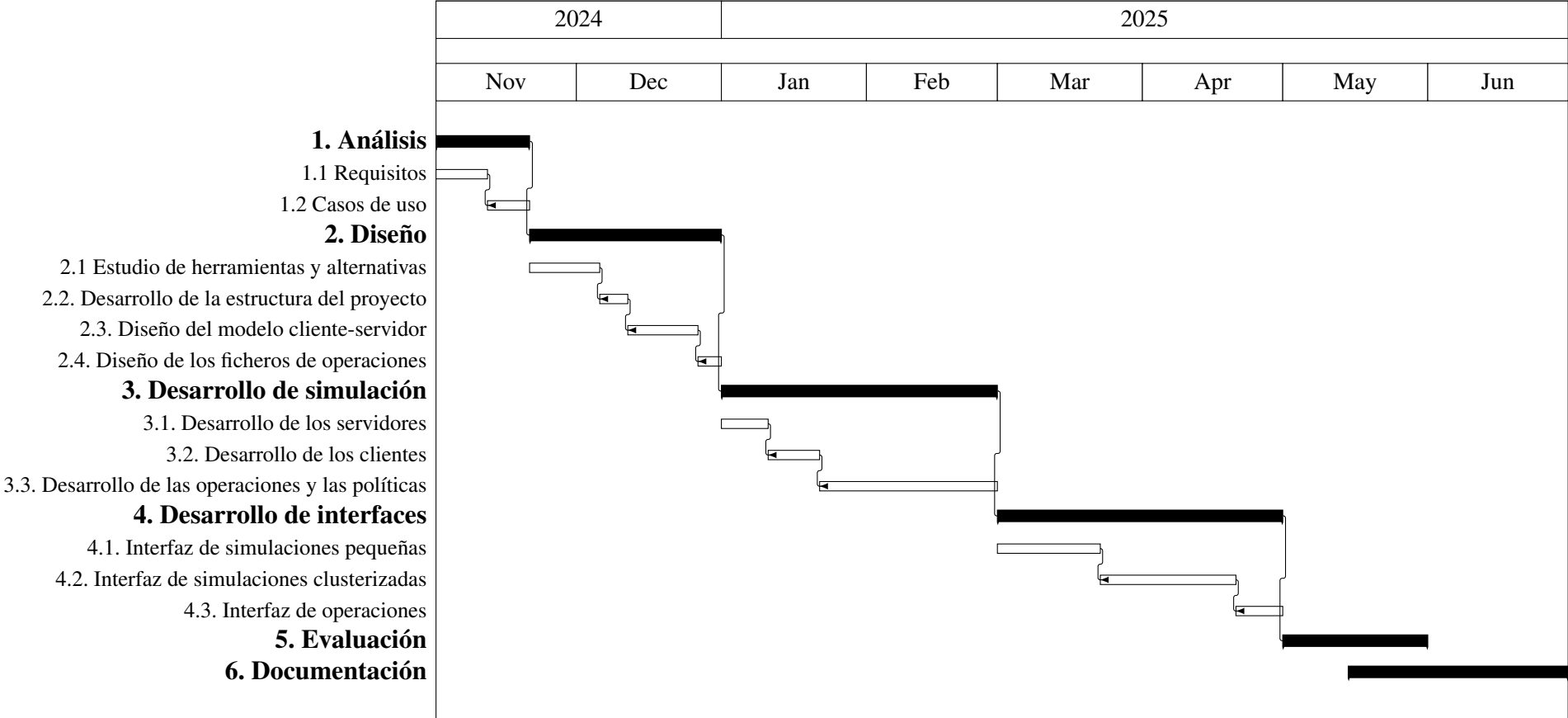


Fig. 7.1. Diagrama de Gantt del proyecto

7.3. Presupuesto

Esta sección define el presupuesto del proyecto que tiene en cuenta tanto los costes directos como los indirectos para realizar una oferta final. Dado que el proyecto es un proyecto de desarrollo con orientaciones no-comerciales, sino de estudio, algunos costes se aproximan al coste de desarrollo del proyecto, más que a una oferta comercial.

7.3.1. Costes directos

Lo primero que hay que tener en cuenta son los costes directos, en el ámbito de este proyecto, se definen dos costes directos: el coste del personal y el coste del equipamiento. Primero, hay que calcular el coste del personal, es decir, el coste asociado al pago de los sueldos de las personas empleadas en este proyecto. Para el proyecto se han definido tres miembros concretos:

- **Analista:** es el encargado de realizar la fase de análisis, es decir, realizar un estudio de los casos de uso y requisitos del proyecto.
- **Programador:** es el encargado de realizar las fases de diseño y de implementación con el objetivo de desarrollar el sistema final.
- **Evaluador:** es el encargado de realizar la fase de evaluación, es decir, la ideación y las pruebas de los tests.

Con esto en mente, la tabla 7.1 muestra el coste asociado al personal teniendo en cuenta los roles definidos.

TABLA 7.1.
COSTE DEL PERSONAL

Rol	Horas	Coste por Hora	Total
Analista	40h	40 €/h	1600€
Programador	250h	35 €/h	8750€
Evaluador	25h	25 €/h	625€
Total	315h		10975€

Además, hay que tener en cuenta el coste del equipamiento, tanto hardware como software. Sin embargo, todas las herramientas usadas en este proyecto son de uso libre y gratuitas, por lo que sólo se calcula el coste del hardware. Para calcular el coste del equipamiento, hay que tener en cuenta su coste amortizado que se calcula con la siguiente fórmula, con:

- C : Coste de amortización del objeto

- c : Coste unitario del objeto
- u : Uso del recurso en horas
- t : Tiempo de amortización del recurso en horas (duración del proyecto, 8 meses en el Gantt)
- a : Amortización del recurso

$$C = \frac{c \times u \times t}{a}$$

Con esta fórmula en mente, se define la tabla 7.2 que expone los costes del equipamiento.

TABLA 7.2.
COSTE DEL EQUIPAMIENTO

Objeto	Coste (c)	Uso (u)	Tiempo de uso (t)	Amortización	Coste amortizado (C)
Ordenador	800€	50 %	8 meses	48 meses	66,67€
Monitor	80 €	45 %	8 meses	60 meses	6€
Teclado	26 €	75 %	8 meses	36 meses	3.25€
Ratón	20€	70 %	8 meses	36 meses	2.33€
Total	926€				78.25€

7.3.2. Costes indirectos

Los costes indirectos son costes que surgen como consecuencia del desarrollo del proyecto, pero no por el proyecto per se. En este caso, los costes indirectos son el consumo de electricidad y la conexión a Internet.

Para el cálculo del consumo energético, hay que tener en cuenta que un ordenador consume unos 90W, un teclado consume 30W, un teclado consume cerca de 3W y un ratón consume cerca de 1W, resultando en un total de 124W. Como la duración del proyecto se ha estimado en 315h, la energía consumida asciende a un total de $124W \times 315h = 39060Wh$. Teniendo en cuenta que el coste del consumo promedio de la Península Ibérica asciende a un total de 0,1676 €/KWh, el coste total debido a la energía consumida asciende a **6.55€**.

A esto, hay que sumarle el coste de un proveedor de *Wifi*. El coste promedio de estos proveedores ronda los 30€ al mes, que multiplicado por los 8 meses del proyecto asciende a un total de 240€. Esto hace que los costes indirectos totales asciendan a la suma de **246.55€**.

7.3.3. Costes totales

Con todo esto, la suma de los costes directos y los costes indirectos asciende a un coste total de **11221.55€**.

7.3.4. Oferta

Antes de adjuntar la oferta, se muestra la tabla 7.3 que resume la información del proyecto.

TABLA 7.3.
INFORMACIÓN DEL PROYECTO

Título	Desarrollo de un simulador de sistemas de ficheros para entornos distribuidos y paralelos
Autor	Manuel Gómez-Plana Rodríguez
Fecha de inicio	01/11/2024
Fecha de fin	30/06/2025
Duración	8 meses
Coste total	11221.55€

A continuación, se adjunta la oferta final del proyecto en la tabla 7.4. En esta se contemplan aspectos como el beneficio, el riesgo y los impuestos.

TABLA 7.4.
OFERTA FINAL

Concepto	Incremento	Coste parcial	Coste agregado
Coste total		11221.55€	11221.55€
Riesgo	20 %	2244.31€	13465.86€
Beneficios	15 %	2019.88€	15485.74€
Impuestos	21 %	3252€	18737.74€
Total	56 %		18737.74€

8. MARCO REGULATORIO Y ENTORNO SOCIO-ECONÓMICO

Este capítulo trata sobre el marco regulatorio, es decir, los aspectos legales que afectan al proyecto; y sobre el entorno socio-económico, es decir, los ámbitos en los que podría llegar a impactar este sistema.

8.1. Marco regulatorio

Esta sección trata el proyecto desde un punto de vista legal, estudiando las leyes que se aplican a un proyecto de esta naturaleza, así como las licencias de las distintas herramientas que se han usado para la implementación del proyecto.

8.1.1. Licencias

Las herramientas que se han usado tienen una serie de licencias que dictaminan el uso que se le puede dar a cada una de ellas, algunas pudiendo incluso limitar la distribución comercial del producto desarrollado. La mayoría de herramientas son de uso libre, pero, a pesar de ello, es necesario un estudio de sus licencias. A continuación, se adjunta un listado de las herramientas usadas y sus licencias:

- **Python:** el lenguaje de programación tiene una licencia *PSFL* que permite el uso, la modificación y la distribución de software sin restricciones [59].
- **C++:** no tiene licencia al ser un estándar, no un software.
- **Inquirer:** esta biblioteca pública de *Python* cuenta con una licencia *MIT*, que permite el uso, la modificación y la distribución de software sin restricciones [60].
- **Gnuplot:** es de uso gratuito y permite la modificación gratis, pero la distribución de versiones modificadas está algo restringida [61].
- **SimGrid:** usa la licencia *LGPL*, una licencia menos restrictiva que la licencia *GPL*, y que permite el uso, modificación y distribución bajo la misma normativa que *GPL* [62], [63].

8.1.2. Leyes aplicables

Al ser un proyecto de estudio, no destinado a la comercialización de la herramienta, las leyes aplicables son reducidas. El proyecto se protege automáticamente por los derechos de propiedad intelectual y se debe cumplir con las licencias que, en este caso, no hay

problemas ya que todas permiten el uso de las herramientas de forma gratuita y sin trabas legales.

Otras leyes normalmente aplicables a este tipo de proyectos relacionadas con la ley de protección de datos personales no se aplican a este proyecto, ya que no se maneja ningún tipo de dato personal del usuario. El código de este simulador se recoge en este [repositorio](#) de acceso libre, disponible para cualquier persona que esté interesada en el proyecto.

8.2. Entorno socio-económico

El principal uso de esta herramienta es para investigar la escalabilidad de un grupo de servidores desplegados en una red. El ámbito de la simulación es, previsiblemente, el campo de estudio al que mejor se adhiere este sistema.

Este proyecto ofrece, en resumidas cuentas, una herramienta potencialmente útil con un desarrollo posible mucho más extenso para investigadores y para ingenieros de operaciones que quieran simular el consumo energético y temporal de unos servidores desplegados sin la necesidad de hacer pruebas con sistemas reales.

Adicionalmente, este simulador se puede ver como un proyecto pedagógico, que sirva para introducir a los estudiantes a los conceptos de la simulación, las plataformas distribuidas y la paralelización de los servicios. Además, es destacable su uso para concienciar sobre el consumo energético de los servidores, un aspecto de creciente interés en el panorama de la computación.

9. CONCLUSIONES Y TRABAJOS FUTUROS

Este capítulo culmina la memoria con unas conclusiones del proyecto y una sección que habla de trabajos futuros relacionados con la mejora y la publicación de la herramienta de manera oficial.

9.1. Conclusiones

Este capítulo contiene las conclusiones del proyecto, comprobando si los objetivos iniciales se han cumplido; y unas conclusiones personales que hablan ligeramente de la experiencia desarrollando el proyecto.

9.1.1. Conclusiones del proyecto

Inicialmente, se habían definido una serie de objetivos que se debían cumplir una vez se hubiese acabado con el desarrollo del simulador, presentes en el apartado [1.2. Objetivos](#). Estos objetivos hablaban de las capacidades que debía tener el usuario sobre la configuración de la simulación, así como sobre las prestaciones que debería tener el simulador, como su precisión, sus capacidades de comunicación entre los clientes y los servidores o su habilidad para generar gráficas.

A lo largo de la memoria, se ha reflejado cada uno de estos objetivos de manera indirecta. En las distintas secciones, sobre todo en el capítulo [4. Diseño](#) y en el capítulo [5. Implementación y despliegue](#), se ha expuesto la manera en la que el usuario puede y debe configurar la simulación, ejecutarla y visualizar los resultados.

Con todo el desarrollo realizado a lo largo de unos cuantos meses de trabajo, la herramienta final cumple todo lo que se había propuesto tanto en los objetivos como en los requisitos, dejando al usuario interactuar con el sistema tal y cómo se había propuesto en los casos de uso. El simulador, además, cuenta con la precisión de *SimGrid* para medir la energía y con una precisión a nivel de operación para medir el consumo temporal, si bien algunas licencias son tomadas para simplificar la complejidad del sistema final.

9.1.2. Conclusiones personales

El desarrollo de este proyecto ha sido un desafío constante, no sólo por la dificultad presentada a la hora de escribir un código correcto, si no por la complejidad del modelo de un sistema que permitiese medir con una precisión razonable el consumo de unos servidores simulados. La gran cantidad de variabilidad en las distintas políticas implementadas han causado que la comunicación cliente-servidor fuese algo más extensa de lo

que inicialmente se había planeado.

Con todo esto, el desarrollo del proyecto de final de grado ha sido un proceso divertido pero estresante, interesante pero retante, desalentador a ratos, pero motivante en otros momentos... Un proceso que dura tanto como este *TFG* y que exige tantas horas es, al final de todo, un proyecto del que uno se siente orgullo, independientemente de la calificación final.

Por último, quería agradecer a mi tutor, Félix García Carballeira, que no sólo me propuso este proyecto que me pareció interesante desde que lo escuché, si no que me ha ayudado a aclarar las dudas que tenía con *SimGrid*, que me propuso la idea de usar *Gnuplot* para generar gráficos automáticamente y que me ha ayudado con el modelado del comportamiento de los clientes y los servidores.

9.2. Trabajos futuros

Como último punto de esta memoria, se proponen una serie de trabajos futuros que se podrían hacer para mejorar este simulador y que no han sido posibles desarrollar para este *TFG*.

- **Interfaz Gráfica:** el desarrollo de una interfaz gráfica que tenga las mismas funcionalidades que las interfaces de líneas de comandos no se ha podido llevar a cabo debido a las restricciones temporales del proyecto.
- **Más configuraciones a los servidores:** se podría aumentar las configuraciones de los servidores incluyendo aspectos más refinados a los discos, como la paginación y los bloques.
- **Más políticas de acceso:** se podrían implementar aún más políticas de acceso que determinen la manera en la que se comunican los clientes con los servidores.
- **Simulación de varias entidades en el mismo host:** un proyecto interesante sería modificar el código de la simulación para permitir varios actores sobre la misma máquina host, por ejemplo, implementar servidor que tenga cuatro puertos escuchando.
- **Elementos más complejos:** como último proyecto que se plantea para el futuro de este simulador, se recomienda la implementación de elementos que permitan la descripción de una plataforma más compleja, como balanceadores de carga o buffers.

BIBLIOGRAFÍA

- [1] A. Arun Padvekar y V. Badriprasad Gupta, “Comparative Analysis of Monolithic vs. Distributed Architecture,” *International Journal of Advanced Research in Science, Communication and Technology*, vol. 4, n.º 3, pp. 437-438, jun. de 2024, Accedido el 11 de junio de 2025. [En línea]. Disponible en: <https://ijarsct.co.in/Paper18946.pdf>.
- [2] Hivenet. “Comprensión de los sistemas distribuidos: un ejemplo completo de sistemas distribuidos en acción.” Accedido el 11 de junio de 2025. (n.d.), [En línea]. Disponible en: <https://www.hivenet.com/es/post/understanding-distributed-systems-a-comprehensive-example-of-distributed-systems-in-action>.
- [3] P. Powell e I. Smalley. “What is monolithic architecture?” Accedido el 11 de junio de 2025. (2024), [En línea]. Disponible en: <https://www.ibm.com/think/topics/monolithic-architecture>.
- [4] C. Harris. “Microservices vs. monolithic architecture.” Accedido el 11 de junio de 2025. (n.d.), [En línea]. Disponible en: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>.
- [5] M. Tanner. “What Is a Monolithic Application? Everything You Need to Know.” Accedido el 11 de junio de 2025. (2024), [En línea]. Disponible en: <https://vfunction.com/blog/what-is-monolithic-application/#:~:text=Deploying%20a%20monolithic%20application%20typically,unit%2C%20making%20application%20integration%20easier..>
- [6] HP. “Computer History: A Timeline of Computer Programming Languages.” Accedido el 11 de junio de 2025. (2018), [En línea]. Disponible en: <https://www.hp.com/us-en/shop/tech-takes/computer-history-programming-languages>.
- [7] J. Richman. “What Are Distributed Architectures: 4 Types & Key Components.” Accedido el 11 de junio de 2025. (2024), [En línea]. Disponible en: <https://estuary.dev/blog/distributed-architecture/#:~:text=Distributed%20architecture%20refers%20to%20a,relaying%20on%20a%20central%20server..>
- [8] S. Surender, “RESOURCE SHARING BY COMPUTER NETWORKS,” *International Journal of Creative Research Thoughts*, vol. 10, n.º 11, pp. 398-401, nov. de 2022, Accedido el 11 de junio de 2025. [En línea]. Disponible en: <https://ijcrt.org/papers/IJCRT2211156.pdf>.
- [9] Mailrelay. “ARPANET Un pilar fundamental en la evolución de la comunicación digital y el marketing.” Accedido el 11 de junio de 2025. (n.d.), [En línea]. Disponible en: <https://mailrelay.com/es/glosario/arpanet/>.

- [10] RedHat. “¿Qué es la arquitectura orientada a los servicios (SOA)?” Accedido el 11 de junio de 2025. (2023), [En línea]. Disponible en: <https://www.redhat.com/es/topics/cloud-native-apps/what-is-service-oriented-architecture#:~:text=y%20los%20microservicios-,SOA:%20arquitectura%20orientada%20a%20los%20servicios,red%20con%20un%20lenguaje%20com%C3%BAn..>
- [11] Amazon. “¿Cuál es la diferencia entre SOA y microservicios?” Accedido el 11 de junio de 2025. (2024), [En línea]. Disponible en: <https://aws.amazon.com/es/compare/the-difference-between-soa-microservices/>.
- [12] V. Božić. “Microservices Architecture.” Accedido el 11 de junio de 2025. (2023), [En línea]. Disponible en: https://www.researchgate.net/publication/369039197_Microservices_Architecture.
- [13] Britannica. “Entrada: computer simulation.” Accedido el 12 de junio de 2025. (2025), [En línea]. Disponible en: <https://www.britannica.com/technology/computer-simulation>.
- [14] T. Robinson. “10 reasons why Microsoft Flight Simulator is a complete game changer.” Accedido el 12 de junio de 2025. (2020), [En línea]. Disponible en: <https://www.aerosociety.com/news/10-reasons-why-microsoft-flight-simulator-is-a-complete-game-changer/>.
- [15] K. Kosiuczenko, “Numerical simulation of nuclear blast wave impact on protective elements of anthropogenic structures,” *Inżynieria Bezpieczeństwa Obiektów Antropogenicznych*, pp. 12-22, mar. de 2025, Accedido el 13 de junio de 2025. [En línea]. Disponible en: [10.37105/iboa.248](https://doi.org/10.37105/iboa.248).
- [16] H. P. Schröder y L. Ciucan-Rusu, “Business Simulation,” jul. de 2012.
- [17] El Mundo de los Datos. “Distribución t de Student: Funcionamiento, propiedades y aplicaciones en estadística.” Accedido el 13 de junio de 2025. (2023), [En línea]. Disponible en: <https://elmundodelosdatos.com/distribucion-t-de-student-funcionamiento-propiedades-y-aplicaciones-en-estadistica/>.
- [18] D. Goldsman, R. Nance y J. Wilson, “A brief history of simulation,” ene. de 2010, pp. 310-313. doi: [10.1109/WSC.2009.5429341](https://doi.org/10.1109/WSC.2009.5429341).
- [19] D. Kang y B. K. Choi, “The extended activity cycle diagram and its generality,” *Simulation Modelling Practice and Theory*, vol. 19, n.º 2, pp. 785-800, 2011. doi: <https://doi.org/10.1016/j.simpat.2010.11.004>. [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S1569190X10002327>.
- [20] Avenga. “Emulator Vs. Simulator: A Guide For Software Testing.” Accedido el 12 de junio de 2025. (2025), [En línea]. Disponible en: <https://www.avenga.com/magazine/emulator-vs-simulator/>.

- [21] A. Draniceanu. “Emulators vs Simulators: A Mobile Tester’s Guide to Key Differences.” Accedido el 12 de junio de 2025. (2025), [En línea]. Disponible en: <https://www.testim.io/blog/emulator-vs-simulator-a-mobile-testers-guide-to-key-differences/#:~:text=For%20low%2Dlevel%20debugging%2C%20emulators%20are%20generally%20preferable,need%20to%20prepare%20for%20some%20performance%20trade%2Doffs..>
- [22] I. McGregor, “The relationship between simulation and emulation,” vol. 2, ene. de 2003, 1683-1688 vol.2. doi: [10.1109/WSC.2002.1166451](https://doi.org/10.1109/WSC.2002.1166451).
- [23] H. Casanova, A. Giersch, A. Legrand, M. Quinson y F. Suter, “Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms,” *Journal of Parallel and Distributed Computing*, vol. 74, n.º 10, pp. 2899-2917, 2014. doi: [10.1016/j.jpdc.2014.06.008](https://doi.org/10.1016/j.jpdc.2014.06.008). [En línea]. Disponible en: <https://hal.science/hal-01017319v2>.
- [24] SimGrid. “Simulation of Distributed Computer Systems.” Accedido el 15 de junio de 2025. (n.d.), [En línea]. Disponible en: <https://simgrid.org/>.
- [25] SimGrid. “Projects that use SimGrid.” Accedido el 15 de junio de 2025. (2025), [En línea]. Disponible en: <https://simgrid.org/usages.html>.
- [26] M. M. Tikir, M. A. Laurenzano, L. Carrington y A. Snaveley, “PSINS: An Open Source Event Tracer and Execution Simulator for MPI Applications,” en *Euro-Par 2009 Parallel Processing*, H. Sips, D. Epema y H.-X. Lin, eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 135-148.
- [27] T. Hoefler, T. Schneider y A. Lumsdaine, “LogGOPSIm: simulating large-scale applications in the LogGOPS model,” en *Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing*, ép. HPDC ’10, Chicago, Illinois: Association for Computing Machinery, 2010, pp. 597-604. doi: [10.1145/1851476.1851564](https://doi.org/10.1145/1851476.1851564). [En línea]. Disponible en: <https://doi.org/10.1145/1851476.1851564>.
- [28] G. Zheng, G. Kakulapati y L. Kale, “BigSim: a parallel simulator for performance prediction of extremely large parallel machines,” en *18th International Parallel and Distributed Processing Symposium, 2004. Proceedings.*, 2004, pp. 78-. doi: [10.1109/IPDPS.2004.1303013](https://doi.org/10.1109/IPDPS.2004.1303013).
- [29] R. Bagrodia, E. Deelman y T. Phan, “Parallel Simulation of Large-Scale Parallel Applications,” *IJHPCA*, vol. 15, pp. 3-12, feb. de 2001. doi: [10.1177/109434200101500101](https://doi.org/10.1177/109434200101500101).
- [30] P. Dickens, P. Heidelberger y D. Nicol, “Parallelized direct execution simulation of message-passing parallel programs,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 7, n.º 10, pp. 1090-1105, 1996. doi: [10.1109/71.539740](https://doi.org/10.1109/71.539740).
- [31] Amazon. “¿Qué es la computación en malla?” Accedido el 13 de junio de 2025. (2024), [En línea]. Disponible en: <https://aws.amazon.com/what-is/grid-computing/#:~:text=Grid%20computing%20is%20a%20computing,infrastructure%20and%20combine%20the%20results..>

- [32] R. Buyya y M. Murshed, “GridSim: a toolkit for the modeling and simulation of distributed resource management and scheduling for Grid computing,” *Concurrency and Computation: Practice and Experience*, vol. 14, n.º 13-15, pp. 1175-1220, 2002. doi: <https://doi.org/10.1002/cpe.710>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.710>. [En línea]. Disponible en: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.710>.
- [33] S. Ostermann, R. Prodan y T. Fahringer, “Dynamic Cloud provisioning for scientific Grid workflows,” nov. de 2010, pp. 97-104. doi: [10.1109/GRID.2010.5697953](https://doi.org/10.1109/GRID.2010.5697953).
- [34] R. Calheiros, R. Ranjan, A. Beloglazov, C. De Rose y R. Buyya, “CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms,” *Software Practice and Experience*, vol. 41, pp. 23-50, ene. de 2011. doi: [10.1002/spe.995](https://doi.org/10.1002/spe.995).
- [35] A. Núñez, J. Vázquez-Poletti, A. Caminero, J. Carretero e I. Llorente, “Design of a New Cloud Computing Simulation Platform,” jun. de 2011, pp. 582-593. doi: [10.1007/978-3-642-21931-3_45](https://doi.org/10.1007/978-3-642-21931-3_45).
- [36] *Berkeley Open Infrastructure for Network Computing*, Accedido: 13 de junio de 2025, 2013. [En línea]. Disponible en: <https://boinc.berkeley.edu/>.
- [37] M. Taufer, A. Kerstens, T. Estrada, D. Flores y P. Teller, “SimBA: A Discrete Event Simulator for Performance Prediction of Volunteer Computing Projects,” jun. de 2007, pp. 189-197. doi: [10.1109/PADS.2007.27](https://doi.org/10.1109/PADS.2007.27).
- [38] T. Estrada, M. Taufer, K. Reed y D. Anderson, “EmBOINC: An emulator for performance analysis of BOINC projects.,” ene. de 2009, pp. 1-8.
- [39] A. Montresor y M. Jelasity, “PeerSim: A scalable P2P simulator,” en *2009 IEEE Ninth International Conference on Peer-to-Peer Computing*, 2009, pp. 99-100. doi: [10.1109/P2P.2009.5284506](https://doi.org/10.1109/P2P.2009.5284506).
- [40] I. Baumgart, B. Heep y S. Krause, “OverSim: A Flexible Overlay Network Simulation Framework,” jun. de 2007, pp. 79-84. doi: [10.1109/GI.2007.4301435](https://doi.org/10.1109/GI.2007.4301435).
- [41] J. S. Bucy, G. R. Ganger et al., *The DiskSim simulation environment version 3.0 reference manual*. School of Computer Science, Carnegie Mellon University Pittsburgh, PA, USA, 2003.
- [42] “IEEE Guide for Software Requirements Specifications,” *IEEE Std 830-1984*, pp. 1-26, 1984. doi: [10.1109/IEEESTD.1984.119205](https://doi.org/10.1109/IEEESTD.1984.119205).
- [43] Lucidchart. “Qué es el lenguaje unificado de modelado (UML).” Accedido el 16 de junio de 2025. (n.d.), [En línea]. Disponible en: <https://www.lucidchart.com/pages/es/que-es-el-lenguaje-unificado-de-modelado-uml#:~:text=un%20diagrama%20UML-,%20BFQu%3%A9%20es%20UML?,en%20estructura%20como%20en%20comportamiento..>

- [44] SimGrid. “The Modern Age of Computer Systems Simulation.” Accedido el 15 de noviembre de 2024. (n.d.), [En línea]. Disponible en: <https://simgrid.org/doc/latest/>.
- [45] pandas. “pandas.” Accedido el 17 de junio de 2025. (2025), [En línea]. Disponible en: <https://pandas.pydata.org/>.
- [46] seaborn. “seaborn: statistical data visualization.” Accedido el 17 de junio de 2025. (2024), [En línea]. Disponible en: <https://seaborn.pydata.org/>.
- [47] matplotlib. “Matplotlib: Visualization with Python.” Accedido el 17 de junio de 2025. (2025), [En línea]. Disponible en: <https://matplotlib.org/>.
- [48] R. “The R Project for Statistical Computing.” Accedido el 17 de junio de 2025. (n.d.), [En línea]. Disponible en: <https://www.r-project.org/>.
- [49] MathWorks. “MATLAB.” Accedido el 17 de junio de 2025. (2025), [En línea]. Disponible en: <https://es.mathworks.com/products/matlab.html>.
- [50] F. Garcia-Carballeira, D. Camarmas-Alonso, A. Caderon-Mateos y J. Carretero, “A new Ad-Hoc parallel file system for HPC environments based on the Expand parallel file system,” en *2023 22nd International Symposium on Parallel and Distributed Computing (ISPDC)*, 2023, pp. 69-76. doi: [10.1109/ISPDC59212.2023.00015](https://doi.org/10.1109/ISPDC59212.2023.00015).
- [51] AuHau y magmax. “inquirer 3.4.0.” Accedido el 17 de junio de 2025. (2024), [En línea]. Disponible en: <https://pypi.org/project/inquirer/>.
- [52] SimGrid. “SimGrid Plugins.” Accedido el 20 de marzo de 2025. (n.d.), [En línea]. Disponible en: <https://simgrid.org/doc/latest/Plugins.html#host-energy>.
- [53] C. Busquets. “10 reglas heurísticas de Nielsen y cómo aplicarlas.” Accedido el 19 de junio de 2025. (2025), [En línea]. Disponible en: <https://www.uifrommars.com/10-reglas-heuristicas-como-aplicarlas/>.
- [54] D. K. v. Duyne, J. A. Landay y J. I. Hong, *The Design of Sites: Patterns for Creating Winning Web Sites (2nd Edition)*. USA: Prentice Hall PTR, 2006.
- [55] IOR. “Introduction.” Accedido el 28 de julio de 2025. (2018), [En línea]. Disponible en: <https://ior.readthedocs.io/en/latest/>.
- [56] S. Laoyan. “Qué es la metodología waterfall y cuándo utilizarla.” Accedido el 20 de junio de 2025. (2025), [En línea]. Disponible en: <https://asana.com/es/resources/waterfall-project-management-methodology>.
- [57] Ryte. “Modelo en Espiral.” Accedido el 20 de junio de 2025. (2021), [En línea]. Disponible en: https://es.ryte.com/wiki/Modelo_en_Espiral#:~:text=El%20desarrollo%20o%20modelo%20en,puede%20entregar%20el%20producto%20terminado..

- [58] S. Laoyan. “¿Qué es la Metodología Agile y cómo revoluciona la gestión de proyectos?” Accedido el 20 de junio de 2025. (2025), [En línea]. Disponible en: <https://asana.com/es/resources/agile-methodology>.
- [59] Python. “History and License.” Accedido el 20 de junio de 2025. (2025), [En línea]. Disponible en: <https://docs.python.org/3/license.html>.
- [60] J. García. “¿Qué es la licencia MIT?” Accedido el 20 de junio de 2025. (2024), [En línea]. Disponible en: <https://localhorse.net/article/que-es-la-licencia-mit#:~:text=%C2%BFQu%C3%A9%20es%20la%20Licencia%20de,a%20su%20simplicidad%20y%20flexibilidad..>
- [61] Gnuplot. “gnuplot FAQ.” Accedido el 20 de junio de 2025. (2021), [En línea]. Disponible en: <https://gnuplot.sourceforge.net/faq/faq.html>.
- [62] Mozilla. “GPL.” Accedido el 20 de junio de 2025. (2025), [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Glossary/GPL>.
- [63] Mozilla. “LGPL.” Accedido el 20 de junio de 2025. (2025), [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Glossary/LGPL>.

ANEXO A. USO DE IA GENERATIVA

DECLARACIÓN DE USO DE IA GENERATIVA EN EL TRABAJO DE FIN DE GRADO

He usado IA Generativa en este trabajo

Marca lo que corresponda:

SI	NO
----	----

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: reflexión sobre comportamiento ético y responsable

Ten presente que el uso de IA Generativa conlleva unos riesgos y puede generar una serie de consecuencias que afecten a la integridad moral de tu actuación con ella. Por eso, te pedimos que contestes con honestidad a las siguientes preguntas (Marca lo que corresponda):

Pregunta		
1. En mi interacción con herramientas de IA Generativa he remitido datos de carácter sensible con la debida autorización de los interesados.		
SÍ, he usado estos datos con autorización	NO, he usado estos datos sin autorización	NO, no he usado datos de carácter sensible
2. En mi interacción con herramientas de IA Generativa he remitido materiales protegidos por derechos de autor con la debida autorización de los interesados.		
SÍ, he usado estos materiales con autorización	NO, he usado estos materiales sin autorización	NO, no he usado materiales protegidos
3. En mi interacción con herramientas de IA Generativa he remitido datos de carácter personal con la debida autorización de los interesados.		
SÍ, he usado estos datos con autorización	NO, he usado estos datos sin autorización	NO, no he usado datos de carácter personal
4. Mi utilización de la herramienta de IA Generativa ha respetado sus términos de uso , así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.		
SÍ	NO	

Si **NO** has contado con la autorización de los interesados en alguna de las preguntas 1, 2 ó 3, explica brevemente el motivo (*por ejemplo, “los materiales estaban protegidos pero permitían su uso para este fin” o “los términos de uso, que se pueden encontrar en esta dirección (...), impiden el uso que he hecho, pero era imprescindible dada la naturaleza del trabajo”*).

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IA Generativa. Incluye un ejemplo por cada tipo de uso realizado donde se indique: [Añade un ejemplo].

Declaro haber hecho uso del sistema de IA Generativa ChatGPT para:

Documentación y redacción:

- Búsqueda de información o respuesta a preguntas concretas

He usado ChatGPT para resolver dudas específicas sobre la programación en Python.

Desarrollar contenido específico

Se ha hecho uso de IA Generativa como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- Asistencia en el desarrollo de líneas de código (programación)

Lo he usado para resolver dudas en funciones específicas de Python y los scripts de Gnuplot.

Parte 3: reflexión sobre utilidad

Por favor, aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IA Generativa en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

La IA me ha servido para resolver algunos errores de Python con los que había gastado ya mucho tiempo. Si bien pienso que la IA puede ser contraproducente en el aprendizaje, pienso también que optimiza procesos más tediosos de búsqueda en páginas web sobre la documentación de lenguajes. No creo que haya algo del trabajo que no pueda haber hecho sin IA, pero pienso que esta me ha ayudado a no gastar demasiado tiempo en búsquedas para resolver algunos errores concretos a los que me he enfrentado.